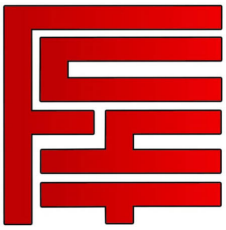


UNIVERSIDAD MAYOR DE SAN SIMÓN
FACULTAD DE CIENCIAS Y TECNOLOGÍA
CARRERA DE INGENIERÍA DE SISTEMAS



SISTEMA INTELIGENTE DE ENTRENAMIENTO EN GIMNASIOS CON REALIDAD AUMENTADA E INTELIGENCIA ARTIFICIAL

Proyecto de Grado presentado para optar al Diploma Académico de Licenciatura en Ingeniería de Sistemas

Presentado por: Erika Chino Alanoca

Tutor: Mgr. Jimmy Villarroel Novillo

Cochabamba – Bolivia

2026

Dedicatoria

A mis padres Federico Chino Salinas y Silvia Alanoca Gutierrez, quienes siempre me apoyaron y me brindaron su amor incondicional, gracias por enseñarme que con esfuerzo y dedicación se pueden alcanzar los sueños.

A mis hermanas, mi mejor amiga y a toda mi familia por su apoyo constante, por haber confiado en mí y en mis capacidades, ahora cumplo uno de mis sueños y agradezco a todos los que, de una u otra manera, fueron parte de este camino.

AGRADECIMIENTOS

Doy gracias a Dios por darme la vida, la salud y la sabiduría para llegar hasta este momento y permitirme cumplir una de las metas más importantes de mi vida.

A mi familia, gracias por su apoyo, por creer en mí y por acompañarme durante todo este camino. Muchas veces fueron mi motivación para seguir adelante, incluso cuando las cosas no fueron fáciles.

Agradezco a la Universidad Mayor de San Simón y a la Facultad de Ciencias y Tecnología por haber sido parte fundamental de mi formación y por brindarme las herramientas y conocimientos que me permitieron crecer tanto profesional como personalmente.

A mi tutor, Lic. Jimmy Villarroel Novillo, gracias por su paciencia, orientación y por compartir sus conocimientos durante el desarrollo de este proyecto.

Finalmente, gracias a mis amigos por acompañarme durante estos años, por los buenos momentos, las experiencias y todos los recuerdos que me llevo de esta etapa.

Gracias a todos los que, de una u otra manera, fueron parte de este camino.

FICHA RESUMEN

Grado:	Licenciatura en Ingeniería de Sistemas
Modalidad:	Proyecto de Grado
Facultad:	Ciencias y Tecnología
Carrera:	Ingeniería de Sistemas
Título del trabajo:	Sistema Inteligente de Entrenamiento en Gimnasios con Realidad Aumentada e Inteligencia Artificial
Autora:	Erika Chino Alanoca
Tutor:	Mgr. Jimmy Villarroel Novillo

En el primer capítulo se analiza el abandono temprano de los usuarios de gimnasios, asociado a la falta de orientación, motivación y seguimiento personalizado, y se formulan el problema, los objetivos y el alcance de un sistema de entrenamiento para Android.

En el segundo capítulo se desarrolla el marco teórico: metodologías de desarrollo de software tradicionales y ágiles (se justifica Mobile-D), programación móvil, realidad aumentada, inteligencia artificial, fundamentos del entrenamiento físico y tecnologías de desarrollo.

En el tercer capítulo se describe el área de aplicación, la adopción de la metodología Mobile-D (exploración, inicialización, producción, estabilización y pruebas), los roles y las técnicas de recolección de información.

En el cuarto capítulo se documenta el desarrollo de la aplicación CoachApp con React Native, Expo y TypeScript sobre Firebase (autenticación, base de datos, almacenamiento, Cloud Functions y App Check), con arquitectura cliente–servidor en capas Modelo–Vista–Controlador. El sistema incluye un catálogo de 35 tipos de máquinas, un generador de rutinas basado en reglas con un recomendador de TensorFlow Lite, un reconocedor de máquinas por imagen (MobileNetV2) y un detector YOLO26m publicado en Ultralytics Platform, un escáner que ancla la ficha de la máquina en el entorno con ARCore (o la superpone sobre la cámara en teléfonos sin soporte), el registro de peso, IMC y alimentación, y reportes de progreso.

En el quinto capítulo se presentan y discuten los resultados: el reconocedor acierta la máquina en el 38,5 % de las fotos (61,4 % entre las tres primeras sugerencias) en un escenario de campo simulado, por lo que el usuario siempre confirma el resultado; el recomendador reproduce las reglas con una exactitud del 99,8 % sobre datos sintéticos, lo que mide su fidelidad y no la calidad de las rutinas. El detector YOLO26m alcanza una mAP50 de 48,5 % en validación. Se declaran las limitaciones: el anclaje con ARCore no se ha validado en un teléfono físico y no sigue a la máquina si esta se mueve, y la validación con usuarios y con fotografías de gimnasios reales queda pendiente.

En el sexto capítulo se presentan las conclusiones por objetivo y las recomendaciones para trabajos futuros. El sistema no reemplaza la supervisión de entrenadores ni de nutricionistas.

ÍNDICE GENERAL

I INTRODUCCIÓN **1**

1.1 Antecedentes . . . 1
1.2 Análisis del problema . . . 2
1.2.1 Descripción del problema . . . 2
1.2.2 Definición del problema . . . 2
1.3 Objetivos . . . 2
1.3.1 Objetivo general . . . 2
1.3.2 Objetivos específicos . . . 2
1.4 Área de conocimiento . . . 3
1.5 Alcance . . . 3
1.6 Justificación . . . 3
1.6.1 Justificación técnica . . . 3
1.6.2 Justificación económica . . . 4
1.6.3 Justificación social . . . 4

II MARCO TEÓRICO **5**

2.1 Metodología de Desarrollo . . . 5
2.2 Metodologías de desarrollo de software tradicionales vs ágiles . . . 5
2.2.1 Metodologías tradicionales . . . 5
2.2.2 Metodologías ágiles para móvil . . . 5
2.3 Mobile-D . . . 6
2.3.1 Fases y etapas . . . 6
2.4 Programación Móvil . . . 7
2.5 Sistema Operativo . . . 7
2.5.1 Android . . . 7
2.5.2 Arquitectura de Android . . . 8
2.5.3 Ventajas y limitaciones . . . 8
2.6 Realidad Aumentada . . . 9
2.6.1 Componentes de un sistema de Realidad Aumentada . . . 9
2.6.2 Tecnologías para el desarrollo de Realidad aumentada móvil . . . 9
2.6.3 Alcance de la realidad aumentada en este trabajo . . . 10
2.7 Inteligencia Artificial . . . 10
2.7.1 Aprendizaje automático . . . 10
2.7.2 Redes neuronales . . . 11
2.7.3 Visión por computadora . . . 11
2.7.4 Modelos ligeros para dispositivos móviles . . . 11
2.7.5 Detección de objetos y la familia YOLO . . . 11
2.7.6 Antecedente: visión artificial para la ejecución de ejercicios . . . 12
2.7.7 Modelos de lenguaje Extenso (LLM) . . . 12
2.8 Fundamentos del entrenamiento físico . . . 12
2.8.1 Entrenamiento físico . . . 12
2.8.2 Principios del entrenamiento físico . . . 12
2.8.3 Biomecánica básica . . . 13
2.8.4 Lesiones comunes por mala técnica . . . 13
2.8.5 Máquinas de gimnasio . . . 13
2.8.6 Prescripción del entrenamiento de fuerza . . . 14
2.8.7 Indicadores nutricionales y metabólicos . . . 14
2.9 Plataforma de desarrollo . . . 14

2.9.1 Visual Studio Code . . . 14
2.10 Frameworks para desarrollo Mobile . . . 14
2.10.1 React Native . . . 15
2.11 Bases de datos de aplicaciones Móviles . . . 15
2.11.1 Firebase . . . 15
2.12 Herramientas . . . 16
2.12.1 Control de versiones . . . 16
2.12.2 Herramientas de Gestión . . . 16
III ÁREA DE APLICACIÓN 17
3.1 Entrenamiento en gimnasios . . . 17
3.1.1 Características del entrenamiento en gimnasios . . . 17
3.1.2 Objetivos de entrenamiento en gimnasios . . . 18
3.1.3 Niveles de experiencia de un usuario . . . 18
3.1.4 Tipos de entrenamiento . . . 19
3.2 Máquinas y equipos de gimnasio . . . 19
3.2.1 Características y clasificación . . . 20
3.2.2 Grupos musculares que se trabajan . . . 22
3.2.3 Ajuste y uso correcto de las máquinas . . . 30
3.3 Ejercicios y técnica de ejecución . . . 31
3.3.1 Tipos de ejercicios . . . 31
3.3.2 Postura y ejecución correcta . . . 32
3.3.3 Errores frecuentes durante los ejercicios . . . 32
3.4 Rutinas de entrenamiento . . . 33
3.4.1 Series, repeticiones, cargas y descansos . . . 33
3.4.2 Organización de las sesiones de entrenamiento . . . 33
3.4.3 Personalización según objetivos y nivel de experiencia . . . 34
3.5 Datos físicos y seguimiento del progreso . . . 34
3.5.1 Peso corporal y estatura . . . 34
3.5.2 Índice de masa corporal y sus limitaciones . . . 34
3.5.3 Registro de entrenamientos y evolución física . . . 35
3.6 Registro de la alimentación . . . 35
3.6.1 Calorías y macronutrientes . . . 36
3.6.2 Diario de alimentación . . . 36
IV DESARROLLO DEL PROYECTO 38
4.1 Fase de Exploración . . . 38
4.1.1 Establecimiento de los interesados . . . 38
4.1.2 Actores del sistema . . . 38
4.1.3 Roles y Tareas . . . 38
4.1.4 Requerimientos iniciales . . . 38
4.1.5 Definición del alcance . . . 42
4.2 Fase de Inicialización . . . 42
4.2.1 Entorno de trabajo . . . 42
4.2.2 Configuración del entorno de desarrollo . . . 42
4.2.3 Configuración para aplicaciones móviles android . . . 43
4.2.4 Planificación inicial . . . 43
4.2.5 Diseño de la arquitectura del sistema . . . 46
4.3 Fase de Producción . . . 50
4.3.1 Estructura de directorios . . . 51
4.3.2 Iteración 1: Autenticación, registro y onboarding . . . 52

4.3.3 Iteración 2: Guía de máquinas y ejercicios . . . 58
4.3.4 Iteración 3: Escáner de máquinas con superposición de información . . . 61
4.3.5 Iteración 4: Rutinas personalizadas y planificación . . . 64
4.3.6 Iteración 5: Nutrición, diario de alimentación e IMC . . . 71
4.3.7 Iteración 6: Progreso físico, entrenamientos y reportes . . . 74
4.3.8 Fase de estabilización . . . 77
4.4 Prueba final . . . 80
4.4.1 Verificación técnica . . . 81
4.4.2 Pruebas funcionales en dispositivos . . . 89
4.4.3 Pruebas funcionales con usuarios . . . 90
4.4.4 Pruebas de aceptación por objetivo . . . 90
4.4.5 Cumplimiento del alcance y desviaciones . . . 91

V RESULTADOS Y DISCUSIÓN 93
5.1 Resultados por objetivo específico . . . 93
5.2 Resultados de los modelos de inteligencia artificial . . . 94
5.2.1 Reconocimiento de máquinas . . . 94
5.2.2 Detector de máquinas en la nube . . . 94
5.2.3 Recomendador de rutinas . . . 94
5.3 Discusión . . . 95
5.3.1 Frente al problema de adherencia . . . 95
5.3.2 Frente a la visión artificial para la ejecución de ejercicios . . . 95
5.3.3 Sobre la realidad aumentada . . . 95
5.4 Amenazas a la validez y limitaciones . . . 95

VI CONCLUSIONES Y RECOMENDACIONES 97
6.1 Conclusiones . . . 97
6.2 Recomendaciones . . . 99

BIBLIOGRAFÍA 101

A CUADROS COMPARATIVOS DEL MARCO TEÓRICO 109

B REGLAS DE SEGURIDAD DE FIREBASE 114
B.1 Reglas de Cloud Firestore . . . 114
B.2 Reglas de Cloud Storage . . . 115
B.3 Funciones del servidor . . . 115

C CASOS DE PRUEBA FUNCIONALES 116

ÍNDICE DE TABLAS

4-1 Roles y tareas de Coach App (Elaboración propia) . . . 38
4-2 Requerimientos funcionales del sistema (elaboración propia) . . . 40
4-3 Requerimientos no funcionales del sistema (elaboración propia) . . . 40
4-4 Cuadro de análisis de procesos (Elaboración propia) . . . 46
4-5 Cuadro de análisis de planificación de fases . . . 46
4-6 Correspondencia entre las capas conceptuales y las capas del cliente . . . 47
4-7 Story Card 01: Inicio de sesión . . . 54
4-8 Story Card 02: Registro de usuario . . . 55
4-9 Story Card 03: Configuración inicial del perfil . . . 57
4-10 Story Card de la guía de máquinas y ejercicios . . . 58
4-11 Story Card del escáner de máquinas con superposición de información . . . 61
4-12 Story Card de rutinas personalizadas y planificación . . . 64

4-13 Reglas de frecuencia semanal según el nivel . . . 65
4-14 Reglas de variables de carga según el objetivo . . . 65
4-15 Reglas de división semanal . . . 66
4-16 Reglas de ajuste según los datos físicos . . . 66
4-17 Story Card de nutrición, diario de alimentación e IMC . . . 71
4-18 Clasificación del IMC según la OMS . . . 71
4-19 Reglas de las metas nutricionales diarias . . . 73
4-20 Story Card de progreso físico, entrenamientos y reportes . . . 74
4-21 Componentes de cada módulo por capa . . . 77
4-22 Tamaño del código fuente por capa . . . 78
4-23 Verificaciones estáticas del código . . . 79
4-24 Suites de pruebas unitarias . . . 80
4-25 Estrategia de pruebas . . . 81
4-26 Resultados del generador para las doce combinaciones de entrada . . . 82
4-27 Pruebas de valores límite del cálculo y la clasificación del IMC . . . 82
4-28 Otras pruebas de la lógica de negocio . . . 83
4-29 Escenarios de prueba de las reglas de seguridad . . . 85
4-30 Exactitud del reconocimiento de máquinas (validación cruzada, 35 clases) . . . 85
4-31 Calibración del umbral de confianza . . . 86
4-32 Métricas del detector de máquinas en la partición de prueba . . . 86
4-33 Distribución de los ejemplos anotados por clase en `gym-machines-data` . . . 87
4-34 Métricas del detector final (YOLO26m) en la partición de validación . . . 88
4-35 Características del dispositivo de prueba . . . 88
4-36 Desempeño del recomendador de rutinas sobre datos sintéticos . . . 89
4-37 Distribución de los casos de prueba funcionales . . . 90
4-38 Resultados de las pruebas de usabilidad . . . 90
4-39 Trazabilidad entre objetivos, requerimientos y pruebas . . . 91
4-40 Contraste entre lo planificado y lo implementado . . . 92

5-1 Cumplimiento de los objetivos específicos . . . 93

A-1 Comparación de Metodologías tradicionales de desarrollo de software . . . 109
A-2 Comparación de metodologías ágiles móviles . . . 110
A-3 Comparación de Android vs iOS . . . 111
A-4 Comparación de frameworks de desarrollo Mobile . . . 112
A-5 Comparación de bases de datos para aplicaciones móviles . . . 113

B-1 Funciones del servidor y controles aplicados . . . 115

C-1 Casos de prueba funcionales . . . 116

ÍNDICE DE FIGURAS

2-1 Fases y etapas de la metodología Mobile-D . . . 6
2-2 Arquitectura de Android . . . 8

3-1 Ejemplos de máquinas y equipos de gimnasio . . . 21
3-2 Músculos del pecho, la espalda y el hombro . . . 23
3-3 Músculos del brazo y del antebrazo . . . 24
3-4 Músculos de la cadera y del muslo . . . 26
3-5 Músculos de la pierna y de la pantorrilla . . . 27

3-6 Músculos superficiales y profundos de la pared abdominal 29

4-1 Diagrama de casos de uso del sistema 41
4-2 Arquitectura cliente–servidor y capas del cliente 47
4-3 Modelo de datos en Cloud Firestore 48
4-4 Esquema de navegación de usuario 50
4-5 Diagrama de secuencia del inicio de sesión 53
4-6 Pantallas de inicio de sesión con correo y contraseña o Google 54
4-7 Pantalla de registro para usuarios nuevos 56
4-8 Pantallas del *onboarding* 57
4-9 Proceso de construcción y carga del catálogo de máquinas 59
4-10 Guía de máquinas 60
4-11 Diagrama de estados del escáner de máquinas 62
4-12 Flujo de reconocimiento y anclaje espacial de la ficha 63
4-13 Escáner de máquinas con superposición de información 64
4-14 Flujo del generador de rutinas basado en reglas 67
4-15 Selección de la estructura de la rutina con respaldo por reglas 68
4-16 Módulo de rutinas 69
4-17 Creación de una rutina personalizada 70
4-18 Flujo de registro de un alimento 72
4-19 Módulo de nutrición 73
4-20 Reporte de progreso físico en PDF generado en el dispositivo de prueba 75
4-21 Módulo de progreso 76
4-22 Pantalla de inicio 76

CAPÍTULO I: INTRODUCCIÓN

En la actualidad, la práctica del ejercicio físico en gimnasios ha cobrado gran relevancia como parte de un estilo de vida saludable. Sin embargo, la evidencia indica que menos del 40 % de los socios de gimnasios logra mantener un ejercicio regular a largo plazo (Gjestvang et al., 2021), lo que convierte la adherencia en un problema relevante. Esta problemática puede afectar especialmente a los principiantes, quienes pueden enfrentar barreras como el desconocimiento del uso correcto de las máquinas, el alto costo de los entrenadores personales y la ausencia de planificación estructurada.

En respuesta a esta necesidad, se han desarrollado diversas soluciones tecnológicas que buscan mejorar la experiencia del usuario en el gimnasio, como aplicaciones móviles con realidad aumentada (Cruz, 2024) y sistemas de información para rutinas y dietas (Fernández, 2018). No obstante, muchas de estas herramientas carecen de personalización profunda y seguimiento inteligente.

Este proyecto propone el desarrollo de una aplicación móvil para Android que integra inteligencia artificial y realidad aumentada, entendida como la superposición de información digital sobre la imagen en vivo de la cámara que, en los teléfonos compatibles con ARCore, se ancla además a un punto del entorno (sección 4.1.5). La aplicación se orienta a guiar a los usuarios en el uso correcto de máquinas de ejercicio, planificar rutinas personalizadas y realizar un seguimiento del progreso físico. La solución busca ampliar el acceso a una orientación básica y estructurada, sin sustituir al profesional, y fomentar la constancia en el entrenamiento físico, utilizando tecnologías modernas como React Native, Firebase, TensorFlow Lite, ARCore y modelos de detección de la familia YOLO.

Con esto, se pretende contribuir no solo al bienestar individual, sino también a la optimización de recursos en gimnasios, ofreciendo una herramienta accesible, interactiva y orientada a la autogestión del entrenamiento.

1.1. Antecedentes

Yépez (2017) presentó un proyecto de grado cuyo objetivo principal es la creación de un gimnasio que combine asesoramiento presencial y virtual, orientado a la disciplina llamada Street Workout, que consiste en utilizar el propio peso corporal para ejercitarse.

Gjestvang et al. (2020) realizaron un estudio longitudinal de un año, con nuevos socios de centros de fitness en Noruega, para identificar los motivos y las barreras en la iniciación y la adherencia al ejercicio en gimnasios. Investigaciones del mismo grupo señalan que menos del 40 % de los socios mantiene un ejercicio regular a largo plazo, aunque pocos cancelan su membresía (13,4 % a los 12 meses) (Gjestvang et al., 2021). Estos datos describen un problema de adherencia más que de abandono formal, y justifican explorar herramientas tecnológicas que apoyen la constancia en programas de ejercicio estructurados.

Cruz (2024) desarrolló, como proyecto de grado en la Universidad Mayor de San Simón, una aplicación móvil para gimnasios que utiliza escaneo de código QR con realidad aumentada para mejorar la experiencia de los usuarios y la gestión de entrenamientos, y que permite obtener instrucciones detalladas en tiempo real sobre los ejercicios.

Fernández (2018), en su proyecto de grado, desarrolló un sistema de información cuyo objetivo principal es brindar apoyo a los usuarios o clientes de los gimnasios, proporcionándoles rutinas de ejercicios y dietas, como guía sobre cómo usar las máquinas y administrar sus alimentos para alcanzar el objetivo deseado.

BodBot, creada por Laux et al. (2016), es una plataforma de fitness que ofrece entrenamientos con inteligencia artificial personalizados según los objetivos, equipos, capacidades físicas y dificultad del usuario; proporciona una guía paso a paso para cada ejercicio, personalización por peso corporal y registro diario de alimentación.

Ejercicios en el Gimnasio, desarrollada por Leap Fitness Group (2021), es una aplicación para principiantes y asiduos al gimnasio que funciona como un entrenador personalizado: crea rutinas, muestra la postura correcta de los ejercicios mediante instrucciones visuales y escritas, y registra el progreso con estadísticas y gráficos.

1.2. Análisis del problema

1.2.1. Descripción del problema

Este proyecto surge para atender barreras que los usuarios de gimnasios pueden enfrentar, particularmente la falta de orientación sobre el uso de las máquinas y la falta de estructura en sus entrenamientos. La evidencia disponible describe una adherencia baja a largo plazo: menos del 40 % de los socios mantiene un ejercicio regular (Gjestvang et al., 2021). Los problemas que el proyecto se propone atender son:

- **Falta de conocimiento sobre el uso adecuado de las máquinas y la postura correcta:** Esto puede llevar a errores que causen lesiones y dificulten el progreso hacia los objetivos físicos.
- **Costo elevado de los entrenadores personales:** Muchas personas no pueden acceder a una guía profesional debido al costo asociado, lo cual puede dejar a muchos usuarios sin un apoyo adecuado.
- **Falta de seguimiento del progreso:** La falta de un sistema para registrar y analizar el progreso personal puede desmotivar a los usuarios, ya que no tienen una forma clara de visualizar sus avances.
- **Carencia de planificación estructurada del entrenamiento:** Sin una rutina clara y bien planificada, muchos usuarios pueden sentirse perdidos y no aprovechar el tiempo de entrenamiento, lo cual puede fomentar la desmotivación.

Estas barreras se plantean como hipótesis de trabajo de la autora; el efecto de la aplicación sobre la adherencia no fue evaluado en este proyecto (sección 5.4).

1.2.2. Definición del problema

¿Cómo desarrollar una aplicación para gimnasios que ofrezca seguimiento y rutinas de ejercicio personalizadas con inteligencia artificial?

1.3. Objetivos

1.3.1. Objetivo general

Desarrollar un sistema inteligente para el entrenamiento en gimnasios, proporcionando una guía estructurada y seguimiento personalizado de los ejercicios.

1.3.2. Objetivos específicos

- Guiar sobre el uso correcto de máquinas.
- Instruir en la ejecución correcta de ejercicios específicos.
- Diseñar rutinas de entrenamiento personalizadas y estructuradas, adaptadas a los objetivos de cada usuario.

- Construir herramientas de planificación que permitan a los usuarios organizar sus entrenamientos de manera eficiente.
- Registrar la ingesta de alimentos con el fin de controlar y evaluar las calorías consumidas por el usuario.
- Implementar un sistema de seguimiento del progreso físico.
- Generar reportes personalizados de progreso físico.

1.4. Área de conocimiento

Área: Programación Móvil

Subárea: Sistema de Información

1.5. Alcance

- El sistema funcionará en sistemas operativos Android.
- Interfaz intuitiva para el registro de usuarios.
- Guía sobre el uso correcto de máquinas de gimnasio.
- Registrar el progreso físico (peso, estatura, edad, masa corporal).
- Creación de rutinas personalizadas según los objetivos del usuario.
- El registro de alimentación comprende un diario con conteo calórico y de macronutrientes por porción, apoyado en USDA FoodData Central (U.S. Department of Agriculture, Agricultural Research Service [USDA ARS], s. f.), y una calculadora de IMC; no constituye asesoría nutricional.
- La realidad aumentada superpone la ficha de la máquina sobre la imagen de la cámara y, en los teléfonos compatibles con ARCore, la ancla a un punto del entorno; el ancla no sigue a la máquina si esta se mueve y el reconocimiento ocurre por captura, no de forma continua.
- El reconocimiento de la máquina por imagen es una sugerencia: solo se acepta automáticamente con una confianza de al menos 0,8 y, en los demás casos, el usuario la confirma.
- El sistema no verifica automáticamente la postura del usuario durante la ejecución de los ejercicios.
- El sistema no reemplaza la supervisión profesional de entrenadores o nutricionistas.
- No se contempla la integración con dispositivos externos (smartwatch, banda de frecuencia cardíaca, entre otros).
- El sistema funcionará con conexión a internet.
- La aplicación será para dispositivos móviles Android.
- El sistema estará disponible únicamente en idioma español.
- La aplicación requerirá permisos de acceso a la cámara y almacenamiento del dispositivo para el registro de progresos y la visualización de ejercicios.

1.6. Justificación

1.6.1. Justificación técnica

El proyecto es técnicamente viable gracias a la combinación de tecnologías modernas y flexibles. Se empleó React Native con Expo y TypeScript para desarrollar una aplicación Android con una sola base de código, que permite una futura extensión a otras plataformas.

La autenticación, la base de datos y el almacenamiento se gestionaron con Firebase, lo que aporta escalabilidad, sincronización en tiempo real y reglas de seguridad por usuario. Las consultas a servicios externos se canalizan mediante Cloud Functions protegidas con App Check, de modo

que las claves de las interfaces externas no se incluyen en la aplicación.

El componente de inteligencia artificial comprende dos modelos de TensorFlow Lite que se ejecutan en el dispositivo: un recomendador de rutinas, que aprende las reglas de prescripción del entrenamiento, y un clasificador de máquinas por imagen. Ambos cuentan con respaldo por reglas o confirmación del usuario cuando su confianza es baja. Como alternativa opcional, un detector de la familia YOLO publicado en Ultralytics Platform reconoce la máquina y devuelve su posición en la imagen; con esa posición, un módulo propio basado en ARCore fija la información de la máquina en el entorno real.

Además, se integró una guía visual con imágenes y animaciones de ejecución que muestran cómo realizar cada ejercicio, lo cual favorece la comprensión de la técnica.

En conjunto, estas tecnologías permiten un sistema funcional y con potencial de expansión futura.

1.6.2. Justificación económica

La aplicación representa una alternativa de bajo costo frente a los servicios tradicionales de entrenamiento personalizado. Al automatizar la generación de rutinas, ofrece orientación básica y estructurada sin los costos de contratar un entrenador, aunque no sustituye la supervisión profesional.

Para los gimnasios, la herramienta puede convertirse en un complemento que optimice el tiempo de los entrenadores y facilite la autogestión del entrenamiento de los usuarios. Asimismo, el desarrollo se apoya en tecnologías de código abierto (React Native, Expo, TensorFlow Lite) y en servicios en la nube cuyo costo depende del uso (Firebase, Ultralytics Platform), lo que reduce los costos iniciales de desarrollo y mantenimiento.

1.6.3. Justificación social

Este proyecto busca reducir las barreras que limitan la práctica constante del ejercicio físico, ofreciendo orientación y seguimiento a través de la tecnología. Está dirigido principalmente a usuarios principiantes o personas que no tienen acceso a un entrenador personal. Contribuye a la promoción de un estilo de vida saludable, apoya la constancia y ayuda a reducir el riesgo de lesiones al explicar el uso correcto de las máquinas.

CAPÍTULO II: MARCO TEÓRICO

2.1. Metodología de Desarrollo

Una metodología de desarrollo de software es un marco de trabajo que se usa para estructurar, planificar y controlar el proceso de desarrollo de sistemas de información (Gabriel Maide & Pacienzia, 2020). Asimismo el objetivo de las distintas metodologías es el intentar organizar los equipos de trabajo para que estos desarrollen las funciones de un programa de la mejor manera posible (Santander Open Academy, 2020). Una metodología establece un conjunto de prácticas, técnicas y procesos que permiten mejorar la eficiencia del trabajo, garantizar la calidad y reducir la incertidumbre durante el ciclo de vida del software (Pressman, 2010).

Las metodologías de desarrollo proporcionan lineamientos que facilitan la coordinación del equipo, la definición clara de los requerimientos, la gestión de riesgos y la evaluación del avance del proyecto. El uso de la metodología adecuada contribuye a la minimización de riesgos, estandarización de procedimientos y mejorar la productividad.

Las metodologías de desarrollo de software son enfoques que proporcionan un marco claro para planificar, ejecutar y entregar proyectos de software, estas pueden ser lineales o iterativas, se ajustan a diferentes tipos de proyectos y equipos de acuerdo a las necesidades específicas de cada proyecto o equipo (Abstracta Team, 2024).

2.2. Metodologías de desarrollo de software tradicionales vs ágiles

Las metodologías de desarrollo de software se agrupan principalmente en tradicionales y ágiles, cada una con enfoques y características distintas.

2.2.1. Metodologías tradicionales

Las metodologías tradicionales, conocidas también como predictivas, son enfoques secuenciales y lineales, basados en una planificación detallada y estructurada, donde cada fase debe completarse antes de iniciar la siguiente, tampoco se puede volver hacia atrás una vez se ha cambiado de fase. Estas metodologías, no se adaptan nada bien a los cambios, y el mundo actual cambia constantemente (Santander Open Academy, 2020).

Este enfoque es ideal para proyectos en los que los requerimientos están bien definidos desde el principio.

La comparación detallada de las metodologías Waterfall, Espiral, Incremental, Prototipado y Diseño rápido de aplicaciones (RAD), con su descripción y características, se presenta en la Tabla A-1 del Anexo A.

2.2.2. Metodologías ágiles para móvil

Las metodologías ágiles aplicadas al desarrollo móvil surgen como respuesta a la necesidad de adaptarse a un entorno cambiante y a ciclos de producción rápidos. Estas metodologías están centradas en la entrega continua de valor, la colaboración constante con el cliente y la capacidad de adaptarse a nuevos requerimientos.

Las metodologías ágiles son las más utilizadas en la actualidad debido a su alta flexibilidad y agilidad (Santander Open Academy, 2020). Se basan en la metodología incremental, en la que en cada ciclo de desarrollo se agregan nuevas funcionalidades a la aplicación final.

La comparación de las metodologías Scrum, Extreme Programming (XP), Mobile-D y Kanban, con

su descripción y características, se presenta en la Tabla A-2 del Anexo A.

2.3. Mobile-D

La metodología Mobile-D es un enfoque ágil diseñado para el desarrollo de software en dispositivos móviles. Surgió como resultado de investigaciones realizadas por el VTT, con el objetivo de responder a los desafíos propios del entorno móvil, como ciclos de producción acelerados, restricciones de hardware, diversidad de plataformas y cambios frecuentes en los requisitos (Abrahamsson et al., 2004).

Mobile-D es adecuada para proyectos que requieren rapidez, flexibilidad y alta calidad. Está pensada para equipos pequeños y para proyectos de corta duración (el artículo original menciona entregas en menos de diez semanas; Abrahamsson et al., 2004), por lo que su aplicación a este proyecto, individual y de mayor duración, constituye una adaptación. La metodología combina prácticas de diversas metodologías ágiles y de ingeniería de software.

2.3.1. Fases y etapas

El desarrollo mediante el enfoque de Mobile-D se compone de 5 fases como se muestra en la Figura 2-1, estas fases son: Exploración, Inicialización, Producción, Estabilización y Pruebas del sistema. En cada iteración, el método organiza el trabajo en días de planificación, de trabajo y de entrega (Abrahamsson et al., 2004).

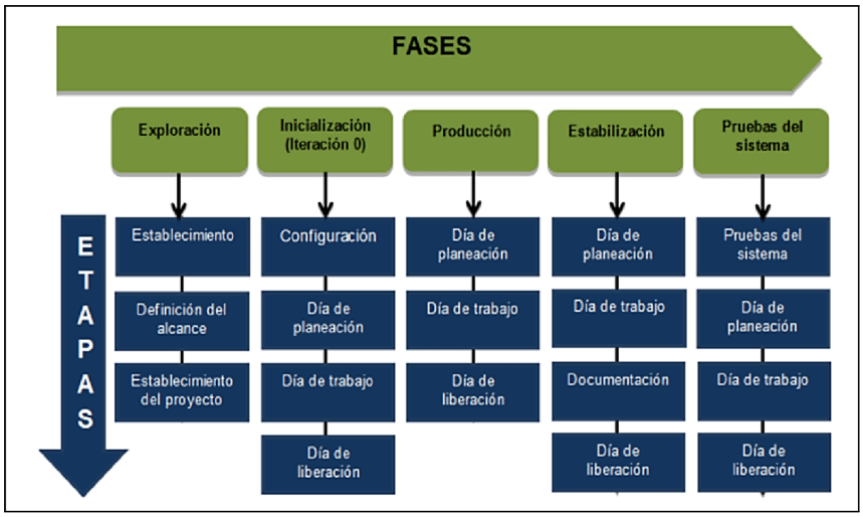


Figura 2-1. Fases y etapas de la metodología Mobile-D

Fuente: Leyva et al. (2016).

- **Fase de Exploración**
  Esta es la fase inicial donde se define el proyecto y se establecen las bases técnicas y organizativas. En esta fase se definen los objetivos, el alcance, los actores involucrados y las restricciones técnicas y funcionales del proyecto. También se seleccionan las tecnologías principales y se configura el entorno de trabajo.
- **Fase de Inicialización**
  En esta fase se prepara el equipo, las herramientas y la arquitectura técnica para comenzar las iteraciones de desarrollo. Dentro de esta fase se verifican los recursos humanos, tecnológicos y físicos que se necesitarán. Esta fase se divide en cuatro etapas: la puesta en marcha del proyecto, la planificación inicial, el día de prueba y día de salida (Gómez & Hernández, 2016).

- **Fase de Producción**
  Esta fase se caracteriza por el uso de ciclos cortos e incrementales. En esta etapa se implementan funcionalidades mediante prácticas de desarrollo guiado en pruebas TDD, la integración continua y el desarrollo iterativo. Su objetivo principal es construir versiones funcionales del producto de manera progresiva, garantizando la calidad y retroalimentación constante durante todo el proceso.
- **Fase de Estabilización**
  En esta fase se integran los componentes generados durante la producción para asegurar que el sistema funcione correctamente. Esta etapa los desarrolladores optimizan el rendimiento, mejoran la consistencia interna del software y realizan las refactorizaciones necesarias. No se añaden nuevas funcionalidades, sino que se fortalece la calidad general del producto.
- **Fase de Pruebas**
  En esta fase se verifica que la aplicación cumpla con los requerimientos funcionales y de calidad definidos al inicio del proyecto. Esta etapa incluye pruebas de aceptación, pruebas de dispositivos reales y corrección de errores detectados. El propósito de esta etapa es garantizar que el sistema sea estable, usable y se encuentre listo para ser entregado al cliente o desplegado en un entorno final.

2.4. Programación Móvil

La programación móvil consiste en diseñar, implementar, probar y desplegar software que se ejecuta en dispositivos móviles. Este tipo de desarrollo de software tiene como objetivo crear aplicaciones que se ejecuten en varios dispositivos móviles, incluidos Android, iOS y otras plataformas menos populares. La plataforma móvil abarca dos enfoques principales de desarrollo nativo y multiplataforma (AppMaster, 2023).

Una aplicación móvil se define como un software diseñado para ejecutarse en dispositivos portátiles como teléfonos inteligentes y tabletas, con el fin de realizar tareas específicas para el usuario (Apps, 2011).

En el ámbito de desarrollo de software, los sistemas operativos móviles dominantes son Android y iOS, los cuales concentran prácticamente la totalidad del mercado global de smartphones. Android, mantenido por Google, es un sistema abierto basado en Linux que ofrece gran flexibilidad, soporte para múltiples fabricantes y acceso a herramientas de desarrollo como Android Studio y las bibliotecas Jetpack (Google, 2023). Por su parte, iOS desarrollado por Apple, se caracteriza por ser un sistema operativo cerrado, de alto control de calidad, diseñado exclusivamente para ejecutarse en el hardware de Apple y herramientas como Xcode y SwiftUI que permiten crear aplicaciones con elevado rendimiento y experiencia de usuario (Apple Inc., 2023).

Las diferencias entre ambos sistemas operativos (propietario, licencia, lenguajes, entorno de desarrollo, arquitectura, presencia en el mercado boliviano, personalización, fragmentación y distribución de aplicaciones) se resumen en la Tabla A-3 del Anexo A.

2.5. Sistema Operativo

2.5.1. Android

Android es un sistema operativo y una plataforma de desarrollo libre basada en Linux y de código abierto desarrollado por Google (Tomás, 2019). Este sistema operativo es utilizado en tablets, netbooks, reproductores de música e incluso, PC's (Sanz, 2019). Hoy en día podemos encontrar relojes, gafas, cámaras, TV, sistema para automóviles, electrodomésticos y una gran variedad de sistemas empotrados que se basan en este sistema operativo por lo que lo hace adaptable a

cualquier tipo de hardware (Tomás, 2019).

Su dominio en el mercado global, superando el 70 % de cuota del mercado, y su prevalencia superior al 90 % en el contexto boliviano, lo posicionan como el estándar en el desarrollo Mobile en la región (StatCounter Global Stats, 2025). El desarrollo Android ha ido evolucionando puesto que permite que fabricantes, desarrolladores y la comunidad global adapten el sistema, añadiendo funciones nuevas y personalicen la interfaz, lo que facilita la creación de aplicaciones innovadoras y la configuración del sistema a distintos entornos y necesidades (Open Handset Alliance, 2022).

2.5.2. Arquitectura de Android

Desde una perspectiva técnica, Android se compone de cuatro capas principales. En la base se encuentra el kernel de Linux, responsable de gestionar procesos, la administración de memoria y la seguridad del sistema. Sobre él se ubica la capa del Hardware Abstraction Layer (HAL), que permite que diferentes fabricantes integren su propio hardware. La tercera capa de las APIs de Android, que ofrecen servicios como gráficos, multimedia, ubicación y sensores. Finalmente, el framework de aplicaciones, que facilita la creación de aplicaciones mediante bibliotecas de alto nivel y componentes como Activities, Services y Content Providers (Meier & Lake, 2018).

El siguiente gráfico muestra la arquitectura de Android. Como se puede observar en la Figura 2-2, su arquitectura se encuentra formada por cuatro capas.

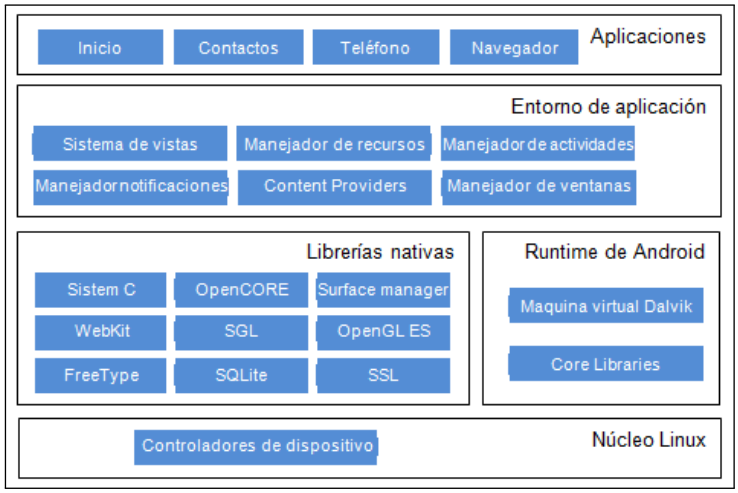


Figura 2-2. Arquitectura de Android

Fuente: Gómez (2015).

2.5.3. Ventajas y limitaciones

Android destaca por que ofrece ventajas como:

- Flexibilidad para funcionar en múltiples tipos de hardware.
- Libertad de personalización de interfaz.
- Libertad de configuración del sistema.
- Compatibilidad amplia con una gran variedad de herramientas y servicios de Google.
- Alta calidad de gráficos y sonido.
- Portabilidad garantizada, ya que sus aplicaciones son desarrolladas en Java.
- Nivel de seguridad aceptable.

Sin embargo, esto también genera limitaciones, como la fragmentación entre dispositivos y dife-

rentes velocidades en la distribución de actualizaciones del sistema.

2.6. Realidad Aumentada

La realidad aumentada (RA) es una combinación visual de elementos reales y virtuales que interaccionan entre ellos (Navarro et al., 2018), también es la integración en tiempo real de la información digital en el entorno de un usuario. La tecnología de RA superpone el contenido al mundo real, lo que enriquece la percepción de la realidad del usuario en lugar de reemplazarla (Hayes & Downie, 2025). A diferencia de la realidad virtual, que sustituye completamente el entorno físico, la RA superpone información digital como imágenes, modelos 3D, texto o animaciones sobre la realidad observada (Azuma, 1997), y hoy se despliega en dispositivos como smartphones, tablets o lentes inteligentes (Hayes & Downie, 2025).

2.6.1. Componentes de un sistema de Realidad Aumentada

Un sistema de Realidad Aumentada es una convergencia de hardware y software que opera un bucle continuo de retroalimentación, en otras palabras, un sistema de RA debe cumplir con tres características principales, combinar lo real y lo virtual, operar interactivamente y registrar elementos en 3D (Azuma, 1997).

Un sistema de RA incluye los siguientes componentes esenciales:

- **Dispositivo de visualización:** Permite mostrar la mezcla entre la imagen real y los elementos virtuales. Puede ser un teléfono móvil, tablet, gafas inteligentes o pantalla montada en la cabeza (Head-Mounted Display).
- **Cámara o sensores de captura:** Registra el entorno físico para detectar superficies, marcadores o puntos de referencia. Por ejemplo: cámaras RGB, sensores de profundidad, giroscopio, acelerómetro.
- **Módulo de seguimiento (tracking):** Se encarga de identificar la posición y orientación del usuario o del dispositivo. Esto incluye: marker-based, markerless, SLAM, reconocimiento de superficies.
- **Motor de renderizado:** Genera y superpone objetos virtuales en el entorno real con coherencia visual y espacial.
- **Interfaz de interacción:** Permite al usuario manipular objetos virtuales mediante gestos, toques o movimiento.

2.6.2. Tecnologías para el desarrollo de Realidad aumentada móvil

Para el desarrollo móvil, las principales tecnologías de RA son:

- **ARCore:** Es el SDK de realidad aumentada de Google que ofrece APIs multiplataforma para crear nuevas experiencias envolventes en Android, iOS, Unity y la Web. Transforma la manera en que las personas juegan, compran, aprenden, crean y experimentan el mundo juntas mediante la comprensión contextual de las personas, los lugares y las cosas (Google for Developers, 2025).
- **ARKit:** Es el framework de realidad aumentada de Apple, que combina la cámara y los sensores del dispositivo, entre ellos LiDAR en los modelos compatibles, para anclar contenido virtual al entorno (Apple Inc., 2025).

Para el desarrollo de este trabajo interesa sobre todo cómo ARCore resuelve el registro tridimensional. Su seguimiento de movimiento se basa en una técnica de localización y mapeo simultáneos (SLAM): detecta puntos de interés en la imagen de la cámara y los combina con los datos iner-

ciales del dispositivo para estimar su posición y orientación a lo largo del tiempo; además agrupa esos puntos para reconocer superficies planas, horizontales o verticales (Google for Developers, s. f.-e). Sobre ese conocimiento del entorno se pueden crear *anclas*, es decir, poses fijas respecto del mundo real: a medida que ARCore refina su estimación del espacio, el ancla se corrige y el contenido asociado a ella permanece en su lugar en vez de ir a la deriva (Google for Developers, s. f.-f). Cuando el dispositivo lo permite, ARCore también estima la profundidad de la escena comparando imágenes tomadas desde ángulos distintos mientras la persona mueve el teléfono, con apoyo de aprendizaje automático y, si existe, de un sensor de tiempo de vuelo; esa información hace que una prueba de impacto (*hit test*) funcione incluso sobre superficies sin textura (Google for Developers, s. f.-b), una técnica que cuenta con descripción en la literatura científica (Valentin et al., 2018). Por último, una aplicación puede declarar ARCore como requerido o como opcional; en el segundo caso sigue funcionando en teléfonos sin soporte, y las funciones de realidad aumentada exigen Android 7.0 (API 24) o superior (Google for Developers, s. f.-c).

2.6.3. Alcance de la realidad aumentada en este trabajo

Un sistema de RA completo exige cumplir las tres características descritas por Azuma (1997): combinar lo real y lo virtual, ser interactivo en tiempo real y estar registrado en tres dimensiones. La aplicación las aborda en dos niveles. En los teléfonos Android compatibles con ARCore, el escáner usa un módulo nativo propio (`CoachAR`) que, una vez reconocida la máquina, ancla su nombre e instrucciones a un punto del entorno real y los mantiene allí mientras la persona se mueve; esa es la forma de RA con registro tridimensional (sección 4.3.4). En los demás dispositivos, y cuando el usuario lo prefiere, el escáner recurre a una cámara convencional con la ficha superpuesta en pantalla, que satisface la combinación de lo real y lo virtual y, solo en parte, la interacción en tiempo real.

Conviene tener presentes dos límites. El primero es que el ancla se fija a un punto de una superficie estimado a partir de la caja delimitadora de la máquina, no a la geometría de la máquina, y por eso no la sigue si se desplaza. El segundo es que el reconocimiento ocurre por captura y no de forma continua sobre el video. En esta memoria el término «realidad aumentada» se usa con esos matices (sección 4.1.5).

2.7. Inteligencia Artificial

La inteligencia artificial (IA) es una tecnología transformadora que permite a las máquinas llevar a cabo tareas de resolución de problemas similares a las de los humanos. Desde el reconocimiento de imágenes y la generación de contenido creativo hasta la creación de predicciones basadas en datos, la IA permite a las empresas tomar decisiones más inteligentes a escala (Amazon Web Services, 2025).

La IA consiste en enseñar a las computadoras a hacer las actividades increíbles que nuestros cerebros pueden hacer, desde comprender el mundo que las rodea hasta aprender temas nuevos y generar ideas originales (Google Cloud, 2025). En el contexto de las aplicaciones móviles, la IA permite integrar funciones avanzadas como la personalización del contenido, la detección automática de movimiento, el análisis de rendimiento del usuario y la interacción mediante voz a texto.

2.7.1. Aprendizaje automático

El aprendizaje automático o Machine Learning es una subárea de la inteligencia artificial dedicado a comprender y construir métodos para imitar la forma en que aprenden los humanos (Pykes, 2024). El aprendizaje automático, es la capacidad que tiene un programa para mejorar su desempeño en una tarea a partir de la experiencia acumulada (Mitchell, 1997).

2.7.2. Redes neuronales

Una red neuronal es un programa o modelo de machine learning que toma decisiones de forma similar al cerebro humano, al emplear procesos que imitan la forma en que las neuronas biológicas trabajan juntas para identificar fenómenos, sopesar opciones y llegar a conclusiones. Se basan en datos de entrenamiento para aprender y mejorar su precisión con el tiempo (IBM, 2025).

2.7.3. Visión por computadora

La visión artificial es un campo de la inteligencia artificial, que permite a los ordenadores y sistemas interpretar y analizar datos visuales, así como también obtener información pertinente a partir de imágenes digitales, videos y otros datos visuales (Google Cloud, 2025). En el contexto de las aplicaciones móviles modernas, permite interpretar información visual mediante algoritmos que detectan objetos, poses, personas entre otras características del entorno.

2.7.4. Modelos ligeros para dispositivos móviles

Los clasificadores de imágenes diseñados para dispositivos móviles equilibran precisión y consumo de recursos. MobileNetV2 (Sandler et al., 2018) es una arquitectura de este tipo, basada en bloques residuales invertidos y cuellos de botella lineales. Una práctica habitual consiste en reutilizar una red preentrenada como extractora de características y entrenar solo una capa final para el problema propio, lo que permite trabajar con pocos datos; los modelos resultantes pueden ejecutarse en el dispositivo con TensorFlow Lite, sin enviar imágenes a un servidor.

2.7.5. Detección de objetos y la familia YOLO

Un clasificador responde qué hay en una imagen; un detector de objetos responde además dónde, pues devuelve para cada objeto una caja delimitadora, su clase y un grado de confianza. Esa localización es la que permite, en este proyecto, colocar información junto a la máquina en lugar de limitarse a nombrarla. YOLO (*You Only Look Once*) planteó la detección como un único problema de regresión resuelto por una red que mira la imagen completa una sola vez, lo bastante rápida para el tiempo real (Redmon et al., 2016). Desde entonces la familia ha crecido con versiones sucesivas que mejoran la arquitectura y el entrenamiento (Terven et al., 2023).

Las versiones recientes se distribuyen mediante la biblioteca Ultralytics, que reúne en un mismo paquete el entrenamiento, la validación y la exportación a varios formatos de despliegue (Jocher et al., 2023). Su código y sus modelos se publican con licencia AGPL-3.0 o con una licencia empresarial (Ultralytics, s. f.-c), condición que debe considerarse si el producto se distribuye fuera del ámbito académico (sección 6.2). YOLO11, la generación con la que se entrenó una primera versión del detector de este proyecto, incorpora los bloques C3k2 y un módulo de atención (C2PSA) para extraer mejor las características, y se ofrece en tamaños desde nano hasta extragrande, de modo que sirve tanto para dispositivos con pocos recursos como para servidores (Khanam & Hussain, 2024). YOLO26, publicada después, elimina la supresión de no máximos (NMS) durante la inferencia mediante un diseño de doble cabeza, suprime la pérdida focal de distribución (DFL) para simplificar la cabeza de detección y la exportación, y se entrena con un optimizador híbrido llamado MuSGD (Jocher et al., 2026; Sapkota et al., 2025). Sus autores reportan una inferencia en CPU hasta un 43 % más rápida que la de YOLO11 en el tamaño nano (Ultralytics, s. f.-c). La versión final del detector de este trabajo usa YOLO26 en su tamaño mediano (m). Se trata de preimpresiones y de documentación del fabricante, aún sin revisión por pares, de modo que esas cifras se toman como referencia y no como un resultado propio.

Un detector se evalúa con métricas distintas a las de un clasificador. La intersección sobre la unión (IoU) mide cuánto se solapan la caja predicha y la real, y una detección se considera correcta si su-

pera un umbral de IoU. La precisión es la proporción de detecciones correctas entre las emitidas; la exhaustividad (*recall*), la proporción de objetos reales que se encontraron; y la precisión promedio (AP) resume el compromiso entre ambas como el área bajo la curva precisión–exhaustividad. Su promedio entre clases es la mAP (Padilla et al., 2021). Es habitual reportar la mAP50 (IoU de al menos 0,5) y la mAP50–95 (promedio entre umbrales de 0,5 a 0,95), esta última más exigente con el ajuste de la caja.

Un modelo así puede ejecutarse en el propio teléfono o en un servidor. Ultralytics Platform permite publicarlo como *endpoint* dedicado: un servicio con dirección propia, protegido con una clave, que recibe una imagen y devuelve las detecciones. En su configuración gratuita escala a cero cuando no se usa, por lo que la primera solicitud tras un período inactivo puede tardar hasta alrededor de un minuto (Ultralytics, s. f.-b). Ejecutar el modelo en la nube permite usar redes más grandes que las que soporta un teléfono, a cambio de depender de la conexión, aceptar la latencia de red y enviar la imagen a un servicio externo.

2.7.6. Antecedente: visión artificial para la ejecución de ejercicios

Silva Amado (2024) propone un sistema de visión artificial que estima la pose con MediaPipe, calcula ángulos articulares y los compara con umbrales para identificar la ejecución de ejercicios; su evaluación es cualitativa. Este enfoque verifica *cómo* se ejecuta un movimiento, mientras que el presente trabajo reconoce *con qué máquina* se entrena y no estima la pose del usuario (capítulo V).

2.7.7. Modelos de lenguaje Extenso (LLM)

Los modelos de lenguaje, son modelos básicos entrenados sobre inmensas cantidades de datos, lo que los hace capaces de comprender y generar lenguaje natural y otros tipos de contenido para realizar una amplia variedad de tareas. Por lo general, los LLM se basan en arquitecturas de aprendizaje profundo, como Transformer, desarrollado por Google en 2017, y se pueden entrenar con miles de millones de textos y otros tipos de contenido.

Debido a su escalabilidad, los LLM se pueden entrenar con otros tipos de datos, como código, imágenes, audio y video, entre otros (Google Cloud, 2025). Su versatilidad y capacidad de aprendizaje los convierten en herramientas clave en el desarrollo de aplicaciones modernas, incluyendo sistemas móviles inteligentes y soluciones que integren realidad aumentada e inteligencia artificial.

2.8. Fundamentos del entrenamiento físico

2.8.1. Entrenamiento físico

El entrenamiento es un conjunto de ejercicios y actividades que se realizan regularmente para mejorar la salud y el estado físico de una persona. Este tipo de entrenamiento puede incluir variedad de actividades, cada una enfocada en diferentes aspectos del cuerpo y objetivos personales (Orientanet, s. f.).

Hoy en día, la tecnología como sensores, aplicaciones móviles, realidad aumentada e inteligencia artificial permiten un monitoreo exhaustivo de la técnica y el análisis del movimiento, ofreciendo retroalimentación personalizada en tiempo real. Estas innovaciones pueden optimizar el rendimiento de los usuarios y ayudar a quienes comienzan en el ejercicio a realizar las actividades de manera más segura, contribuyendo a reducir el riesgo de lesiones y a fomentar un estilo de vida saludable.

2.8.2. Principios del entrenamiento físico

El entrenamiento físico está fundamentado en tres principios esenciales, estos son:

- **Fuerza:** Se centra en incrementar la capacidad del músculo para generar tensión, mediante ejercicios con resistencias progresivas. Esto no solo mejora el rendimiento deportivo, sino que también contribuye a la salud articular y la densidad ósea.
- **Técnica:** Implica la ejecución correcta de cada movimiento para maximizar los beneficios y minimizar riesgos. Una técnica adecuada asegura que el estímulo recae sobre los músculos deseados y evita compensaciones que puedan generar lesiones.
- **Hipertrofia:** Busca aumentar el tamaño del músculo mediante cargas, repeticiones y volúmenes adecuados. La hipertrofia requiere un equilibrio entre sobrecarga progresiva, nutrición y recuperación para generar crecimiento muscular efectivo.

2.8.3. Biomecánica básica

La biomecánica estudia el movimiento del cuerpo humano aplicando principios mecánicos. Comprenderla permite optimizar los ejercicios, analizar palancas, ángulos de articulaciones y fuerza aplicada, lo que mejora la eficiencia y reduce el riesgo de lesiones. Los aspectos clave incluyen la alineación corporal, el centro de gravedad, estabilidad y amplitud de movimiento.

La biomecánica proporciona las bases para corregir patrones de movimiento disfuncionales, los cuales pueden surgir por hábitos sedentarios, desequilibrios musculares o mala ejecución repetida de ejercicios. Identificar estos patrones permite intervenir de manera temprana mediante correcciones técnicas, fortalecimiento de músculos estabilizadores o ajustes en la postura.

2.8.4. Lesiones comunes por mala técnica

La técnica incorrecta en la ejecución de ejercicios, es una de las principales causas de las lesiones en entornos de entrenamiento físico, especialmente en principiantes. La literatura señala que una postura inadecuada, la falta de control motor y el uso de cargas excesivas incrementan el riesgo de daño musculoarticular (Kolt & Snyder-Mackler, 2007).

El desconocimiento de la técnica o la negligencia en la postura pueden provocar lesiones frecuentes, como:

- Dolor lumbar por levantamiento incorrecto de peso.
- Tendinitis por movimientos repetitivos sin control.
- Esguinces o distensiones musculares por falta de calentamiento o postura incorrecta.
- Lesiones articulares en hombros, rodillas o muñecas por sobrecarga o ángulos inadecuados.

Prevenir estas lesiones requiere educación, supervisión y el uso de tecnologías de monitoreo como sensores de movimiento y análisis de video.

2.8.5. Máquinas de gimnasio

Una máquina de gimnasio es un equipo diseñado para facilitar la realización de ejercicios físicos de manera controlada, segura y específica. Estas máquinas están construidas con estructuras metálicas, poleas, cables, placas de peso o mecanismos electrónicos que permiten guiar el movimiento del usuario. Su diseño busca aislar determinados grupos musculares, mejorar la estabilidad durante el ejercicio y proporcionar una carga regulable para adaptarse al nivel de cada persona.

Las máquinas de gimnasio sirven para entrenar diferentes capacidades físicas como fuerza, resistencia muscular, estabilidad y acondicionamiento cardiovascular. Proporcionan un entorno controlado donde el usuario puede ejecutar movimientos guiados que reducen el riesgo de lesiones, especialmente en personas principiantes o en rehabilitación. También permiten trabajar grupos musculares específicos con mayor precisión, mejorar la postura durante el ejercicio y mantener

una progresión segura mediante cargas ajustables. Además, algunas máquinas monitorean parámetros como ritmo cardíaco, velocidad o calorías, facilitando el seguimiento del rendimiento.

Las máquinas de gimnasio pueden clasificarse de diversas formas. Según su función principal, se distinguen las máquinas de fuerza o resistencia, que utilizan poleas, cables, placas de peso o sistemas hidráulicos para trabajar grupos musculares específicos (por ejemplo, prensa de piernas, máquina de pecho, jalón al pecho o curl de bíceps), y las máquinas cardiovasculares, diseñadas para mejorar la resistencia cardiorrespiratoria y quemar calorías (caminadoras, bicicletas estáticas, elípticas y remadoras). Según el tipo de movimiento, pueden ser de movimiento guiado, en las que la máquina controla la trayectoria y que son ideales para principiantes, o de movimiento libre asistido, que permiten mayor libertad pero brindan estabilidad adicional. Según el mecanismo de resistencia, pueden funcionar con placas de peso (weight stack), peso libre integrado, sistemas hidráulicos o resistencia electromagnética, esta última común en las máquinas cardiovasculares modernas. Finalmente, según el área del cuerpo que trabajan, se agrupan en máquinas para tren superior (pecho, espalda, hombros y brazos), tren inferior (piernas y glúteos) y core (abdominales y zona lumbar).

2.8.6. Prescripción del entrenamiento de fuerza

La prescripción del entrenamiento de fuerza se define mediante variables como la selección de ejercicios, la intensidad, el volumen, la frecuencia semanal y los descansos, y requiere una progresión gradual de la carga (Kraemer & Ratamess, 2004). Estas pautas, consolidadas en la literatura, son la base de conocimiento del generador de rutinas del sistema (sección 4.1.5).

2.8.7. Indicadores nutricionales y metabólicos

El índice de masa corporal (IMC) relaciona el peso con la estatura y se clasifica con los puntos de corte de la Organización Mundial de la Salud (2000). Para estimar el gasto energético en reposo, y con él las metas calóricas diarias, se utiliza la ecuación predictiva de Mifflin et al. (1990), desarrollada en personas sanas. Ambos indicadores son orientativos y no reemplazan la valoración de un profesional de la nutrición.

2.9. Plataforma de desarrollo

2.9.1. Visual Studio Code

Visual Studio Code es un editor de código fuente creado por Microsoft, diseñado para ser ligero, rápido y altamente personalizable (Cuadrado, 2022). Sirve para escribir, corregir y organizar programas en muchos lenguajes como Java, Python, C++, entre otros. Este funciona en Windows, Mac y Linux.

A diferencia de un IDE tradicional que viene completo desde la instalación, VS Code parte de un núcleo ligero y va incorporando funcionalidad mediante extensiones, lo que permite adaptarlo a prácticamente cualquier lenguaje o framework sin cargar con herramientas que el proyecto no necesita.

2.10. Frameworks para desarrollo Mobile

El desarrollo de aplicaciones móviles ha evolucionado en gran manera gracias a la aparición de frameworks que permiten crear interfaces eficientes, multiplataforma y optimizadas para dispositivos con recursos limitados. Estos frameworks ofrecen herramientas, bibliotecas y entornos de trabajo que simplifican el proceso de creación de aplicaciones al abstraer componentes nativos y proporcionar estructuras reutilizables.

Entre los frameworks más utilizados se encuentran React Native, Flutter, Ionic, Xamarin y Jetpack

Compose.

La descripción, características y ventajas de cada uno de estos frameworks se comparan en la Tabla A-4 del Anexo A.

2.10.1. React Native

React Native es un framework para la creación de aplicaciones nativas para iOS y Android, creado por Meta (Facebook). Permite construir aplicaciones multiplataforma utilizando JavaScript y React. A diferencia de las soluciones híbridas tradicionales, React Native genera interfaces nativas, lo que permite lograr un rendimiento cercano al desarrollo completamente nativo (React Native, 2025).

React Native se caracteriza por (Viera, s. f.):

- **Componentes nativos:** Usa componentes de interfaz de usuario nativo en lugar de componentes web.
- **Hot reloading:** Permite ver cambios en tiempo real sin recompilar toda la app, acelerando el desarrollo.
- **Acceso a APIs nativas:** Permite integrar funcionalidades como cámara, GPS y notificaciones directamente del dispositivo.
- **Desarrollo Multiplataforma:** Escribe una sola vez y ejecuta en ambas plataformas (iOS y Android).
- **Gran Comunidad:** Amplia comunidad de desarrolladores y recursos disponibles.
- **Rendimiento:** Ofrece un rendimiento cercano al nativo gracias a su arquitectura.
- **Interfaz de Usuario:** Permite crear interfaces de usuario ricas y fluidas.
- **Integración con Librerías:** Compatible con librerías de terceros y APIs nativas.
- **Soporte para TypeScript:** Permite usar TypeScript para un desarrollo más robusto y seguro.

2.11. Bases de datos de aplicaciones Móviles

Las bases de datos desempeñan un papel fundamental en el desarrollo de aplicaciones móviles, ya que permiten almacenar, administrar y sincronizar la información que utiliza el sistema durante su funcionamiento.

En el entorno móvil actual existen múltiples opciones tecnológicas, entre las cuales destacan Firebase Firestore, SQLite, MongoDB Atlas, MySQL y PostgreSQL.

La descripción, características y ventajas de cada una de estas bases de datos se comparan en la Tabla A-5 del Anexo A.

2.11.1. Firebase

Firebase es una plataforma en la nube desarrollada por Google, facilita el desarrollo de aplicaciones, bases de datos en tiempo real, autenticación y hosting. Esta herramienta permite enviar notificaciones push y analizar el rendimiento de las apps (Isaac, 2025).

En la actualidad, Firebase integra servicios modernos como Cloud Firestore, Google Analytics 4, App Check, Data Connect y Firebase Studio, lo que la convierte en una solución completa para proyectos actuales que buscan escalabilidad, seguridad y analítica avanzada (López Mora, 2025).

2.12. Herramientas

2.12.1. Control de versiones

El control de versiones es un sistema que ayuda a rastrear y gestionar los cambios realizados en un archivo o conjunto de archivos, principalmente utilizado por ingenieros de software para hacer seguimiento a las modificaciones realizadas en el código fuente (B, 2025).

En este proyecto se utilizaron:

1. **Git:** Es un sistema de control de versiones distribuido, gratuito y de código abierto diseñado para manejar proyectos pequeños, como también proyectos muy grandes de forma rápida y eficiente (Git, s. f.).
2. **GitHub:** Es un servicio basado en la nube que aloja un sistema de control de versiones (VCS) llamado Git. Éste permite a los desarrolladores colaborar y realizar cambios en proyectos compartidos, a la vez que mantienen un seguimiento detallado de su progreso (B, 2025).

2.12.2. Herramientas de Gestión

Las herramientas de gestión de proyectos están especialmente diseñadas para asistir a un empleado o equipo en la gestión eficaz de sus proyectos y tareas (Gurnov & Wrike, 2025). Estas herramientas son utilizadas para la organización, planificación, seguimiento y aseguramiento de calidad del proyecto.

- **Trello:** Es una herramienta flexible para la gestión del trabajo, basado en tableros Kanban que facilita la organización visual de tareas mediante listas y tarjetas (Atlassian, 2023). Su enfoque intuitivo permite coordinar actividades, asignar responsabilidades y monitorear el avance de cada fase del proyecto.

CAPÍTULO III: ÁREA DE APLICACIÓN

En este capítulo se presenta información general sobre el entrenamiento en gimnasios, sus objetivos y las características de los ejercicios y equipos. Se describen los grupos musculares, el ajuste de las máquinas, la técnica de ejecución y la organización de rutinas. También se abordan el seguimiento físico, el índice de masa corporal y el registro de la alimentación.

3.1. Entrenamiento en gimnasios

El entrenamiento en gimnasios está compuesto por actividades físicas planificadas mediante el uso de máquinas, pesos libres, implementos y el propio peso corporal. Su organización depende de los objetivos específicos de cada usuario, su experiencia y los recursos disponibles. Para lograrlo de forma efectiva, se combinan ejercicios de fuerza, actividades cardiovasculares y rutinas de movilidad.

La Organización Mundial de la Salud (OMS, 2020) recomienda que los adultos acumulen semanalmente entre 150 y 300 minutos de actividad aeróbica moderada, o entre 75 y 150 minutos de actividad vigorosa, además de realizar actividades de fortalecimiento muscular al menos dos días por semana. Estas recomendaciones constituyen una referencia general de salud y pueden desarrollarse dentro o fuera del gimnasio.

3.1.1. Características del entrenamiento en gimnasios

La característica principal del entrenamiento en el gimnasio es que puede adaptarse a las capacidades y a los objetivos de cada persona. Para esto se seleccionan los ejercicios, la cantidad de peso que se utilizará y los días de entrenamiento. Cada uno de estos elementos permite aumentar la dificultad de forma gradual a medida que la persona mejora.

Entre las características más importantes se encuentran las siguientes:

- **Adaptación a las capacidades y objetivos del usuario:** Permite organizar los ejercicios de acuerdo con la experiencia, la condición física y el objetivo del usuario.
- **Regulación de la dificultad:** Permite ajustar el peso, las repeticiones y las series de cada ejercicio.
- **Progresión del entrenamiento:** Permite aumentar de forma gradual la exigencia de acuerdo con la mejora de las capacidades del usuario.
- **Descanso entre series:** Permite distribuir los ejercicios y los periodos de recuperación.
- **Seguimiento del progreso:** Permite registrar el trabajo realizado y comparar los resultados a lo largo del tiempo.

Una medida descriptiva del trabajo externo es el volumen de carga, que se calcula sumando el producto de las repeticiones y los kilogramos utilizados en cada serie:

$$V_{\text{carga}} = \sum_{s=1}^{S} r_s \, m_s. \tag{3.1}$$

Aquí, S es el número de series, r_s las repeticiones de la serie s y m_s la masa externa en kilogramos. Es una medida descriptiva de volumen, no una medición directa del esfuerzo, del trabajo mecánico o del estímulo muscular. Las comparaciones requieren ejercicios y condiciones semejantes.

3.1.2. Objetivos de entrenamiento en gimnasios

Los objetivos de entrenamiento son los resultados que una persona busca alcanzar mediante la práctica de ejercicios físicos. Definirlos ayuda a seleccionar las actividades, organizar las sesiones y evaluar el progreso. Una persona puede tener varios objetivos, aunque es conveniente identificar cuáles son los prioritarios.

Entre los principales objetivos se encuentran:

- **Mejorar la salud y la condición física:** Busca desarrollar capacidades que faciliten las actividades cotidianas y contribuyan al bienestar. La actividad física regular aporta beneficios para la salud cardiovascular, el sueño y la salud mental (OMS, 2024).
- **Aumentar la fuerza muscular:** Consiste en mejorar la capacidad de los músculos para producir fuerza frente a una resistencia, como levantar una mancuerna o desplazar el peso de una máquina. El progreso puede observarse al realizar un mismo ejercicio con mayor resistencia y en condiciones semejantes (Currier et al., 2023).
- **Desarrollar masa muscular:** También denominado hipertrofia muscular, consiste en aumentar el tamaño de los músculos. Aunque la fuerza y el tamaño muscular están relacionados, no representan el mismo resultado. Por ello, una mejora de fuerza no implica necesariamente un aumento proporcional del tamaño muscular (Currier et al., 2023).
- **Mejorar la resistencia muscular:** Busca aumentar la capacidad para repetir un esfuerzo o mantenerlo durante un tiempo determinado. Por ejemplo, realizar más repeticiones de un ejercicio con el mismo peso puede indicar una mejora de esta capacidad (American College of Sports Medicine [ACSM], 2009).
- **Mejorar la capacidad cardiorrespiratoria:** Permite sostener actividades como caminar rápidamente, pedalear o correr. Puede trabajarse mediante ejercicios aeróbicos y combinarse con el entrenamiento de fuerza (Schumann et al., 2022).
- **Contribuir al control de la grasa corporal:** El ejercicio puede formar parte de una estrategia para reducir la grasa corporal. Jayedi et al. (2024) encontraron reducciones del peso, la circunferencia de cintura y la grasa corporal mediante entrenamiento aeróbico en adultos con sobrepeso u obesidad. Estos resultados no garantizan una pérdida de peso idéntica para todas las personas.
- **Mejorar la amplitud de movimiento:** Busca aumentar el recorrido disponible en determinadas articulaciones. Tanto los estiramientos como algunos programas de fuerza pueden contribuir a este objetivo (Afonso et al., 2021).

3.1.3. Niveles de experiencia de un usuario

En el entrenamiento de fuerza, el nivel de experiencia permite orientar la selección de ejercicios y la organización de las cargas. Su valoración debe considerar la práctica reciente, la continuidad y la familiaridad con los movimientos. Esta clasificación es una referencia para planificar; no es un diagnóstico ni garantiza por sí sola la prevención de lesiones o del abandono.

El ACSM (2009) distingue entre personas principiantes, intermedias y avanzadas. Como referencia, considera aproximadamente seis meses de entrenamiento constante para el nivel intermedio y varios años para el avanzado. Estos periodos son orientativos y no deben utilizarse como único criterio de clasificación.

- **Nivel principiante:** Corresponde a personas que comienzan a entrenar o que no han mantenido una práctica reciente y constante. En esta etapa, el aprendizaje se centra en conocer los equipos, comprender las instrucciones y realizar los movimientos de manera controlada.

La dificultad debe ajustarse a sus capacidades iniciales.

- **Nivel intermedio:** Corresponde a personas que han mantenido una práctica regular y conocen los ejercicios habituales de su programa. Su entrenamiento puede requerir ajustes más específicos en el peso utilizado, las series, las repeticiones y la distribución de las sesiones para continuar progresando.
- **Nivel avanzado:** Corresponde a personas con varios años de entrenamiento constante y experiencia en programas estructurados. Su planificación requiere ajustes relacionados con objetivos específicos, así como un seguimiento de la carga de entrenamiento y la recuperación.

3.1.4. Tipos de entrenamiento

Los tipos de entrenamiento pueden distinguirse según la capacidad física que se busca desarrollar y la forma de organizar los ejercicios. Estas categorías pueden complementarse dentro de una misma rutina.

- **Entrenamiento de fuerza o contra resistencia:** Consiste en realizar movimientos frente a una resistencia proporcionada por máquinas, mancuernas, barras, bandas elásticas o el propio peso corporal. Puede orientarse al desarrollo de fuerza, masa muscular o resistencia muscular, según la organización del programa (Currier et al., 2023).
- **Entrenamiento aeróbico:** Comprende actividades sostenidas, como caminar en una cinta, pedalear en una bicicleta estática o utilizar una elíptica. Su dificultad puede regularse mediante la velocidad, la resistencia y la duración. También puede organizarse en intervalos que alternen esfuerzo y recuperación; esto no significa necesariamente que sea de alta intensidad.
- **Entrenamiento de flexibilidad y movilidad:** Incluye actividades destinadas a trabajar la amplitud de movimiento. Los estiramientos son una alternativa, pero no la única. Afonso et al. (2021) encontraron que tanto los estiramientos como el entrenamiento de fuerza podían mejorar la amplitud articular en los estudios analizados.
- **Entrenamiento de equilibrio y control del movimiento:** Incluye tareas que requieren mantener la estabilidad corporal y controlar los cambios de posición. Puede incorporar distintos apoyos y desplazamientos. La OMS (2024) destaca especialmente su importancia en las personas mayores.
- **Entrenamiento combinado o concurrente:** Integra actividades aeróbicas y ejercicios de fuerza dentro de un programa, aunque no necesariamente en la misma sesión. Schumann et al. (2022) encontraron que esta combinación no perjudicó, en promedio, las mejoras de fuerza máxima ni el crecimiento muscular frente al entrenamiento de fuerza por separado. Sin embargo, las mejoras de fuerza explosiva, entendida como la capacidad de producir fuerza rápidamente, podían verse reducidas, especialmente al realizar ambas actividades en una misma sesión.

3.2. Máquinas y equipos de gimnasio

Las máquinas y los equipos son un conjunto de herramientas físicas diseñadas para facilitar la aplicación de resistencias externas durante la realización de ejercicios de manera controlada y segura. Su propósito principal es estimular capacidades físicas como la fuerza, la resistencia muscular, la estabilidad de las articulaciones y la capacidad cardiovascular. Para ello, permiten ajustar la dificultad del ejercicio según las capacidades y la experiencia de cada persona. La elección del equipo depende del movimiento que se desea realizar, las características físicas y las preferencias del usuario, así como los recursos disponibles en el gimnasio.

Haugen et al. (2023) encontraron que tanto las máquinas como los pesos libres pueden mejorar la fuerza y aumentar el tamaño de los músculos, proceso conocido como hipertrofia muscular. Sin embargo, las mejoras de fuerza suelen ser mayores al evaluarlas con el mismo tipo de equipo utilizado durante el entrenamiento. Por ejemplo, quienes entrenan con pesos libres tienden a mejorar más en pruebas realizadas con esos implementos.

3.2.1. Características y clasificación

Las máquinas y los equipos de gimnasio permiten realizar ejercicios mediante diferentes sistemas, como placas de peso, discos, cables y mecanismos de frenado. Sus características determinan cómo se realiza cada movimiento y qué ajustes puede realizar el usuario.

Las máquinas y los equipos utilizados para realizar ejercicios en el gimnasio pueden clasificarse según su funcionamiento y uso principal. Entre ellos se encuentran:

- **Máquinas de trayectoria guiada:** Estas máquinas cuentan con mecanismos que dirigen el movimiento a lo largo de un recorrido establecido. Generalmente permiten ajustar el asiento, los apoyos y la resistencia. Algunos ejemplos son la prensa de piernas, la extensión de cuádriceps y la máquina de empuje de pecho. Aunque facilitan el seguimiento de una trayectoria, requieren una colocación adecuada y una carga acorde con las capacidades del usuario.
- **Máquinas de poleas:** Estas máquinas transmiten la resistencia mediante cables y poleas. Según su diseño, permiten modificar la altura del cable, el agarre y la posición corporal para realizar distintos ejercicios. Algunos ejemplos son el jalón al pecho, las extensiones de tríceps y los cruces en polea. La libertad de movimiento depende de la configuración de cada equipo.
- **Pesos libres:** Incluyen barras, mancuernas y discos que se desplazan sin una trayectoria mecánica fija. El usuario debe controlar tanto el recorrido del implemento como su posición corporal durante el ejercicio. Estos equipos permiten realizar una amplia variedad de movimientos y ajustar la resistencia mediante diferentes pesos.
- **Equipos cardiovasculares:** Estos equipos están destinados principalmente a actividades como caminar, correr, pedalear o remar. Incluyen cintas de correr, bicicletas estáticas, elípticas y máquinas de remo. Permiten regular variables como la velocidad, la inclinación o la resistencia, según el modelo. Algunos incorporan pantallas que muestran el tiempo, la distancia y estimaciones del gasto energético o la frecuencia cardíaca.

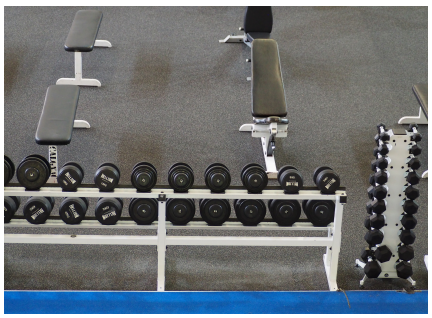
La Figura 3-1 presenta ejemplos de las cuatro categorías. Las fotografías ilustran los equipos; no constituyen una demostración de la técnica de ejecución de los ejercicios.



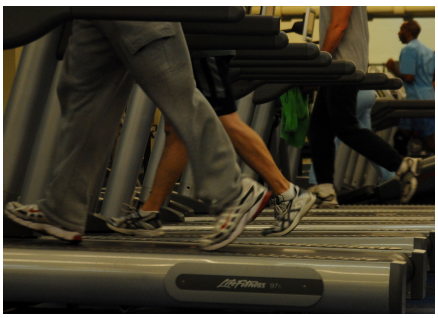
(a) Trayectoria guiada.



(b) Máquina de poleas.



(c) Pesos libres.



(d) Equipos cardiovasculares.

Figura 3-1. Ejemplos de máquinas y equipos de gimnasio

Fuente: Elaboración propia (montaje), con fotografías sin modificaciones de: (a) Teddyhtsai (2021), CC0; (b) SAgbley (2021), CC BY-SA 4.0; (c) KeepActive Australia (2023), CC BY-SA 4.0, Photo by: KeepActive.com.au; (d) Alarcon (s. f.), U.S. Air Force, dominio público en Estados Unidos.

3.2.2. Grupos musculares que se trabajan

Los ejercicios realizados en el gimnasio involucran músculos que producen el movimiento y otros que ayudan a mantener la posición del cuerpo. Por ello, aunque un ejercicio se asocie principalmente con el pecho, la espalda o las piernas, generalmente participan varios músculos de forma coordinada.

Para describir el entrenamiento, los grupos musculares pueden organizarse de forma práctica en tres regiones: tren superior (miembros superiores y tórax), tren inferior (miembros inferiores) y zona media o tronco. Esta clasificación ayuda a identificar los grupos musculares que participan en los ejercicios realizados con cada máquina o equipo.

También conviene diferenciar entre ejercicios multiarticulares, en los que participan varias articulaciones, y monoarticulares, centrados principalmente en una. El ACSM (2009) contempla ambos tipos dentro de los programas de fuerza. Esta distinción permite describir cada ejercicio con mayor precisión que una clasificación basada únicamente en el nombre de la máquina.

3.2.2.1. Grupos musculares del tren superior

Esta región comprende los músculos del pecho, la espalda, los hombros y los brazos. Participan en movimientos como empujar, tirar, levantar los brazos y sujetar objetos.

- **Pectorales:** El pectoral mayor participa en movimientos que acercan el brazo al frente del cuerpo. Se trabaja en ejercicios como el press de pecho y las aperturas.
- **Músculos de la espalda:** El dorsal ancho interviene al acercar el brazo al tronco y llevarlo hacia atrás. El trapecio y los romboides participan en el movimiento y control de las escápulas, conocidas como omóplatos. Los jalones y remos involucran varios de estos músculos.
- **Deltoides:** Se encuentran en los hombros y participan en la elevación del brazo en distintas direcciones. Su intervención depende del movimiento realizado.
- **Bíceps y otros flexores del codo:** Ayudan a doblar el codo. Participan en las flexiones de codo y también colaboran en ejercicios de tracción.
- **Tríceps:** Permite extender el codo, como ocurre en las extensiones de tríceps y los ejercicios de empuje.
- **Músculos del antebrazo:** Intervienen en el agarre y el control de la muñeca.

Estas funciones se describen en el capítulo de musculatura del miembro superior de Betts et al. (2022). Las Figuras 3-2 y 3-3 muestran la localización de varios de estos músculos. En los rótulos, *pectoralis major* corresponde al pectoral mayor; *latissimus dorsi*, al dorsal ancho; *deltoid*, al deltoides; *biceps brachii*, al bíceps braquial; y *triceps brachii*, al tríceps braquial.

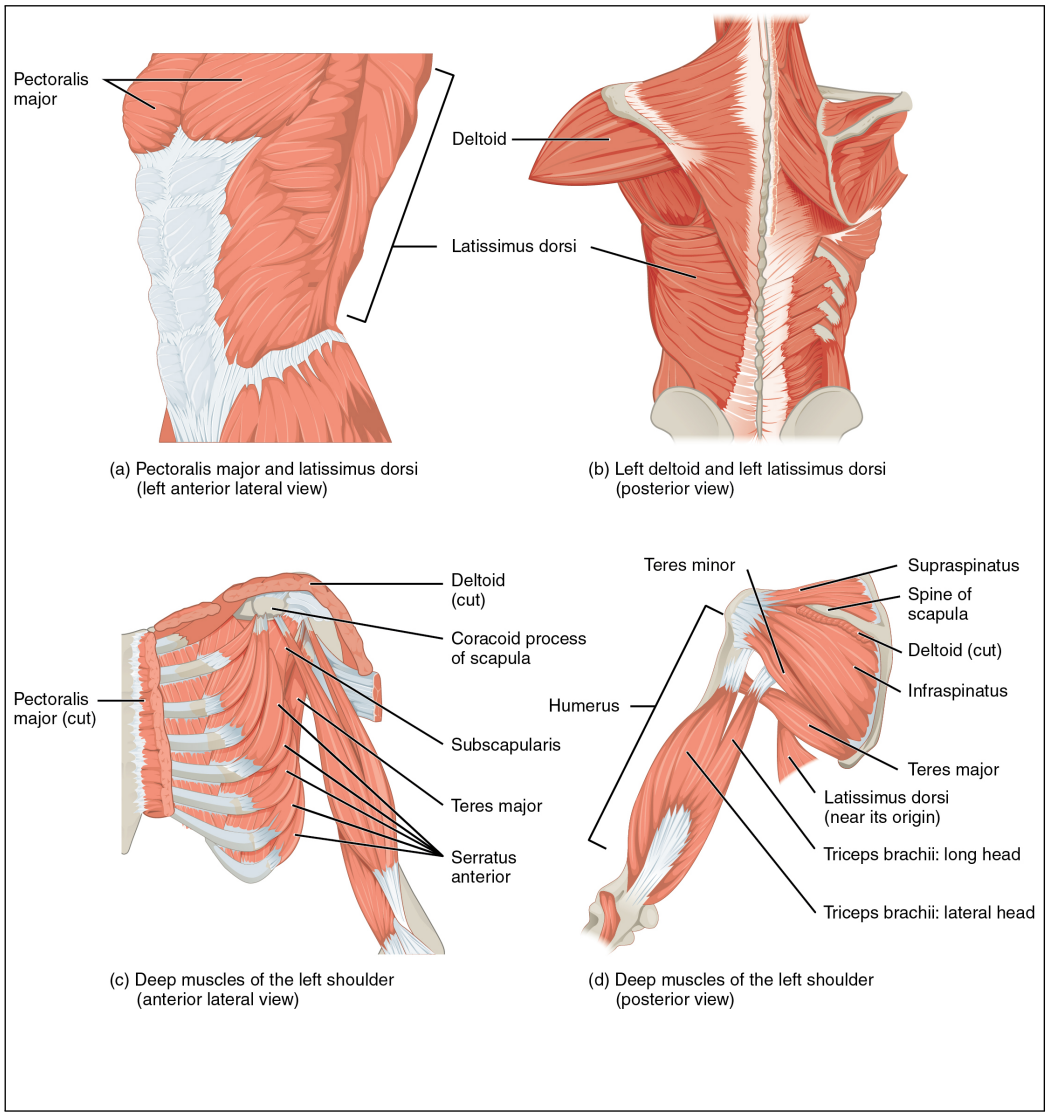


Figura 3-2. Músculos del pecho, la espalda y el hombro

Fuente: Reproducida de Betts et al. (2022, Figura 11.23), OpenStax, licencia CC BY-NC-SA 4.0. Se conservan los rótulos originales en inglés.

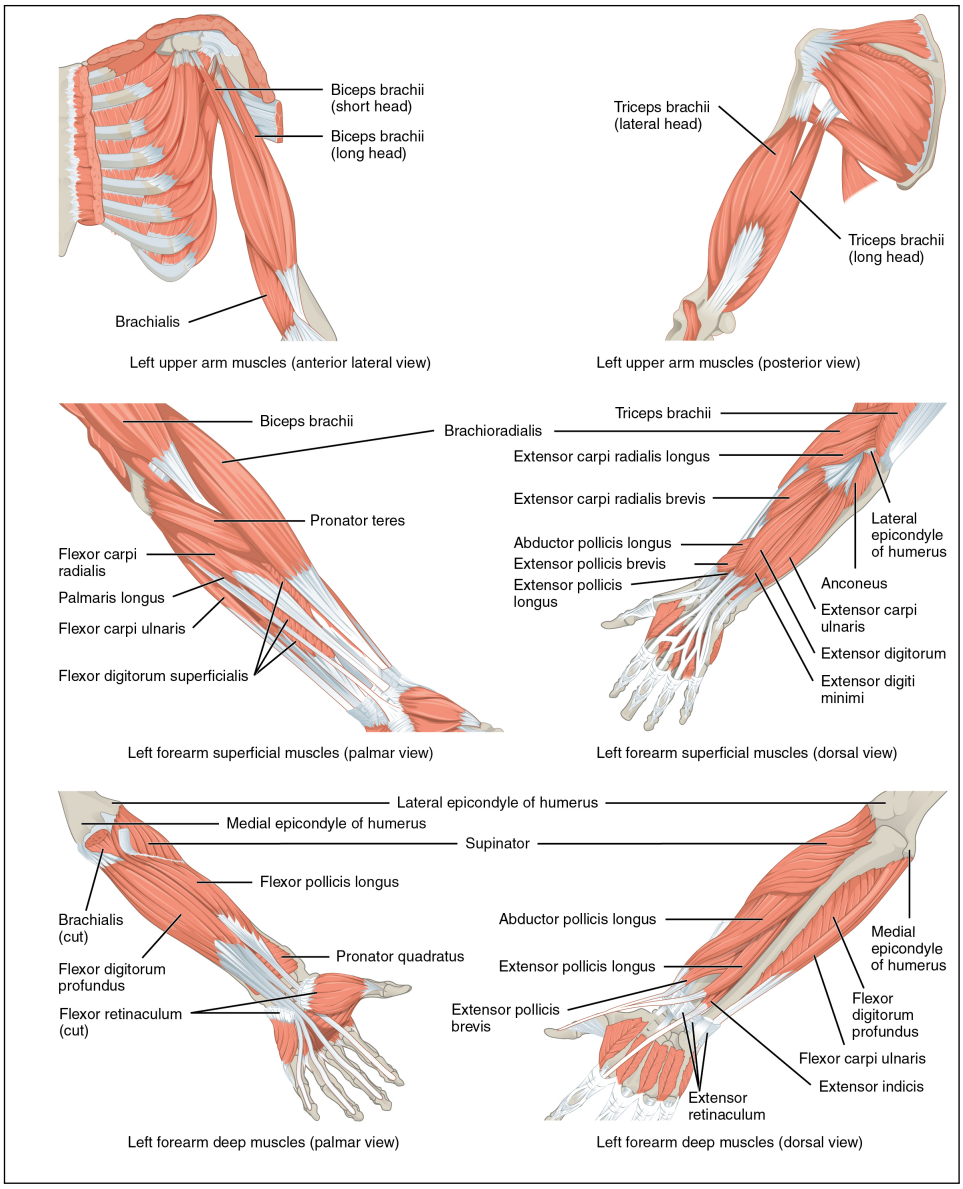


Figura 3-3. Músculos del brazo y del antebrazo

Fuente: Reproducida de Betts et al. (2022, Figura 11.25), OpenStax, licencia CC BY-NC-SA 4.0. Se conservan los rótulos originales en inglés.

3.2.2.2. Grupos musculares del tren inferior

Esta región comprende los músculos de la cadera, los muslos y las piernas. Participan en acciones como levantarse, caminar, subir escalones y mantener el equilibrio.

- **Cuádriceps:** Se encuentran en la parte anterior del muslo y permiten extender la rodilla. Se trabajan en ejercicios como las extensiones de rodilla, las sentadillas y la prensa.
- **Isquiotibiales:** Están en la parte posterior del muslo. Participan en la flexión de rodilla y, en su mayoría, en la extensión de cadera. Se trabajan mediante ejercicios como el curl femoral.
- **Glúteos:** El glúteo mayor participa en la extensión de cadera; el glúteo medio y el menor contribuyen a separar el muslo y controlar la posición de la pelvis.
- **Aductores:** Se ubican principalmente en la cara interna del muslo y acercan la pierna hacia la línea central del cuerpo.
- **Pantorrillas:** El gastrocnemio y el sóleo permiten elevar el talón, como al ponerse de puntillas.

Estos grupos y sus funciones se describen en Betts et al. (2022). Las Figuras 3-4 y 3-5 permiten identificar sus posiciones. Algunos rótulos equivalentes son *gluteus maximus* (glúteo mayor), *rectus femoris* (recto femoral, parte del cuádriceps), *biceps femoris* (bíceps femoral, parte de los isquiotibiales), *gastrocnemius* (gastrocnemio) y *soleus* (sóleo).

Los ejemplos anteriores identifican grupos relevantes, pero no representan un aislamiento absoluto. Una prensa de piernas, por ejemplo, requiere la participación coordinada de varios músculos.

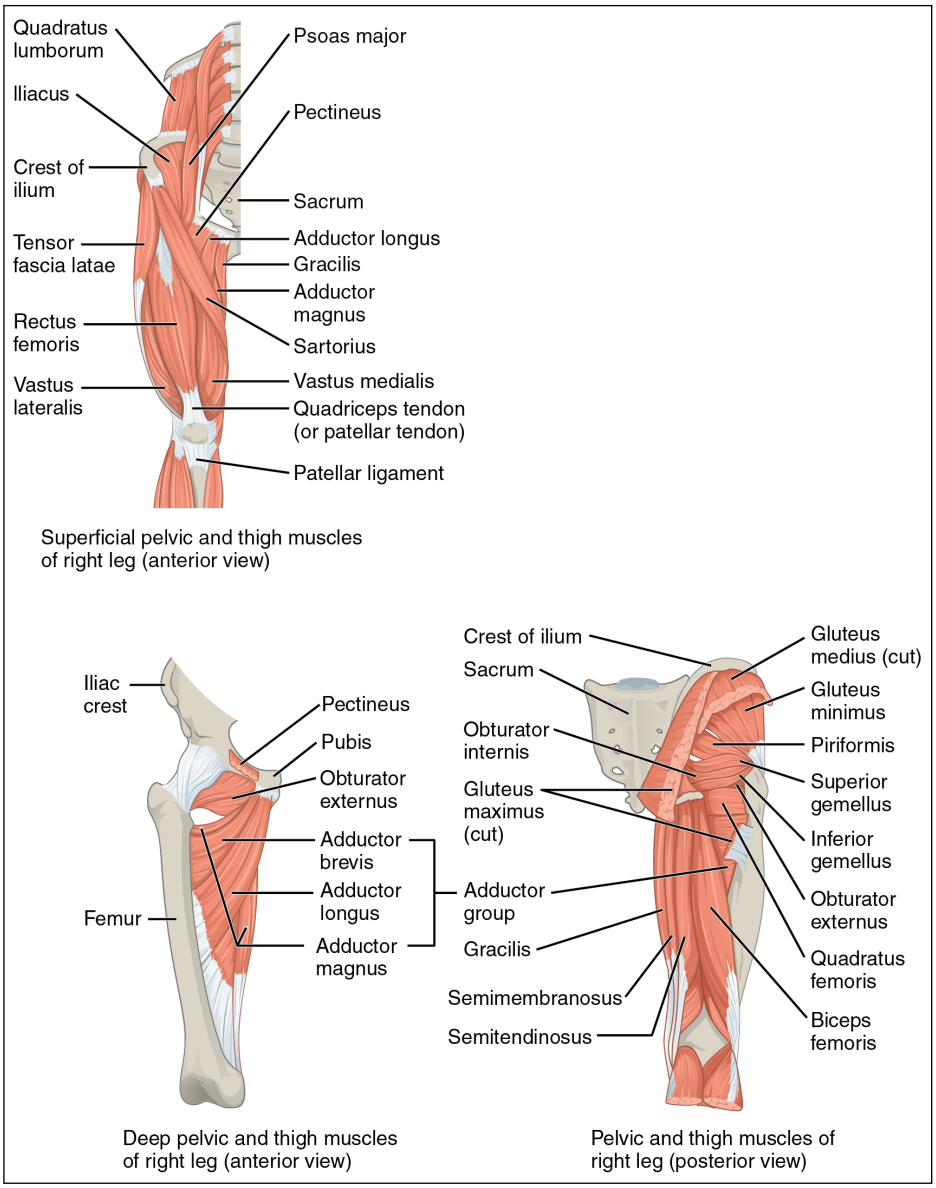


Figura 3-4. Músculos de la cadera y del muslo

Fuente: Reproducida de Betts et al. (2022, Figura 11.29), OpenStax, licencia CC BY-NC-SA 4.0. Se conservan los rótulos originales en inglés.

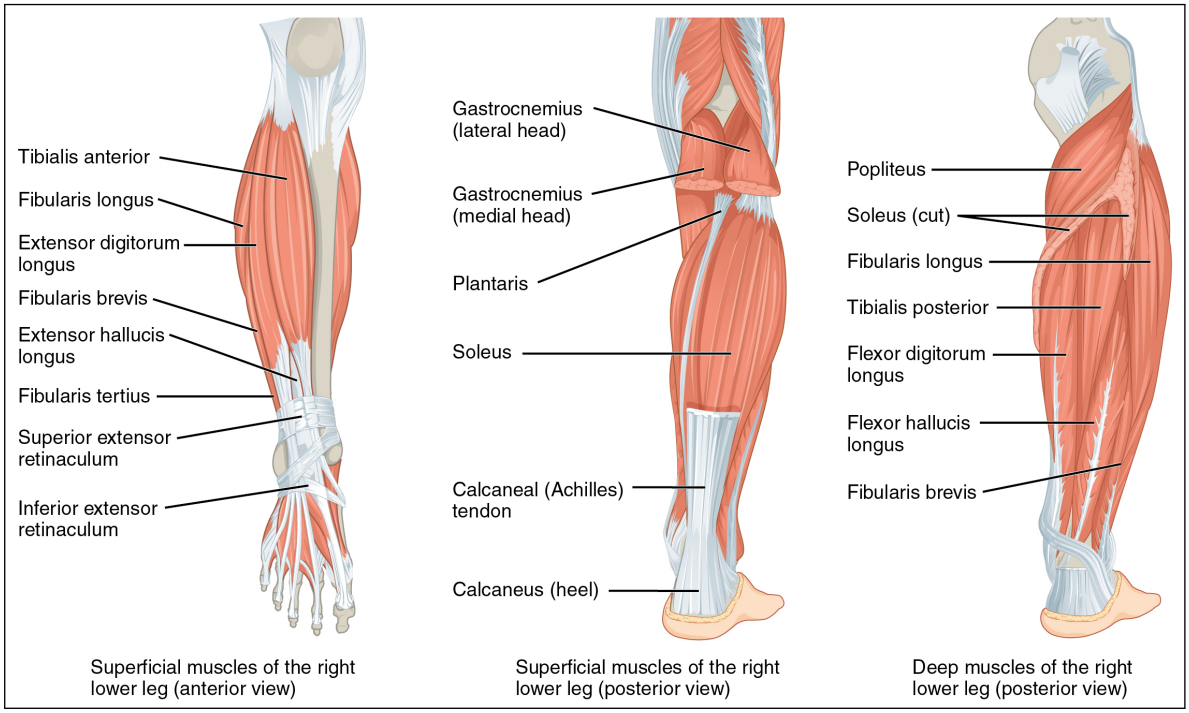


Figura 3-5. Músculos de la pierna y de la pantorrilla

Fuente: Reproducida de Betts et al. (2022, Figura 11.32), OpenStax, licencia CC BY-NC-SA 4.0. Se conservan los rótulos originales en inglés.

3.2.2.3. Músculos de la zona central del cuerpo

La zona central, frecuentemente denominada *core*, comprende músculos que participan en el movimiento y control del tronco. No se limita a los abdominales visibles.

- **Recto abdominal:** Participa en la flexión del tronco, es decir, al aproximar el tórax hacia la pelvis.
- **Oblicuos internos y externos:** Intervienen en la rotación, la inclinación lateral y el control del tronco.
- **Transverso abdominal:** Contribuye a comprimir el contenido abdominal y al control de la pared abdominal.
- **Erectores de la columna y otros músculos posteriores:** Participan en la extensión y el mantenimiento de la posición del tronco.

En algunos ejercicios estos músculos generan movimiento; en otros, trabajan para limitar movimientos no deseados. Por ejemplo, durante una plancha se busca mantener una posición relativamente estable en lugar de flexionar repetidamente el tronco. Las funciones anatómicas se describen en Betts et al. (2022).

La Figura 3-6 muestra músculos de la pared abdominal. Los rótulos *rectus abdominis*, *external oblique* y *transversus abdominis* corresponden al recto abdominal, el oblicuo externo y el transverso abdominal, respectivamente. La lámina representa parte de la musculatura central; no incluye todos los músculos que contribuyen al control del tronco.

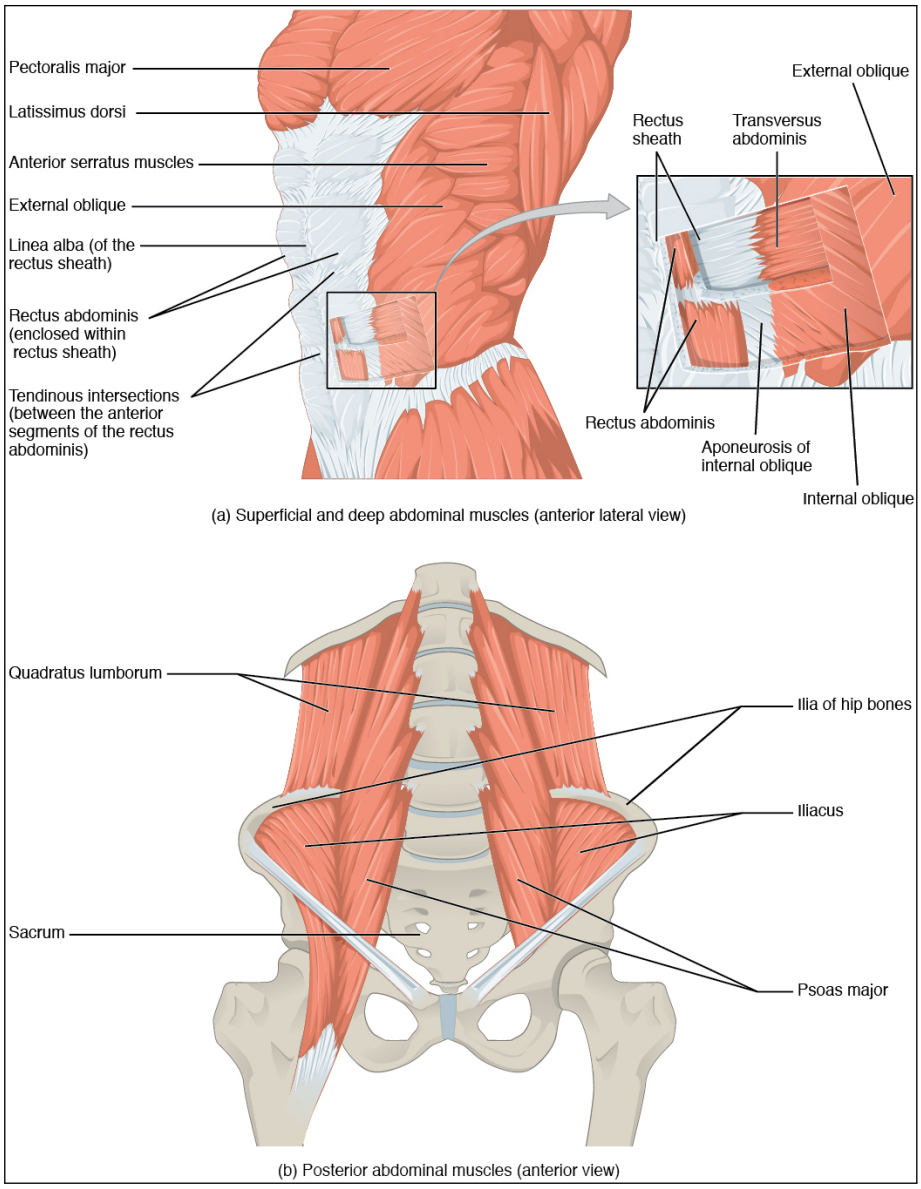


Figura 3-6. Músculos superficiales y profundos de la pared abdominal

Fuente: Reproducida de Betts et al. (2022, Figura 11.16), OpenStax, licencia CC BY-NC-SA 4.0. Se conservan los rótulos originales en inglés.

3.2.2.4. Participación de los músculos durante un ejercicio

Para describir un ejercicio, resulta útil diferenciar tres funciones:

- **Músculos principales:** Contribuyen de manera importante a producir el movimiento.
- **Músculos colaboradores:** Ayudan a realizarlo.
- **Músculos estabilizadores:** Contribuyen a mantener las posiciones necesarias para ejecutarlo.

Estas funciones dependen de la tarea. Un músculo puede producir movimiento en un ejercicio y ayudar a estabilizar una articulación en otro. Por eso, un ejercicio no debe describirse como si trabajara un único músculo de manera completamente independiente.

3.2.2.5. Ejercicios multiarticulares y monoarticulares

Otra clasificación se basa en el número de articulaciones que intervienen durante el movimiento. Se distinguen los siguientes tipos:

- **Ejercicios multiarticulares:** Estos ejercicios implican el movimiento de dos o más articulaciones de forma simultánea. Por ejemplo, en una sentadilla participan la cadera, la rodilla y el tobillo; en un press de pecho intervienen el hombro y el codo. La carga utilizada depende del ejercicio y de las capacidades de la persona.
- **Ejercicios monoarticulares:** Estos ejercicios concentran el movimiento principalmente en una articulación, como la extensión de rodilla en máquina o el curl de bíceps con el brazo relativamente fijo.

La palabra «monoarticular» no significa que participe un solo músculo ni que el resto del cuerpo permanezca inactivo.

Rosa et al. (2023), en una revisión sistemática y metaanálisis de siete estudios, encontraron resultados similares de crecimiento muscular global entre ambas modalidades. También señalaron que ciertos ejercicios monoarticulares podrían complementar el desarrollo de regiones específicas de un músculo, aunque esta posibilidad requiere mayor investigación. Por tanto, la elección depende del propósito del programa y no de una superioridad general de una categoría.

3.2.3. Ajuste y uso correcto de las máquinas

El ajuste de una máquina consiste en adaptar sus elementos regulables a las dimensiones antropométricas del usuario y a las demandas biomecánicas del ejercicio previsto. Antes de comenzar se deben reconocer el asiento, el respaldo, los apoyos, los agarres y el mecanismo de selección de la resistencia. Las instrucciones del fabricante son la referencia para cada modelo: dos máquinas destinadas a un mismo ejercicio pueden tener mecanismos y recorridos diferentes. Las recomendaciones prácticas sobre el manejo del equipo deben distinguirse de los resultados de los estudios que evalúan las adaptaciones al entrenamiento.

Asiento, respaldo y puntos de apoyo. La posición inicial debe permitir alcanzar los agarres o apoyos sin adoptar una postura forzada. Los elementos regulables deben quedar fijados antes de aplicar resistencia. Como ejemplo específico, Mayo Clinic (2025) recomienda que, en la máquina de empuje de pecho descrita en su guía, el asiento sitúe los agarres a la altura de los hombros o ligeramente por debajo. Esta indicación corresponde a ese ejercicio y no constituye una regla aplicable a todas las máquinas. La guía también recomienda un movimiento suave y controlado, sin arquear la espalda para completar el empuje.

Selección de la resistencia y familiarización. La carga inicial debe permitir practicar el recorrido y comprender cómo iniciar y terminar el ejercicio. Si la resistencia obliga a perder los apoyos o impide controlar el retorno, conviene reducirla y revisar el ajuste. Las recomendaciones de Mayo Clinic (s. f.) destacan el aprendizaje de la técnica, el control del peso y la ayuda de un profesional cuando la persona comienza a entrenar. Una máquina guiada facilita una trayectoria, pero no verifica por sí misma la colocación corporal ni garantiza que el peso elegido sea adecuado.

Recorrido y finalización. Se debe identificar la posición de inicio, la amplitud prevista y el modo de devolver la resistencia a su apoyo. El recorrido no debe ampliarse a costa de perder el contacto necesario con el asiento o el respaldo. Tampoco es conveniente modificar los ajustes mientras las piezas soportan una carga. Estas son pautas generales de manejo; los topes, seguros y mecanismos de salida deben utilizarse según las instrucciones del equipo.

Preparación y supervisión. McCrary et al. (2015) encontraron que ciertas modalidades de calentamiento dinámico con cargas elevadas mejoraban el rendimiento de fuerza y potencia del tren superior. Sin embargo, no identificaron estudios sobre la prevención de lesiones en esa región. Por ello, estos resultados no deben presentarse como una garantía de prevención de lesiones. Para aprender la técnica y revisar los ajustes del equipo, Mayo Clinic (s. f.) recomienda solicitar ayuda a un profesional cualificado.

3.3. Ejercicios y técnica de ejecución

La técnica de ejecución comprende la posición de inicio, las articulaciones que se mueven, la trayectoria y el control de la resistencia. Describe cómo se realiza una tarea y permite comparar repeticiones bajo condiciones semejantes. Su enseñanza requiere explicar el movimiento con referencias comprensibles, en lugar de limitarse a indicar el nombre del ejercicio.

La biomecánica estudia el movimiento y las fuerzas que actúan sobre el cuerpo. Lorenzetti et al. (2018), en un estudio con 42 participantes, observaron que la separación y la orientación de los pies durante la sentadilla modificaban el movimiento y las demandas de la cadera y la rodilla. Estos resultados ayudan a entender la técnica, pero no permiten atribuir automáticamente lesiones a una postura: el estudio evaluó variables biomecánicas, no la incidencia de lesiones a largo plazo.

3.3.1. Tipos de ejercicios

Además de su finalidad física, los ejercicios pueden describirse mediante la siguiente clasificación práctica:

- **Según las articulaciones que se mueven:** los multiarticulares incluyen movimientos relevantes en varias articulaciones; los monoarticulares concentran el movimiento principalmente en una. Esta distinción no equivale a contar los músculos que participan.
- **Según el apoyo o lado de trabajo:** los bilaterales utilizan ambos miembros en una misma tarea, mientras que los unilaterales dan protagonismo a uno de ellos. Una sentadilla bilateral y una subida a un escalón ilustran esta diferencia.
- **Según el movimiento predominante:** pueden distinguirse empujes, tracciones, movimientos dominantes de rodilla, movimientos de cadera y tareas de control del tronco. Esta organización es funcional y no constituye una clasificación anatómica rígida.
- **Según la acción muscular:** en una acción concéntrica el músculo se acorta mientras produce fuerza; en una excéntrica se alarga mientras resiste la carga; en una isométrica mantiene una longitud aproximadamente constante. En un curl de bíceps, elevar y bajar la mancuerna de forma controlada ejemplifica las dos primeras acciones.

El ACSM (2009) contempla acciones concéntricas, excéntricas e isométricas y ejercicios de una o varias articulaciones dentro de los programas de fuerza. Estas categorías pueden coexistir: un ejercicio puede ser bilateral, multiarticular y contener fases concéntricas y excéntricas.

3.3.2. Postura y ejecución correcta

Una ejecución adecuada permite cumplir el propósito del ejercicio manteniendo el control del cuerpo y de la resistencia. No supone reproducir ángulos idénticos en todas las personas. La posición debe considerar las dimensiones corporales, el equipo, el movimiento previsto y las limitaciones individuales. Los resultados de Lorenzetti et al. (2018) muestran precisamente que modificar la posición de los pies cambia las demandas del ejercicio.

Amplitud de movimiento. Es el recorrido que realiza una articulación durante una repetición. Debe describirse para cada ejercicio y mantenerse suficientemente constante para interpretar el progreso. Alizadeh et al. (2023), en una revisión de 55 estudios, encontraron que el entrenamiento con resistencias externas podía mejorar la amplitud articular. Este hallazgo no implica que deba forzarse cualquier recorrido: una mejora de movilidad y la selección de la amplitud de un ejercicio son cuestiones relacionadas, pero diferentes.

Control de las fases del ejercicio. El desplazamiento de la resistencia y su retorno forman parte de la repetición. En el aprendizaje inicial, es útil reconocer ambas fases y evitar que el impulso sustituya involuntariamente el movimiento previsto. La respiración también debe integrarse en la ejecución, sin convertir la retención prolongada del aire en una instrucción general para principiantes (Mayo Clinic, s. f.).

Observación y retroalimentación. La explicación verbal, la demostración y la observación directa permiten revisar aspectos concretos de la técnica. Coleman et al. (2023) compararon ocho semanas de entrenamiento supervisado y no supervisado en 36 personas entrenadas y encontraron ventajas de la supervisión en algunas adaptaciones musculares. Sus resultados respaldan el valor del acompañamiento en ese contexto, sin demostrar que cualquier corrección postural produzca los mismos efectos.

3.3.3. Errores frecuentes durante los ejercicios

Los errores deben describirse como diferencias respecto al movimiento previsto y no como posturas que inevitablemente causan lesiones. Entre los problemas prácticos que conviene revisar se encuentran:

- **Carga incompatible con el control disponible:** el peso impide completar el recorrido previsto o provoca cambios involuntarios de posición.
- **Ajustes inadecuados:** el asiento, el respaldo o los apoyos no permiten adoptar la posición necesaria para el ejercicio.
- **Pérdida de control en el retorno:** se deja caer la resistencia en lugar de controlar su descenso.
- **Recorrido inconsistente:** la amplitud se reduce entre sesiones sin registrarlo, dificultando interpretar una aparente mejora de rendimiento.
- **Uso involuntario del impulso:** el balanceo del cuerpo sustituye parte del movimiento que se pretendía realizar.
- **Continuar ante dolor:** se ignora una molestia dolorosa para terminar la serie. Mayo Clinic (s. f.) recomienda detener el ejercicio si causa dolor y revisar la actividad o la carga.

La revisión consiste en identificar la dificultad, ajustar la tarea y comprobar nuevamente la ejecución. La fatiga, el desconocimiento del equipo y una instrucción poco clara pueden requerir soluciones diferentes. La sensación de esfuerzo no debe confundirse automáticamente con el dolor ni utilizarse como una medida directa de lesión.

3.4. Rutinas de entrenamiento

Una rutina es una organización de ejercicios y sesiones que establece qué actividades se realizan y cómo se distribuye el trabajo. Debe indicar sus objetivos y permitir revisar lo realizado. No se reduce a una lista de máquinas: también comprende la cantidad de ejercicio, los descansos y los criterios de modificación.

Currier et al. (2023) analizaron diferentes combinaciones de carga, series y frecuencia en adultos sanos. Todas las prescripciones evaluadas mejoraron la fuerza y la hipertrofia frente a no entrenar. Las cargas elevadas destacaron para la fuerza máxima y las prescripciones con varias series para el crecimiento muscular. La existencia de distintas alternativas eficaces permite adaptar la organización a las circunstancias de la persona.

3.4.1. Series, repeticiones, cargas y descansos

- **Repetición:** Es la ejecución de un ciclo del ejercicio. Su registro requiere definir el recorrido y las condiciones de realización.
- **Serie:** Es un conjunto de repeticiones realizadas antes de una pausa. Si se completan tres grupos de diez repeticiones, se han realizado tres series.
- **Carga externa:** Es la resistencia aplicada durante el ejercicio. En ejercicios con pesas puede expresarse en kilogramos, aunque esa cifra no describe por completo la dificultad.
- **Esfuerzo percibido:** Es la valoración de la dificultad que experimenta la persona. Es diferente de la carga externa: un mismo peso puede representar esfuerzos distintos según la persona o la sesión.
- **Descanso:** Es un intervalo entre series o ejercicios. Su duración influye en la recuperación disponible antes del siguiente esfuerzo.

El volumen puede expresarse como número de series o repeticiones. También puede calcularse un volumen de carga sumando los productos de las repeticiones y los kilogramos utilizados en cada serie. Este indicador sirve para describir el trabajo externo, pero no mide directamente el estímulo muscular ni permite equiparar ejercicios con recorridos y mecanismos diferentes.

Singer et al. (2024) reunieron nueve estudios y encontraron indicios de una pequeña ventaja para la hipertrofia con descansos superiores a 60 segundos respecto a intervalos más cortos, con incertidumbre en las estimaciones. Para fuerza, Grgic et al. (2018) encontraron que los descansos superiores a dos minutos podían favorecer los resultados en personas entrenadas. Estas observaciones no establecen un tiempo obligatorio para todos los ejercicios; la pausa debe permitir realizar el siguiente esfuerzo de acuerdo con el objetivo previsto.

3.4.2. Organización de las sesiones de entrenamiento

Las sesiones pueden organizarse trabajando diferentes regiones corporales en un mismo día o distribuyéndolas entre varios días. En ambos casos, es necesario conocer la cantidad total de trabajo y evitar interpretar el nombre de una división semanal como garantía de eficacia.

Una organización comprensible distingue la preparación del ejercicio, las series de trabajo y los descansos. También establece qué actividades tienen prioridad y cómo se distribuirán en la semana. La OMS (2020) recomienda actividades de fortalecimiento de todos los grandes grupos

musculares al menos dos días semanales, además de actividad aeróbica. Esta recomendación de salud pública no constituye una plantilla individual de series o cargas.

La frecuencia expresa cuántas veces se realiza una actividad o se trabaja un grupo muscular en un periodo. Debe distinguirse del número total de días de asistencia al gimnasio: entrenar cuatro días no significa ejercitar cada músculo cuatro veces. El seguimiento de las sesiones permite reconocer esa diferencia y registrar interrupciones o modificaciones.

3.4.3. Personalización según objetivos y nivel de experiencia

Personalizar significa ajustar la selección y la cantidad de ejercicios a los objetivos, la experiencia, el tiempo disponible y los equipos accesibles. También requiere revisar cómo responde la persona al programa. La edad, el peso corporal o el tiempo de inscripción en el gimnasio no bastan por sí solos para determinar la dificultad adecuada.

La progresión puede realizarse aumentando el peso o las repeticiones. Plotkin et al. (2022) compararon ambas estrategias durante ocho semanas en 43 personas con experiencia y observaron adaptaciones musculares con las dos formas de progresión. Por tanto, aumentar la carga externa no es la única manera de modificar la exigencia.

Un criterio práctico es modificar los parámetros después de comprobar que el trabajo previsto puede repetirse con suficiente control. Los cambios deben registrarse para distinguir una progresión planificada de una variación ocasional. Las rutinas también pueden necesitar ajustes por interrupciones, disponibilidad o dificultades de ejecución; personalizar no significa aumentar continuamente la exigencia ni garantizar mejoras al mismo ritmo.

3.5. Datos físicos y seguimiento del progreso

El seguimiento reúne mediciones corporales y registros del rendimiento para observar su evolución. Cada indicador responde a una pregunta diferente: la masa corporal informa sobre el peso total, mientras que las repeticiones o cargas describen el desempeño en una tarea. No deben combinarse automáticamente en una calificación única de salud.

3.5.1. Peso corporal y estatura

En el uso cotidiano se denomina peso corporal a la masa que registra una báscula, generalmente expresada en kilogramos. La estatura es la distancia vertical entre el suelo y la parte superior de la cabeza al permanecer de pie. Ambas mediciones requieren un procedimiento consistente.

El manual de antropometría de la National Health and Nutrition Examination Survey establece protocolos y controles del equipo para obtener mediciones comparables (National Center for Health Statistics [NCHS], 2021). Entre las condiciones relevantes se encuentran retirar el calzado, utilizar instrumentos apropiados y adoptar una posición definida. Para la estatura se utiliza un estadiómetro, que permite situar una pieza horizontal sobre la cabeza. Los valores deben registrarse con sus unidades y sin intercambiar centímetros y metros.

En el seguimiento de una persona conviene mantener condiciones semejantes entre mediciones y anotar circunstancias que puedan afectar la comparación. Un cambio de báscula, ropa o procedimiento introduce variación adicional. La repetición de una medida dudosa ayuda a detectar errores de registro.

3.5.2. Índice de masa corporal y sus limitaciones

El índice de masa corporal (IMC) relaciona el peso con la estatura y se calcula mediante:

$$\mathrm{IMC} = \frac{\text{peso en kilogramos}}{(\text{estatura en metros})^2}. \tag{3.2}$$

Los Centers for Disease Control and Prevention (CDC, 2024) utilizan los siguientes intervalos para adultos de 20 años o más:

- **Bajo peso:** IMC menor que 18,5.
- **Peso saludable:** IMC desde 18,5 y menor que 25.
- **Sobrepeso:** IMC desde 25 y menor que 30.
- **Obesidad:** IMC igual o mayor que 30.

Estos intervalos son categorías de evaluación inicial, no diagnósticos completos. En personas de 2 a 19 años, el CDC utiliza una interpretación relacionada con la edad y el sexo, por lo que no corresponde trasladar directamente los intervalos anteriores.

El IMC no distingue grasa, músculo y hueso ni informa sobre la distribución de la grasa corporal (CDC, 2025). Esto es relevante en personas que entrenan fuerza: un IMC elevado puede estar influido por una mayor masa muscular. Tampoco permite concluir que una persona con un valor dentro del intervalo denominado saludable tenga necesariamente una composición corporal o condición física determinada. Su interpretación debe considerar otros antecedentes y mediciones.

3.5.3. Registro de entrenamientos y evolución física

Un registro útil identifica la fecha, el ejercicio, el equipo, las series, las repeticiones, la carga y los descansos. Puede complementarse con observaciones sobre el esfuerzo y las modificaciones de la sesión. La finalidad es comparar tareas semejantes y reconocer tendencias, en lugar de atribuir significado a una cifra aislada.

Haugen et al. (2023) encontraron que las ganancias de fuerza muestran especificidad respecto a la modalidad entrenada y evaluada. Por ello, mover más kilogramos en una máquina diferente no demuestra necesariamente una mejora equivalente en el ejercicio original. La comparación debe considerar el equipo y las condiciones de ejecución.

La evolución física tampoco se reduce a subir o bajar de peso. Es posible registrar cambios en el desempeño sin variaciones equivalentes de masa corporal. Conviene distinguir entre asistencia, cumplimiento del entrenamiento, rendimiento y mediciones corporales: son dimensiones complementarias, pero ninguna explica por sí sola el estado general de salud.

3.6. Registro de la alimentación

El registro alimentario describe los alimentos y bebidas consumidos durante un periodo. Permite revisar cantidades, horarios y estimaciones de energía y nutrientes. Su interpretación depende de la precisión con la que se identifican los productos, sus porciones y su preparación.

Los registros alimentarios pueden contener errores de declaración y de estimación de las cantidades consumidas. Burrows et al. (2019) compararon métodos de evaluación dietética con el método del agua doblemente marcada y encontraron una subestimación frecuente de la ingesta energética declarada. Por ello, un diario alimentario debe interpretarse como una estimación y no como una medición exacta del consumo.

3.6.1. Calorías y macronutrientes

En nutrición, la energía de los alimentos suele expresarse en kilocalorías (kcal), denominadas habitualmente calorías en el etiquetado. Los macronutrientes energéticos son los carbohidratos, las proteínas y las grasas. Como aproximación general, los carbohidratos y las proteínas aportan 4 kcal por gramo, y las grasas 9 kcal por gramo. Estos factores no explican exactamente todos los valores de las etiquetas, porque existen componentes y métodos de cálculo específicos (Canadian Food Inspection Agency, 2025).

- **Carbohidratos:** Estos contribuyen al aporte energético. Su registro debe considerar la cantidad y el alimento del que proceden, sin asumir que todos los productos con carbohidratos tienen la misma composición.
- **Proteínas:** Son las que aportan aminoácidos utilizados en la formación y renovación de tejidos. Jäger et al. (2017) propusieron un consumo aproximado de 1,4 a 2,0 gramos por kilogramo de peso al día para la mayoría de las personas sanas que realizan ejercicio. Es una referencia de nutrición deportiva y no una indicación universal ni una obligación de utilizar suplementos.
- **Grasas:** Estas forman parte de los alimentos y aportan energía en mayor cantidad por gramo que los otros dos macronutrientes. Su aporte debe interpretarse dentro del conjunto de la alimentación.

El balance energético relaciona la energía ingerida con la utilizada por el organismo. El National Heart, Lung, and Blood Institute (2022) explica que un exceso mantenido de energía puede contribuir al aumento de peso, aunque intervienen factores biológicos, ambientales y de comportamiento. Un cálculo diario no permite predecir con exactitud la evolución individual.

3.6.2. Diario de alimentación

El diario debe describir qué se consumió y cuánto, incluidas las bebidas y los componentes que puedan omitirse, como aceites o salsas. Registrar la información cerca del momento de consumo es una estrategia práctica para disminuir olvidos, aunque no elimina los errores de declaración descritos por Burrows et al. (2019).

- **Identificación del alimento:** nombre, marca cuando corresponda y forma de preparación.
- **Cantidad consumida:** gramos, mililitros o una medida doméstica cuya equivalencia pueda establecerse.
- **Estado del alimento:** distinguir entre crudo y cocido y entre porción comestible y partes no consumidas.
- **Preparaciones mixtas:** describir ingredientes y cantidades cuando no exista una referencia adecuada para el plato completo.
- **Fecha y momento de consumo:** permiten revisar la distribución de lo registrado.

La documentación de Foundation Foods explica cómo calcular nutrientes por porción a partir de valores por 100 gramos de parte comestible (U.S. Department of Agriculture, Agricultural Research Service [USDA ARS], 2024). Si una porción contiene 150 gramos y el alimento aporta 100 kcal por 100 gramos, su aporte estimado es 150 kcal. La misma relación se aplica a los nutrientes expresados sobre esa base:

$$N_{\text{porción}} = N_{100\,\text{g}} \times \frac{m}{100\,\text{g}}, \tag{3.3}$$

donde m es la masa de la porción comestible y $N_{100\,\mathrm{g}}$ representa la cantidad del nutriente por 100 gramos.

Los resultados dependen de la correspondencia entre el alimento de referencia y el consumido. Las diferencias de receta, marca o preparación pueden alterar la estimación. Registrar calorías y macronutrientes no demuestra por sí solo que una dieta cubra todas las necesidades nutricionales.

CAPÍTULO IV: DESARROLLO DEL PROYECTO

En este capítulo se describe el desarrollo del sistema según las fases de la metodología Mobile-D. Se detallan las actividades de cada fase, los artefactos obtenidos y la aplicación de los fundamentos del marco referencial. El desarrollo del sistema busca cumplir los objetivos del proyecto y dar respuesta a la problemática planteada.

4.1. Fase de Exploración

En esta fase se definen los requerimientos, el alcance y las tecnologías para el desarrollo del producto.

4.1.1. Establecimiento de los interesados

Tutor del proyecto(Cliente): Ing. Jimmy Villarroel Novillo.

4.1.2. Actores del sistema

El actor principal del sistema es el usuario, quien completa su perfil, consulta las instrucciones para el uso correcto de las máquinas del gimnasio y genera rutinas de entrenamiento personalizadas. Además, puede registrar su consumo calórico, consultar su índice de masa corporal (IMC) y hacer un seguimiento de su progreso físico.

4.1.3. Roles y Tareas

Para el desarrollo del sistema se han identificado los siguientes usuarios:

Nombre	Descripción	Stakeholder
Usuario Final	Completa su perfil físico, consulta las instrucciones para el uso correcto máquinas del gimnasio, escanea máquinas, obtiene y planifica sus rutina, registra su consumo calórico, consulta su índice de masa corporal IMC y revisa su progreso físico.	Persona que entrena en el gimnasio

Tabla 4-1. Roles y tareas de Coach App (Elaboración propia)

4.1.4. Requerimientos iniciales

En esta etapa se elabora la primera definición de los requerimientos. Los requerimientos funcionales describen las acciones que la aplicación debe permitir al usuario, mientras que los no funcionales establecen las condiciones de calidad que debe cumplir. Esta definición inicial delimita el alcance del sistema y sirve de base para su diseño y desarrollo.

4.1.4.1. Requerimientos funcionales

Los requerimientos funcionales describen las funciones que la aplicación debe ofrecer al usuario. La siguiente tabla presenta los requerimientos generales y sus respectivos subrequerimientos.

CÓDIGO	DESCRIPCIÓN
RF1.1	Permitir el registro de un usuario con nombre, apellido, correo electrónico y contraseña.
RF1.2	Permitir el inicio de sesión mediante correo electrónico y contraseña o mediante una cuenta de Google.
RF1.3	Permitir la recuperación de la contraseña por correo electrónico y el cierre de sesión.
RF1.4	Permitir que el usuario registre, en su primer ingreso, su objetivo de entrenamiento, nivel de experiencia, edad, estatura, peso y días disponibles para entrenar.
RF1.5	Permitir la consulta y edición de los datos del perfil.
RF2.1	Permitir la búsqueda de máquinas por texto, con o sin tildes, y el filtrado del catálogo por grupo muscular y tipo de equipo.
RF2.2	Mostrar la ficha de cada máquina con su descripción, los músculos trabajados, las instrucciones de uso, los errores comunes, las recomendaciones para prevenir lesiones y las variantes comerciales.
RF2.3	Mostrar los ejercicios asociados a cada máquina con su animación, músculo objetivo y nivel de dificultad.
RF2.4	Permitir el escaneo de una máquina con la cámara y mostrar su ficha sobre la imagen en vivo. Si el sistema no logra identificarla automáticamente, permitir que el usuario la seleccione de una lista.
RF3.1	Generar una rutina semanal según el objetivo de entrenamiento y el nivel de experiencia del usuario.
RF3.2	Permitir la creación y eliminación de rutinas personalizadas organizadas por días, con series, repeticiones y tiempos de descanso para cada ejercicio.
RF3.3	Permitir que el usuario agregue ejercicios a su rutina desde la ficha de una máquina y marque una rutina como activa.
RF4.1	Calcular el índice de masa corporal (IMC), clasificar el resultado según los criterios de la OMS y guardar el historial de cálculos.
RF4.2	Permitir el registro del peso, la estatura y las notas de progreso físico, con cálculo automático del IMC.
RF5.1	Permitir la búsqueda de alimentos en español y el registro de la cantidad consumida en gramos. Calcular las calorías y los macronutrientes de la porción registrada.
RF5.2	Permitir que el usuario agregue alimentos de consumo frecuente con un toque o registre un alimento manualmente.
RF5.3	Mostrar el consumo diario de calorías y macronutrientes en relación con las metas del usuario y permitir la eliminación de registros.

CÓDIGO	DESCRIPCIÓN
RF6.1	Mostrar en un gráfico la evolución del peso y del IMC, e indicar la variación acumulada y la categoría actual del IMC.
RF6.2	Permitir que el usuario marque como completado el entrenamiento del día y consulte su racha, los días entrenados durante la semana y el total de entrenamientos completados.

Tabla 4-2. Requerimientos funcionales del sistema (elaboración propia)

4.1.4.2. Requerimientos no funcionales

Los requerimientos no funcionales establecen las condiciones de calidad que debe cumplir la aplicación. La tabla siguiente describe cada requerimiento.

CÓDIGO	DESCRIPCIÓN
RNF1	**Portabilidad.** La aplicación debe poder instalarse y ejecutarse en dispositivos con Android 7.0 (API 24) o superior.
RNF2	**Usabilidad.** La interfaz debe presentarse en español, adaptarse a distintos tamaños de pantalla y permitir el acceso a los módulos principales mediante pestañas de navegación.
RNF3	**Fiabilidad.** Los cambios guardados en las rutinas, los registros y los entrenamientos deben sincronizarse mediante Firebase y reflejarse en pantalla sin que el usuario tenga que recargarla manualmente.
RNF4	**Seguridad.** Cada usuario debe poder consultar y modificar únicamente sus propios datos. El catálogo debe ser de solo lectura para los usuarios de la aplicación. Las reglas de Firestore deben validar los tipos y rangos de los datos al guardar los cambios.
RNF5	**Eficiencia.** El catálogo de máquinas debe descargarse una sola vez por sesión y reutilizarse al volver a consultarlo. Las URL de las imágenes deben conservarse en caché para reducir las solicitudes repetidas.
RNF6	**Mantenibilidad.** El código debe escribirse en TypeScript con el modo estricto habilitado y organizarse según las capas del patrón MVC. La comprobación de tipos y el análisis estático deben completarse sin errores.

Tabla 4-3. Requerimientos no funcionales del sistema (elaboración propia)

4.1.4.3. Análisis de los requerimientos mediante casos de uso

La Figura 4-1 presenta el diagrama de casos de uso del sistema. El actor principal es el usuario; Firebase y las APIs externas participan como actores secundarios.

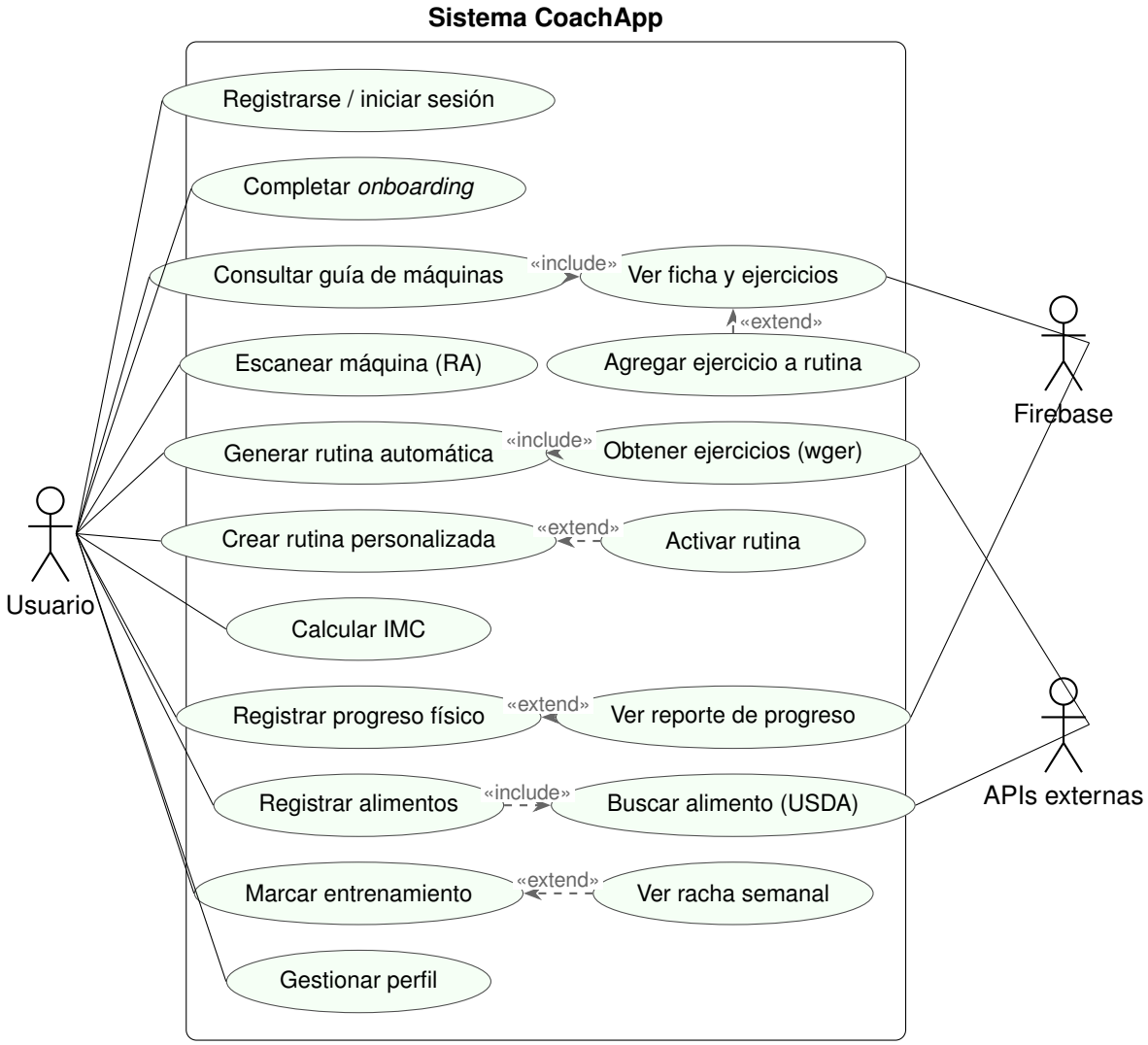


Figura 4-1. Diagrama de casos de uso del sistema

Fuente: Elaboración propia.

4.1.5. Definición del alcance

El alcance del proyecto se determina a través de sus limitaciones:

4.1.5.1. Limitaciones

Las limitaciones de la aplicación desarrollada son:

- La aplicación solo puede ser ejecutada en dispositivos con el sistema operativo Android.
- El reconocimiento de máquinas en la nube y la búsqueda de alimentos requieren conexión a internet.
- El escaneo de máquinas requiere acceso a la cámara. El anclaje de la ficha al entorno solo está disponible en dispositivos compatibles con ARCore.
- El reconocimiento automático se limita a los tipos de máquinas incluidos en el entrenamiento de los modelos.
- La generación de rutinas se basa en reglas de prescripción del ejercicio y en los datos del perfil del usuario.

4.2. Fase de Inicialización

En esta fase se preparó el entorno técnico, se definió la arquitectura del sistema y se diseñó el modelo de datos.

Durante la fase de Inicialización se establecieron las herramientas para implementar la aplicación, gestionar el acceso y la persistencia de datos, e integrar los componentes de reconocimiento de máquinas y generación de rutinas.

4.2.1. Entorno de trabajo

- **React Native:** Framework para crear aplicaciones móviles con React.
- **TypeScript:** Lenguaje basado en JavaScript que incorpora tipado estático.
- **Expo:** Plataforma con herramientas para desarrollar, probar y compilar aplicaciones React Native.
- **Expo Router:** Sistema de navegación que define las rutas a partir de los archivos del proyecto.
- **Firebase:** Plataforma que ofrece servicios de autenticación, bases de datos, almacenamiento de archivos y funciones en la nube.
- **TensorFlow Lite:** Biblioteca para ejecutar modelos de aprendizaje automático en dispositivos móviles y embebidos.
- **Ultralytics YOLO:** Familia de modelos de visión artificial que permite detectar y localizar objetos en imágenes.
- **ARCore:** Plataforma de realidad aumentada que permite seguir el movimiento del dispositivo y anclar elementos virtuales al entorno físico.
- **USDA FoodData Central:** Base de datos sobre la composición nutricional de los alimentos.
- **Git:** Sistema de control de versiones que registra los cambios en los archivos.
- **GitHub:** Plataforma para alojar repositorios Git y colaborar en su desarrollo.

4.2.2. Configuración del entorno de desarrollo

En esta etapa se configuraron los recursos físicos y técnicos necesarios para preparar el entorno de desarrollo de aplicaciones para Android.

4.2.3. Configuración para aplicaciones móviles android

- **Tipo de proyecto.** Aplicación móvil para Android desarrollada con React Native y TypeScript, sobre la plataforma Expo.
- **Librerías principales.**
  - `expo-router` para organizar la navegación entre pantallas mediante rutas basadas en archivos.
  - `firebase` para gestionar la autenticación, consultar y guardar datos en Cloud Firestore, acceder a Cloud Storage e invocar funciones del servidor.
  - `expo-camera` y `expo-image` para capturar imágenes con el escáner y mostrar imágenes y GIF del catálogo.
  - `react-native-fast-tflite` para ejecutar en el dispositivo los modelos de clasificación de máquinas y recomendación de rutinas.
  - `react-hook-form` y `zod` para gestionar los formularios y validar los datos ingresados.
  - `zustand` para gestionar el estado compartido de la aplicación y conservarlo entre sesiones.
  - `react-native-chart-kit` para generar los gráficos del reporte de progreso físico.
- **Configuraciones técnicas.**
  - Generación de APK mediante EAS Build para distribución en Android[cite: 2].
  - Activación de permisos de cámara.
  - Configuración de reglas de seguridad para control de acceso a Cloud Firestore y Cloud Storage[cite: 2, 7].
  - Definición de perfiles de compilación (*development*, *preview* y *production*) en `eas.json` para automatizar el empaquetado en la nube[cite: 3].
  - Uso opcional de funciones de servidor para procesar validaciones y tareas en la nube mediante Firebase[cite: 7].

4.2.4. Planificación inicial

Durante esta etapa se establecerán los requisitos previos técnicos y funcionales de cada proceso, teniendo en cuenta las necesidades del usuario y las condiciones de la infraestructura tecnológica.

4.2.4.1. Análisis de procesos

La implementación de cada proceso requiere que se cumplan las condiciones previas que se detallan a continuación:

Código	Proceso	Pre-requisitos
P01	Autenticación y control de acceso	▪ Configuración de Firebase Authentication para el acceso con correo electrónico y contraseña o con una cuenta de Google. ▪ Validación de las credenciales mediante el proveedor seleccionado. ▪ Creación del documento de perfil en Cloud Firestore durante el primer acceso. ▪ Verificación del estado de la sesión y del perfil para determinar qué pantallas puede abrir el usuario.

Código	Proceso	Pre-requisitos
P02	Configuración inicial del perfil	▪ Usuario autenticado mediante el proceso P01. ▪ Formulario para registrar el objetivo, el nivel de entrenamiento, la edad, la estatura, el peso y los días disponibles. ▪ Validación de los campos obligatorios y de los valores ingresados. ▪ Guardado del perfil en Cloud Firestore antes de habilitar el acceso a las pantallas principales.
P03	Consulta de la guía de máquinas	▪ Catálogo de máquinas y ejercicios disponible para consulta. ▪ Datos descriptivos e imágenes asociados a cada máquina. ▪ Campo de búsqueda y filtros para localizar máquinas en el catálogo. ▪ Ficha con instrucciones de uso para la máquina seleccionada.
P04	Escaneo e identificación de máquinas	▪ Dispositivo Android con cámara y permiso de acceso concedido. ▪ Imagen de la máquina obtenida mediante el escáner. ▪ Modelo local de TensorFlow Lite disponible para clasificar la imagen. ▪ Conexión a Internet y acceso al servicio de detección cuando se utiliza el reconocimiento en la nube. ▪ Ficha disponible en el catálogo y opción de selección manual. ▪ Dispositivo compatible con ARCore para anclar la ficha al entorno físico cuando se utiliza esa modalidad.
P05	Generación de rutinas	▪ Usuario autenticado con el perfil completado en P02. ▪ Objetivo, nivel de entrenamiento y días disponibles registrados en el perfil. ▪ Modelo local de recomendación y catálogo de ejercicios disponibles para conformar la rutina. ▪ Acceso a la función `guardarRutina` para validar la rutina en el servidor y guardarla como activa.

Código	Proceso	Pre-requisitos
P06	Planificación de rutinas	▪ Usuario autenticado con acceso al planificador. ▪ Selección de días de entrenamiento y ejercicios del catálogo. ▪ Registro de series, repeticiones y tiempos de descanso por ejercicio. ▪ Validación de la planificación antes de guardar la rutina personalizada mediante la función `guardarRutina`.
P07	Registro de alimentos	▪ Usuario autenticado y fecha de registro seleccionada. ▪ Texto de búsqueda para consultar alimentos mediante la función `buscarAlimentos`, conectada a USDA FoodData Central. ▪ Selección de un alimento con información nutricional disponible. ▪ Cantidad en gramos para calcular las calorías y los macronutrientes de la porción. ▪ Guardado de la porción en el diario de alimentación del usuario.
P08	Registro corporal	▪ Usuario autenticado con acceso al módulo de progreso físico. ▪ Peso en kilogramos y estatura en centímetros con valores válidos. ▪ Cálculo del índice de masa corporal y determinación de la categoría correspondiente. ▪ Guardado del registro con su fecha para incorporarlo al historial.
P09	Seguimiento de entrenamientos	▪ Usuario autenticado con una rutina activa. ▪ Selección de una sesión de entrenamiento planificada. ▪ Confirmación de que la sesión fue completada. ▪ Registro de la sesión y su fecha para actualizar la racha, el conteo semanal y el total de entrenamientos.
P10	Generación de reportes de progreso	▪ Usuario autenticado con registros corporales almacenados en P08. ▪ Consulta del historial en orden cronológico. ▪ Datos disponibles para mostrar la categoría actual y, cuando existen registros comparables, calcular la variación corporal. ▪ Componente de gráficos para representar la evolución del usuario. ▪ Componente de generación de PDF para exportar el reporte.

Código	Proceso	Pre-requisitos
P11	Acceso a la aplicación	▪ Archivo .apk generado. ▪ Dispositivo compatible. ▪ Permisos de instalación de fuentes desconocidas. ▪ Espacio de almacenamiento suficiente en el dispositivo. ▪ Servidor *backend* disponible 24/7 (Firebase Cloud).

Tabla 4-4. Cuadro de análisis de procesos (Elaboración propia)

4.2.4.2. Planificación de las fases

La funcionalidad se distribuyó en seis iteraciones de producción (Tabla 4-5), ordenadas de modo que cada una se apoye en las anteriores: primero la identidad del usuario, luego el catálogo, que alimenta al escáner y a las rutinas, y finalmente los módulos de seguimiento.

4.2.4.3. Planificación de las fases

El desarrollo se organizó según las fases de Mobile-D. La Tabla 4-5 resume las actividades de cada fase y la secuencia de las seis iteraciones de producción.

Fase	Iteración	Descripción
Exploración	Iteración 1	Identificación de los interesados y actores del sistema, definición de los requerimientos funcionales y no funcionales, y delimitación del alcance.
Inicialización	Iteración 2	Configuración del entorno de desarrollo para Android, análisis de los procesos, planificación de las iteraciones y diseño de la arquitectura, el modelo de datos y la interfaz de usuario.
Producción	Iteración 3	Implementación de la autenticación y el perfil de usuario, la guía de máquinas y ejercicios, el escáner para el reconocimiento de máquinas, la generación y planificación de rutinas, el diario de alimentación y el seguimiento del progreso físico con reportes en PDF.
Estabilización	Iteración 4	Integración de los módulos, refactorización del código y verificación del funcionamiento conjunto de la aplicación.
Finalización	Iteración 5	Preparación de la documentación final, el manual de usuario, los informes de pruebas y la versión de la aplicación para su distribución en formato APK.

Tabla 4-5. Cuadro de análisis de planificación de fases

Fuente: Elaboración propia.

4.2.5. Diseño de la arquitectura del sistema

La arquitectura del proyecto sigue un modelo cliente–servidor. Como se muestra en la Figura 4-2, la aplicación móvil se comunica directamente con los servicios de Firebase para gestionar la autenticación, la base de datos y el almacenamiento, y consulta la API pública de wger (wger Project, s. f.) para obtener información sobre ejercicios. Para consultar los datos nutricionales de USDA y reconocer máquinas mediante Ultralytics, la aplicación utiliza las funciones correspondientes de Cloud Functions.

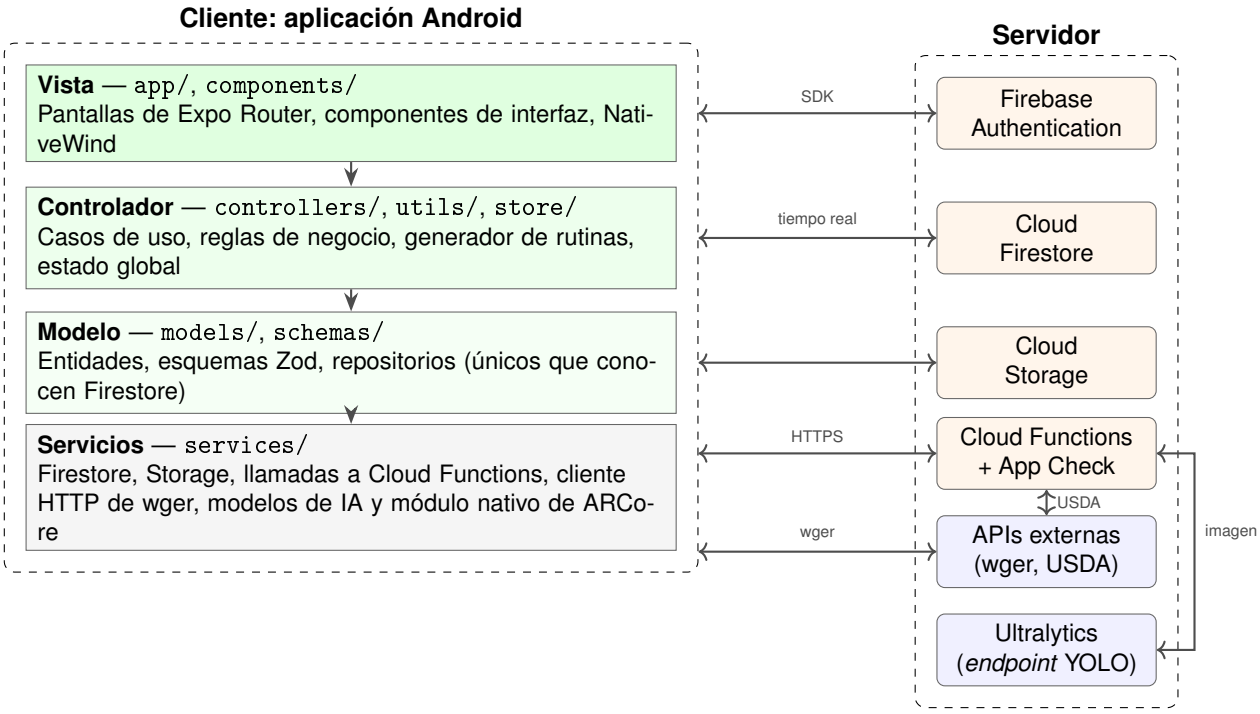


Figura 4-2. Arquitectura cliente–servidor y capas del cliente

Fuente: Elaboración propia.

Capa conceptual	Capa MVC	Implementación
Presentación	Vista	Pantallas y componentes de `app/` y `components/`.
Lógica de negocio	Controlador	Controladores de `controllers/`, reglas puras de `utils/` (IMC, generador de rutinas, estadísticas de entrenamiento).
Servicios (Firebase y APIs)	Modelo y Servicios	Repositorios de `models/repositories/`; `firestoreService`, `storageService`, `functionsService` (llamadas a las funciones en la nube), `wgerService` y `usdaService` (que invoca la función `buscarAlimentos`).
Inteligencia artificial	Controlador y Servicios	Generador de rutinas basado en reglas (`utils/`) y, en `services/ai/`, el recomendador de rutinas y el reconocedor de máquinas, ambos modelos de TensorFlow Lite ejecutados en el dispositivo, más la consulta opcional al detector de la nube mediante `reconocerMaquina`.
Realidad aumentada	Vista y módulo nativo	Vista `CoachAR` (`modules/coach-ar/`, Kotlin) y su interfaz tipada en `services/ar/`; el controlador solo entrega a la vista la caja y la ficha de la máquina confirmada.

Tabla 4-6. Correspondencia entre las capas conceptuales y las capas del cliente

4.2.5.1. Modelo de datos NoSQL

El modelo de datos NoSQL define cómo se organiza, almacena y consulta la información en una base de datos no relacional. Cloud Firestore utiliza un modelo documental en el que los datos se agrupan en colecciones. Cada colección contiene documentos, y cada documento almacena la información en campos. A continuación el esquema diseñado para la base de datos del sistema.

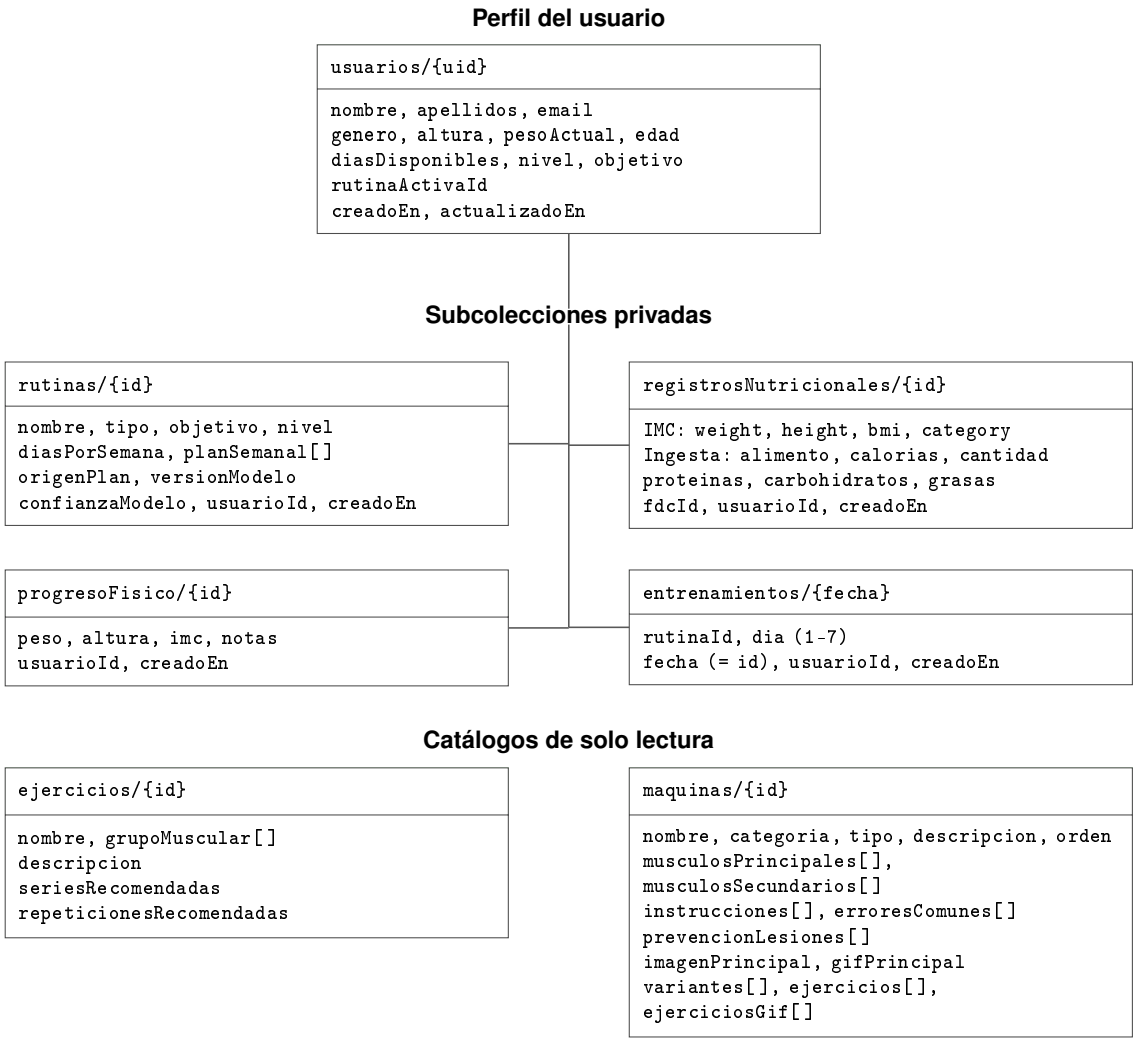


Figura 4-3. Modelo de datos en Cloud Firestore

Fuente: Elaboración propia.

4.2.5.2. Diseño de seguridad y controles

El diseño de seguridad de la aplicación busca proteger los datos personales y preservar la integridad de la información almacenada. Para ello, se establecieron controles de autenticación, acceso a los datos y validación de las operaciones que se ejecutan en el servidor.

- **Base de datos.** Las reglas de seguridad de Cloud Firestore utilizan request.auth para identificar al usuario cuando se autentica, obtener el uid y la información solicitada. Estas reglas permiten definir quién puede leer, crear, actualizar o eliminar documentos. En la aplicación, se utilizan para:
  - Restringir el acceso al perfil, las rutinas y los registros de alimentación, progreso físico y entrenamiento al usuario propietario de los datos.
  - Permitir que los usuarios autenticados consulten los catálogos de máquinas y ejercicios, y reservar su modificación para operaciones autorizadas.
  - Impedir que la aplicación cree rutinas directamente en Firestore. La función `guardarRutina` verifica la identidad del usuario y valida los datos antes de guardarlos.
- **Almacenamiento de archivos.** Las reglas de Cloud Storage restringen la lectura de los re-

cursos del catálogo a usuarios autenticados y controlan quién puede modificar los archivos.

- **Funciones del servidor.** Las funciones de búsqueda de alimentos, guardado de rutinas y reconocimiento de máquinas verifican la identidad del usuario. Firebase App Check, integrado con Play Integrity, complementa este control al verificar el origen de las solicitudes. Las claves de los servicios externos se mantienen en el servidor.

4.2.5.3. Consideraciones éticas y de privacidad

La aplicación almacena datos personales relacionados con la salud (peso, estatura, edad, género y objetivo de entrenamiento) asociados a una cuenta de correo electrónico. Asimismo, el sistema no reemplaza la supervisión de entrenadores ni de nutricionistas, y las estimaciones de calorías, IMC y metas diarias son referenciales. Para entrenar el reconocedor de máquinas se utilizaron imágenes de catálogos y fotografías tomadas en distintos gimnasios del departamento de Cochabamba.

Con el modo de reconocimiento de maquinas en la nube activado, la fotografía que toma el escáner sale del teléfono: viaja a la Cloud Function y de ahí al *endpoint* de Ultralytics, que la procesa para devolver las detecciones. La aplicación no guarda esas fotografías ni las poses de ARCore en Firestore, y la imagen temporal del caché se elimina al reemplazarla o al cerrar la vista.

4.2.5.4. Descripción de la interfaz de usuario

La presente figura 4-4 muestra el recorrido del usuario desde la pantalla de bienvenida hasta las secciones principales de la aplicación. Después de iniciar sesión, el usuario accede a Inicio si su perfil está completo. En caso contrario, primero completa la configuración del perfil. La barra de navegación permite acceder a cuatro pestañas: Inicio, Máquinas, Progreso y Nutrición. Desde Inicio se accede a las rutinas y al perfil. Máquinas reúne el escáner y las fichas de consulta, Progreso presenta el historial y los reportes, y Nutrición contiene el diario de alimentación.

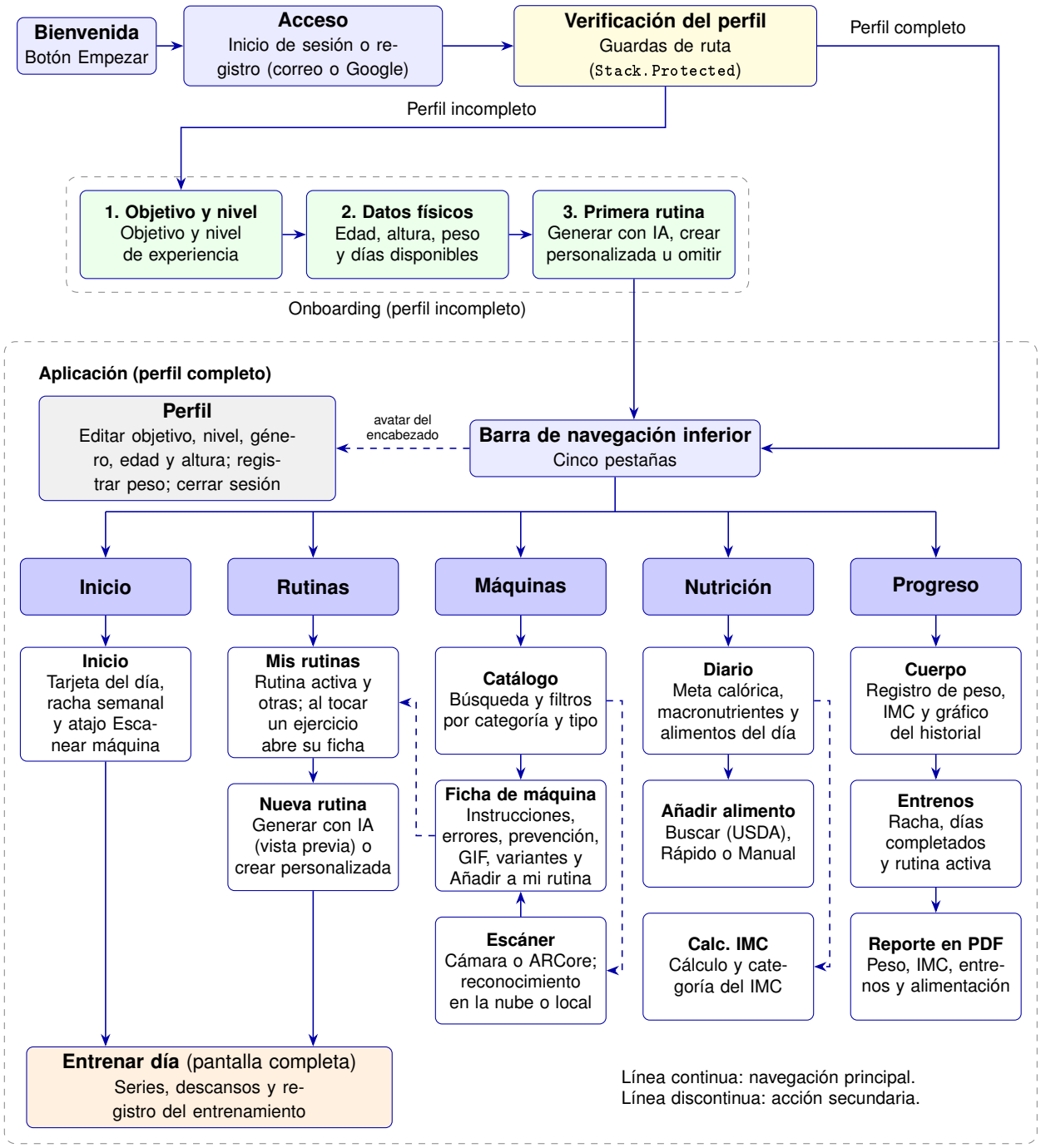


Figura 4-4. Esquema de navegación de usuario

Fuente: Elaboración propia.

4.3. Fase de Producción

En esta fase de producción se realiza el desarrollo de las funcionalidades planteadas en el diseño con React Native. El desarrollo se realizo en iteraciones, según la planificación. En cada iteración se definieron las historias de usuario y sus criterios de aceptación. A partir de ellos, se codificaron los procesos diseñados y se verificó el funcionamiento de cada módulo antes de integrarlo con los demás componentes de la aplicación.

4.3.1. Estructura de directorios

A continuación, el Código 4-1 muestra la estructura de la aplicación y una breve descripción de los directorios más importantes.

```
coach-app/
  src/
    app/
      (auth)/
      (onboarding)/
      (app)/
        routines/
        machines/
        progress/
    components/
    controllers/
    utils/
    hooks/
    store/
    models/
      entities/
      repositories/
    schemas/
    services/
      firebase/
      wger/
      usda/
      exercisedb/
      ai/
      ar/
      reports/
    config/
    constants/
    types/
    media/
  modules/
    coach-ar/
  assets/
    models/
  android/
  ml/
    detection/
    recognition-yolo/
  functions/
    src/
    test/
  tests/
    security/
  docs/
  .github/
    workflows/
```

Código 4-1. Estructura de carpetas del código fuente

- **Carpeta** `src/app/`**:** contiene las pantallas de la aplicación y la definición de las rutas mediante Expo Router. Incluye el diseño raíz, que protege las rutas según el estado de la sesión y del perfil, y los grupos de pantallas de acceso (`(auth)`), de configuración inicial (`(onboarding)`) y de la aplicación autenticada (`(app)`) con su barra de pestañas.
- **Carpeta** `src/components/`**:** contiene los componentes visuales reutilizables, organizados por módulo (inicio, máquinas, rutinas, nutrición, progreso y entrenamiento) y un conjunto de componentes comunes de interfaz.
- **Carpeta** `src/controllers/`**:** contiene la lógica de negocio. Cada controlador conecta las pantallas con los datos: valida la información, coordina los repositorios y devuelve a la vista un resultado listo para mostrar.
- **Carpeta** `src/utils/`**:** contiene las reglas y los cálculos que no dependen de la interfaz, como

el índice de masa corporal, las metas calóricas, el generador de rutinas, las estadísticas de entrenamiento y el reporte de progreso.

- **Carpeta** `src/models/`**:** contiene las entidades del dominio (usuario, máquina, rutina, entrenamiento, registro nutricional y progreso físico) y los repositorios, que son los únicos archivos que conocen la estructura de las colecciones de Firestore.
- **Carpeta** `src/schemas/`**:** contiene los esquemas de validación con Zod que verifican los datos antes de guardarlos y al recibirlos de servicios externos.
- **Carpeta** `src/services/`**:** contiene los archivos que se comunican con servicios externos: la conexión con Firebase (autenticación, Firestore, Storage, Cloud Functions y App Check), las API de wger y de USDA, los modelos de inteligencia artificial, la interfaz de realidad aumentada y la generación del reporte en PDF.
- **Carpetas** `src/hooks/` **y** `src/store/`**:** contienen la lógica de las pantallas y las suscripciones en tiempo real a Firebase, y el estado global de la sesión, del onboarding y de los avisos (Zustand).
- **Carpeta** `modules/coach-ar/`**:** contiene el módulo nativo escrito en Kotlin que usa ARCore para anclar la ficha de la máquina reconocida sobre la imagen de la cámara.
- **Carpeta** `functions/`**:** contiene las Cloud Functions del servidor (Node.js 22): la búsqueda de alimentos en USDA, el guardado de rutinas, el reconocimiento de máquinas con el detector de Ultralytics y el control de cuotas de uso. Las claves de los servicios externos se guardan solo en el servidor.
- **Carpeta** `ml/`**:** contiene los *scripts* en Python con los que se entrenaron, evaluaron y exportaron los modelos de recomendación de rutinas y de reconocimiento de máquinas, junto con sus métricas.
- **Carpeta** `assets/`**:** contiene las imágenes necesarias para el proyecto, logo de la app, etc.
- **Carpeta** `tests/security/`**:** contiene las pruebas de las reglas de seguridad de Firestore y Storage, que se ejecutan con el emulador de Firebase.

4.3.2. Iteración 1: Autenticación, registro y onboarding

4.3.2.1. Pantalla de inicio de sesión.

Esta pantalla permite iniciar sesión con correo electrónico y contraseña o iniciar sesión con una cuenta registrada o de Google. El controlador de autenticación delega la gestión de las credenciales a Firebase Authentication. Al registrar una cuenta con correo electrónico, la aplicación crea el perfil del usuario en Cloud Firestore. Cuando el usuario inicia sesión con Google, la aplicación verifica si ya existe un perfil asociado a su cuenta. Si lo encuentra, conserva los datos registrados. Si es el primer acceso, crea el perfil correspondiente (Código 4-2).

```
async registrar(datos: DatosRegistro): Promise<string> {
  const credencial = await createUserWithEmailAndPassword(
    auth, datos.correo, datos.contrasena);
  const uid = credencial.user.uid;
  await crearPerfilUsuario(uid, {
    nombre: datos.nombre, apellidos: datos.apellidos,
    genero: datos.genero, email: datos.correo,
  });
  return uid;
},

async iniciarSesionConGoogle(idToken: string): Promise<UsuarioSesion> {
  const credencial = GoogleAuthProvider.credential(idToken);
  const { user } = await signInWithCredential(auth, credencial);
  await AuthController.asegurarPerfil(user.uid, {
    nombre: user.displayName, email: user.email,
  });
  return aUsuarioSesion(user);
```

```
19 },
```

Código 4-2. Registro e inicio de sesión con Google en `AuthController`

A continuación, se detalla la lógica de integración para la autenticación: La autenticación comienza cuando el usuario selecciona su cuenta mediante Google Sign-In. Si la operación es exitosa, la aplicación obtiene un token de identidad (idToken) y lo utiliza para generar una credencial con la que inicia sesión en Firebase Authentication. Una vez autenticado el usuario, la aplicación comprueba si existe un perfil asociado a su cuenta en Cloud Firestore. Si el perfil ya existe, conserva los datos registrados y dirige al usuario a la pantalla correspondiente según el estado de su perfil. Si es su primer acceso, crea el perfil y lo dirige a la pantalla de configuración inicial.

Los formularios se validan con esquemas Zod antes de llamar al controlador: el registro exige nombre y apellido, un correo válido, una contraseña de al menos seis caracteres y la coincidencia con su confirmación.

Estado de la sesión. Un *store* de Zustand mantiene el estado global. En el diseño raíz, `useAuthListener` observa la sesión de Firebase y `useProfileListener` consulta el perfil en tiempo real. El acceso a `(app)` se habilita cuando el perfil contiene objetivo y nivel; en caso contrario, el usuario pasa al *onboarding* (Figura 4-5).

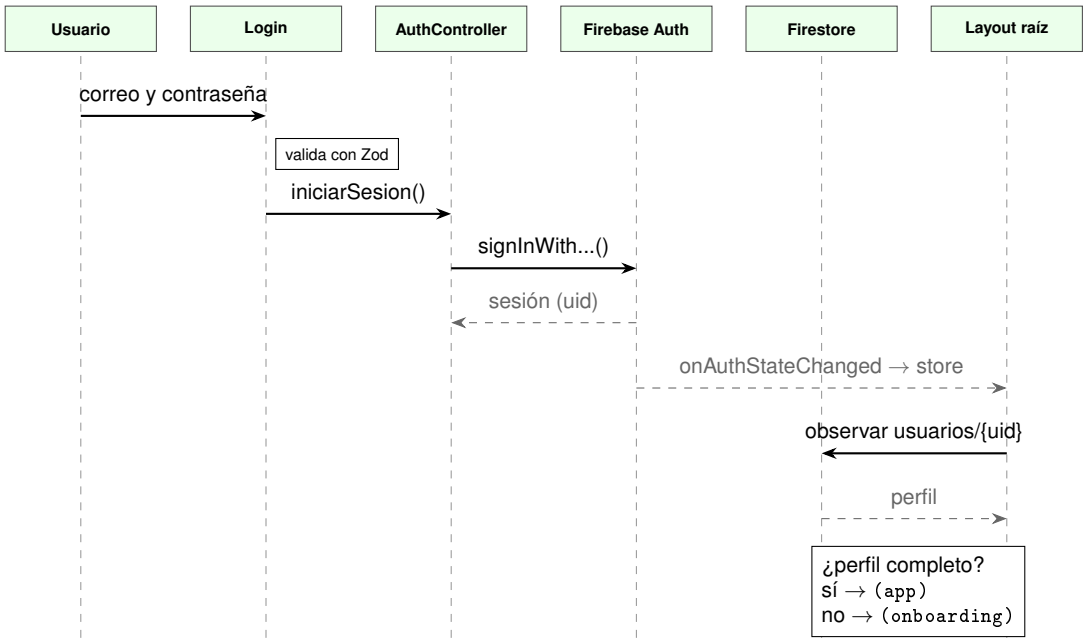


Figura 4-5. Diagrama de secuencia del inicio de sesión

Fuente: Elaboración propia.

Configuración inicial del perfil. El *onboarding* reúne tres pasos: elección del objetivo y nivel, registro de datos físicos y selección del tipo de rutina. Las pantallas comparten un borrador temporal en Zustand; al finalizar, el controlador guarda los datos en Firestore y habilita el acceso a la aplicación. La opción de rutina elegida se abre desde el inicio. Los datos del perfil se utilizan en el generador de rutinas.

A continuación se presentan las *Story Cards* de la primera iteración: la Tabla 4-7 corresponde al inicio de sesión y la Tabla 4-8, al registro de usuario y la configuración de su perfil.

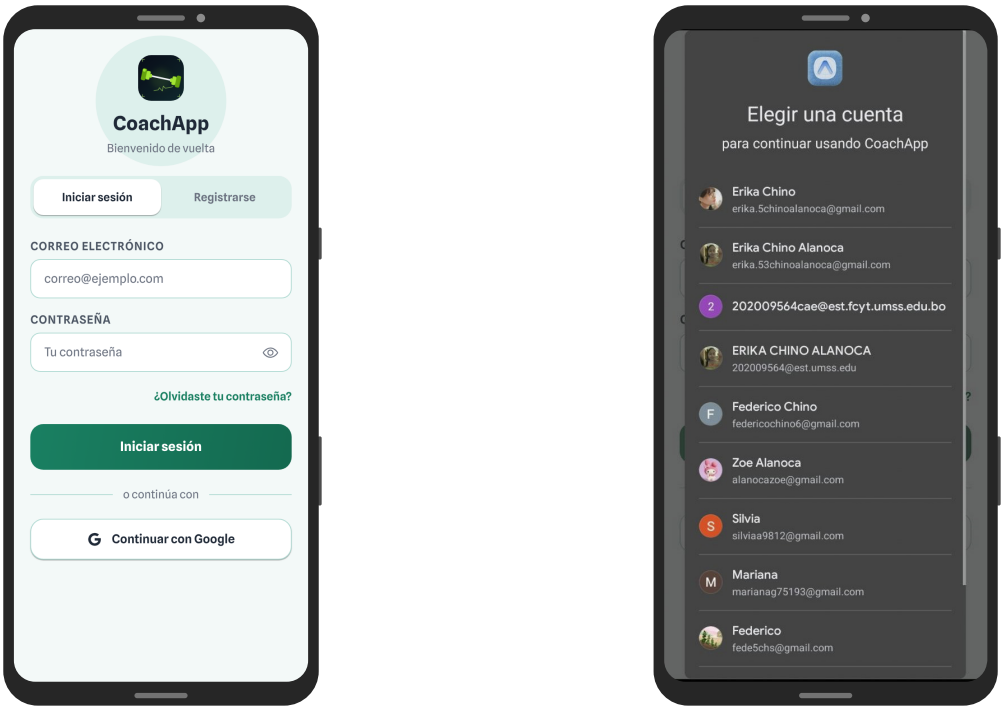
4.3.2.2. Planificación

Story Card	Nombre del proyecto	Estimación
01	CoachApp	20 horas
Nombre. Inicio de sesión		**Fecha.** 10/03/2026
Descripción		
El usuario puede iniciar sesión con su correo electrónico y contraseña o con su cuenta de Google. Tras autenticarse, la aplicación consulta su perfil en Cloud Firestore: si está completo, lo dirige a la pantalla principal; si está incompleto, lo dirige a la configuración inicial.		
Prueba de aceptación		
El usuario intenta iniciar sesión con Google.		
El usuario intenta iniciar sesion con correo y contraseña.		
Iniciar sesión con un perfil incompleto y verificar que la aplicación dirige al usuario a la configuración inicial y restringe el acceso a las pantallas principales hasta completar el perfil.		
Excepciones		**Riesgo.** Alto
Si el correo o la contraseña son incorrectos, o el usuario cancela el acceso con Google, se muestra el mensaje de error correspondiente y no se inicia la sesión.		

Tabla 4-7. Story Card 01: Inicio de sesión

Fuente: Elaboración propia.

4.3.2.3. Codificación



(a) Inicio de sesión (b) Inicio de sesion con Google

Figura 4-6. Pantallas de inicio de sesión con correo y contraseña o Google

Fuente: Elaboración propia.

4.3.2.4. Pantalla de registro.

En esta pantalla el usuario crea una cuenta ingresando su nombre, correo electrónico y contraseña. Firebase Authentication registra la cuenta y la aplicación crea el perfil del usuario en Cloud

Firestore (Código 4-2).

4.3.2.5. Pantalla de registro.

En esta pantalla el usuario crea una cuenta ingresando su nombre, apellido, correo electrónico y contraseña. El formulario se valida con un esquema Zod: exige nombre y apellido, un correo válido, una contraseña de al menos seis caracteres y que coincida con su confirmación. Luego, Firebase Authentication registra la cuenta y la aplicación crea el perfil del usuario en Cloud Firestore.

```
export const registroFormSchema = z
  .object({
    nombre: z.string().trim().min(1, "Ingresa tu nombre").max(100),
    apellidos: z.string().trim().min(1, "Ingresa tu apellido").max(100),
    correo: z.string().trim().email("Correo electrónico inválido"),
    contrasena: z.string().min(6, "Mínimo 6 caracteres"),
    confirmarContrasena: z.string(),
  })
  .refine((d) => d.contrasena === d.confirmarContrasena, {
    message: "Las contraseñas no coinciden",
    path: ["confirmarContrasena"],
  });

async registrar(datos: DatosRegistro): Promise<string> {
  const credencial = await createUserWithEmailAndPassword(
    auth, datos.correo, datos.contrasena);
  const uid = credencial.user.uid;
  await crearPerfilUsuario(uid, {
    nombre: datos.nombre, apellidos: datos.apellidos,
    genero: datos.genero, email: datos.correo,
  });
  return uid;
},
```

Código 4-3. Validación del formulario y registro en `AuthController`

4.3.2.6. Planificación

Story Card	**Nombre del proyecto**	**Estimación**
02	CoachApp	20 horas
Nombre. Registro de usuario y configuración del perfil		**Fecha.** 14/03/2026
Descripción		
El usuario puede crear una cuenta con correo electrónico y contraseña. En el primer acceso completa su perfil con su objetivo, su nivel de entrenamiento y sus datos físicos. Luego puede generar una rutina automática, planificar una propia o continuar sin rutina. Desde su perfil puede actualizar sus datos y cerrar sesión.		
Prueba de aceptación		
Registrar al usuario con un nombre, un correo y una contraseña válidos. Comprobar que se crean la cuenta y el perfil en Cloud Firestore.		
Ingresar dos contraseñas diferentes en el campo contraseña y en confirmar contraseña.		
Ingresar datos invalidos en los campos de registro.		
Excepciones		**Riesgo.** Alto
Si el correo es inválido, las contraseñas no coinciden o la contraseña tiene menos de seis caracteres, se muestra el mensaje de validación correspondiente. Durante la configuración inicial, los datos físicos fuera de los rangos permitidos se rechazan y deben corregirse antes de continuar.		

Tabla 4-8. Story Card 02: Registro de usuario

Fuente: Elaboración propia.

4.3.2.7. Codificación

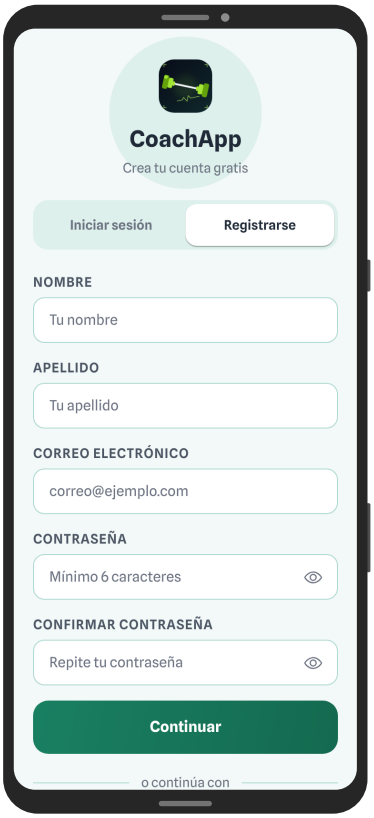


Figura 4-7. Pantalla de registro para usuarios nuevos

Fuente: Elaboración propia.

4.3.2.8. Pantalla de onboarding.

Tras registrarse o iniciar sesión por primera vez, el usuario completa su perfil en tres pasos. En el primero elige su objetivo (perder peso, ganar masa, mantenimiento o resistencia) y su nivel de experiencia (principiante, intermedio o avanzado). En el segundo ingresa su género, edad, altura, peso y los días por semana que puede entrenar. En el tercero elige cómo obtener su primera rutina: generarla con inteligencia artificial, crear una personalizada o continuar sin rutina. Estos datos son la entrada del generador de rutinas de la iteración.

```
export const datosFisicosSchema = z.object({
  genero: generoUsuarioSchema,
  edad: edadSchema,                          // 14 a 90 años
  altura: alturaSchema,                      // 50 a 300 cm
  pesoActual: pesoSchema,                    // 10 a 500 kg
  diasDisponibles: diasDisponiblesSchema, // 2 a 6 días
});

async guardarDatosOnboarding(uid: string, datos: DatosOnboardingInput) {
  const resultado = datosOnboardingSchema.safeParse(datos);
  if (!resultado.success) throw new Error("Datos de onboarding inválidos");
  await AuthController.asegurarPerfil(uid, {
    nombre: auth.currentUser?.displayName ?? null,
    email: auth.currentUser?.email ?? null,
  });
  await actualizarPerfilUsuario(uid, resultado.data);
},

esPerfilCompleto(perfil: Usuario | null): boolean {
  return !!perfil?.objetivo && !!perfil?.nivel;
```

```
21  },
```

Código 4-4. Validación y guardado de los datos del onboarding

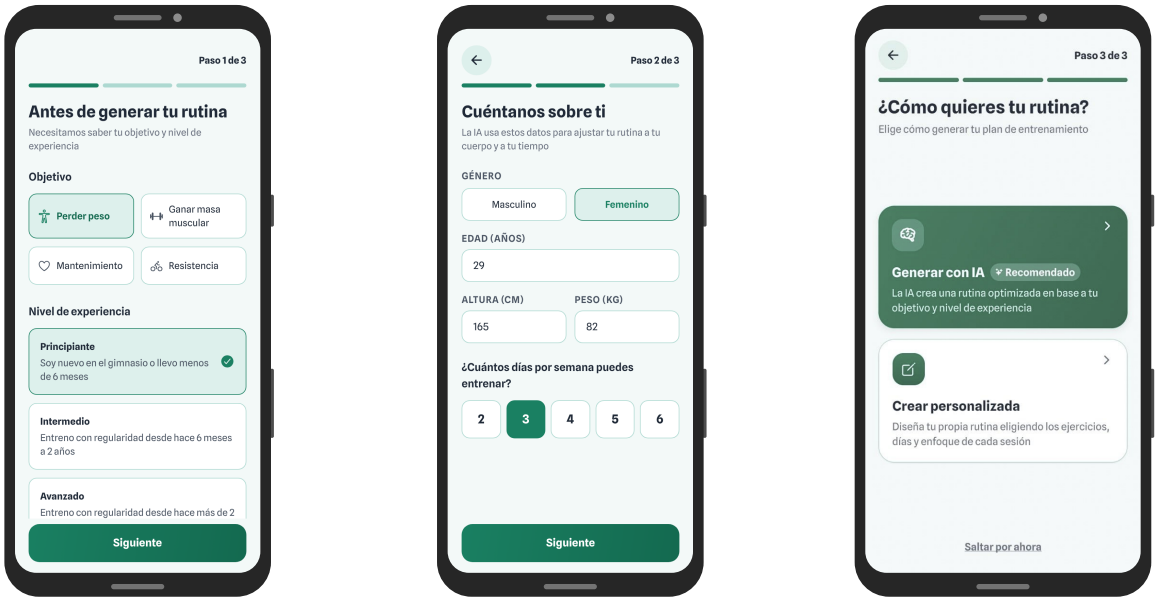
4.3.2.9. Planificación

Story Card	Nombre del proyecto	Estimación
03	CoachApp	15 horas
Nombre. Configuración inicial del perfil		**Fecha.** 19/03/2026
Descripción		
En su primer acceso, el usuario indica su objetivo, su nivel de entrenamiento y sus datos físicos, y elige si desea generar una rutina con inteligencia artificial, crear una propia o continuar sin rutina. Al terminar, la aplicación guarda el perfil y habilita las pantallas principales.		
Prueba de aceptación		
Elegir un objetivo y un nivel, avanzar e ingresar datos físicos válidos. Comprobar que se guardan en el perfil del usuario en Cloud Firestore.		
Probar las tres opciones del último paso. Verificar que se abre el generador de rutinas, el planificador o la pantalla de Inicio, según la opción elegida.		
Cerrar la aplicación a mitad del proceso y volver a abrirla. Verificar que los datos ya ingresados se conservan.		
Excepciones		**Riesgo.** Alto
Si la edad, la altura, el peso o los días disponibles están fuera del rango permitido, o falta elegir el género, se muestra el mensaje de validación correspondiente y no se avanza. Si no se puede guardar el perfil, por ejemplo por falta de conexión, se muestra un aviso y el usuario permanece en el último paso.		

Tabla 4-9. Story Card 03: Configuración inicial del perfil

Fuente: Elaboración propia.

4.3.2.10. Codificación



(a) Objetivo y nivel (b) Datos físicos (c) Rutina inicial

Figura 4-8. Pantallas del *onboarding*

Fuente: Elaboración propia.

4.3.3. Iteración 2: Guía de máquinas y ejercicios

En esta pantalla

4.3.3.1. Planificación

Story Card	**Nombre del proyecto**	**Estimación**
05	CoachApp	Por completar
Nombre. Guía de máquinas y ejercicios		**Fecha.** Por completar
Descripción		
El usuario puede buscar una máquina por su nombre o por el músculo que trabaja, aunque no escriba las tildes, y filtrar el catálogo por grupo muscular y por tipo de equipo. Al abrir la ficha de una máquina, el principiante ve paso a paso cómo se usa, los errores que debe evitar y las pautas para prevenir lesiones. También ve una animación de cada ejercicio con su músculo objetivo y su dificultad, y una galería con distintos modelos comerciales de la máquina para reconocerla en su gimnasio.		
Prueba de aceptación		
Buscar “biceps” o “prensa” y comprobar que se devuelven las mismas máquinas que al buscar “bíceps” o “Prensa”.		
Aplicar los filtros de grupo muscular y de tipo de equipo, solos y combinados entre sí y con la búsqueda, y verificar que el catálogo muestra solo las máquinas que cumplen todos los criterios.		
Abrir cada una de las 35 fichas y verificar que muestra la guía de uso, los errores comunes y las pautas de seguridad.		
Verificar que cada máquina presenta al menos un ejercicio con animación, músculo objetivo y dificultad.		
Abrir la galería de una máquina y comprobar que muestra hasta ocho variantes comerciales.		
Excepciones		**Riesgo.** Por evaluar
Si la búsqueda o la combinación de filtros no coincide con ninguna máquina, el catálogo queda vacío.		

Tabla 4-10. Story Card de la guía de máquinas y ejercicios

Fuente: Elaboración propia.

4.3.3.2. Codificación

Construcción del catálogo. El contenido de la guía se preparó fuera de la aplicación mediante un proceso de cuatro pasos (Figura 4-9):

1. **Selección de máquinas:** a partir del catálogo de un proveedor nacional se definieron 35 tipos de máquina, que agrupan 288 modelos comerciales como variantes; se excluyeron los accesorios (agarres, barras y soportes).
2. **Mapeo de ejercicios:** cada tipo se asoció manualmente con sus ejercicios en wger (wger Project, s. f.; 98 ejercicios) y en free-exercise-db (Yuhonas, s. f.; 91 ejercicios). El mapeo se hizo a mano porque la coincidencia automática por nombre producía asociaciones incorrectas.
3. **Contenido propio:** para cada máquina se redactaron la descripción, los músculos principales y secundarios, la guía de uso, los errores comunes y las pautas de prevención de lesiones, contrastando la información con guías de entrenamiento reconocidas sin copiar sus textos.
4. **Animaciones:** se asociaron 95 animaciones GIF de WorkoutX, al menos una por máquina, descargadas mediante su API oficial y respetando sus condiciones de uso.

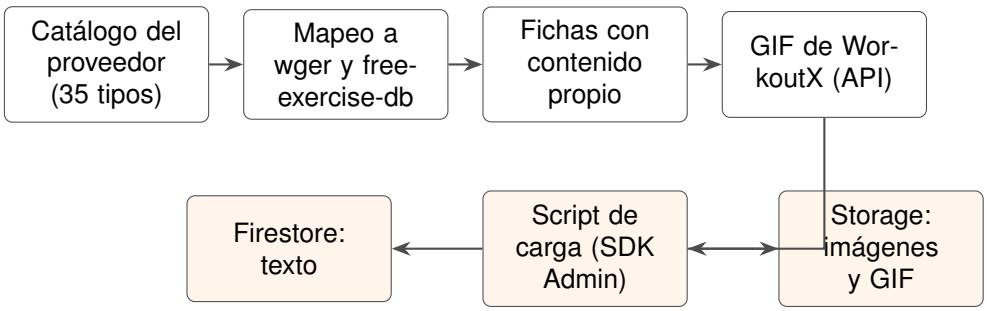


Figura 4-9. Proceso de construcción y carga del catálogo de máquinas

Fuente: Elaboración propia.

Un *script* carga los textos en Firestore y los archivos multimedia en Storage. Puede ejecutarse nuevamente sin duplicar los documentos. Cada ejercicio conserva los identificadores de su fuente para reconocer el origen del contenido.

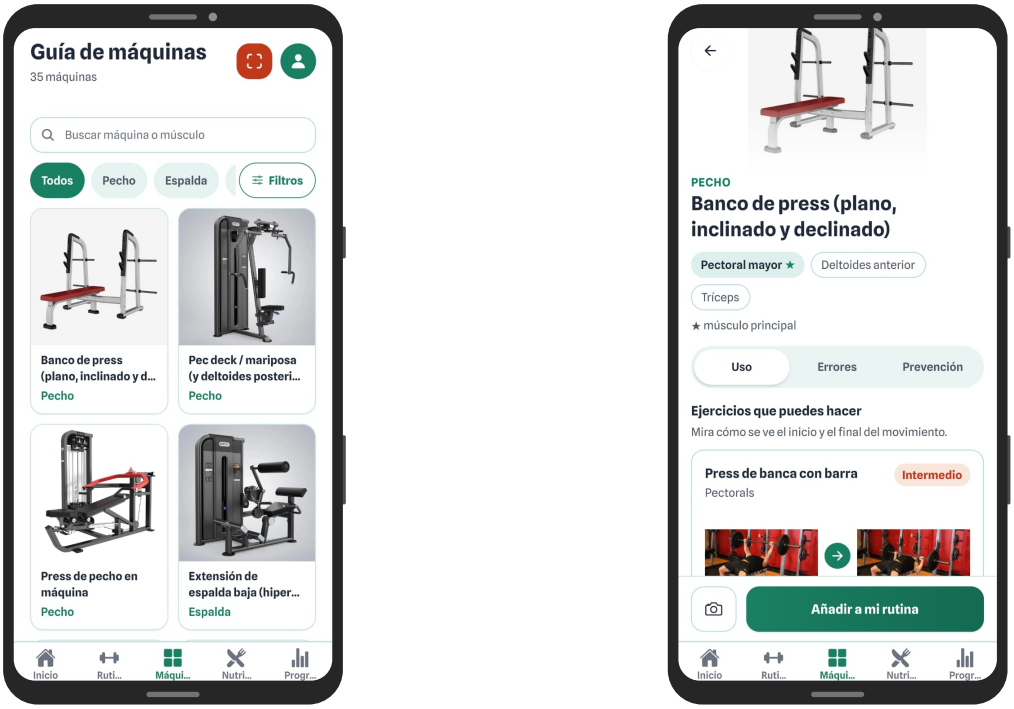
Consulta y filtrado. `machineRepository` descarga los 35 documentos del catálogo una vez por sesión y los mantiene en memoria. Antes de utilizarlos, valida su estructura con Zod y descarta los que no cumplen el esquema. La búsqueda y los filtros se resuelven sobre esa copia local, sin nuevas consultas a la red. El filtrado ignora mayúsculas y tildes (Código 4-5).

```
filtrar(catalogo: Maquina[], filtros: FiltrosMaquinas): Maquina[] {
  const texto = filtros.busqueda ? normalizarTexto(filtros.busqueda) : "";
  return catalogo.filter((m) => {
    if (filtros.categoria && m.categoria !== filtros.categoria) return false;
    if (filtros.tipo && m.tipo !== filtros.tipo) return false;
    if (!texto) return true;
    const campos = [m.nombre, m.categoria, ...m.musculosPrincipales,
      ...m.musculosSecundarios, ...m.ejercicios.map((e) => e.nombre),
      ...m.ejerciciosGif.map((e) => e.nombre)];
    return campos.some((campo) => normalizarTexto(campo).includes(texto));
  });
},
```

Código 4-5. Regla de filtrado del catálogo en `MachinesController`

Imágenes desde Storage. Las rutas de Storage se convierten en URL de descarga mediante `storageService`, que almacena en caché cada URL ya resuelta. En la interfaz, el componente `ImagenStorage`, construido sobre `expo-image`, muestra tanto imágenes estáticas como animaciones GIF.

Interfaz de la guía. El catálogo permite buscar y filtrar las máquinas por grupo muscular y tipo de equipo. Al seleccionar una, se abre su ficha con los ejercicios, las animaciones, las instrucciones de uso, los errores comunes y las pautas de seguridad. La ficha también incluye variantes comerciales y un acceso al escáner.



(a) Catálogo con filtros (b) Ficha de máquina

Figura 4-10. Guía de máquinas

Fuente: Elaboración propia.

4.3.4. Iteración 3: Escáner de máquinas con superposición de información

4.3.4.1. Planificación

Story Card	**Nombre del proyecto**	**Estimación**
05	CoachApp	Por completar
Nombre. Escáner de máquinas con superposición de información		**Fecha.** Por completar
Descripción		
El usuario enfoca una máquina con la cámara y ve su información sobre la imagen real. La aplicación reconoce la máquina a partir de una foto y, si no está segura, el usuario puede confirmarla o elegirla de una lista sin salir del escáner. También puede abrir el escáner desde la ficha de una máquina. El reconocimiento usa un modelo más capaz que indica dónde está la máquina en la imagen, sin que el acceso a la guía dependa de que el servicio esté disponible. En un teléfono compatible con ARCore, la información queda fijada en el lugar donde está la máquina cuando se mueve la cámara.		
Prueba de aceptación		
Con el permiso concedido, comprobar que se muestra la cámara en vivo con el marco de encuadre y que la ficha resumida se superpone sobre la imagen real.		
Con confianza suficiente, verificar que la máquina pasa directamente al estado *detectada*; si no, que se propone como sugerencia y que la lista con búsqueda permite confirmarla o elegir cualquiera de las 35 máquinas.		
Abrir el escáner desde la ficha de una máquina y comprobar que se muestra directamente sobre la cámara.		
Con el modo en la nube activo, verificar que la imagen se envía a la función de reconocimiento y que se devuelven hasta tres candidatas con su caja.		
En un teléfono con ARCore, confirmar la máquina y comprobar que su nombre e instrucción se colocan sobre el punto de la escena detectado.		
Excepciones		**Riesgo.** Por evaluar
Si el servicio de reconocimiento falla, se usa el modelo del dispositivo; si no reconoce ninguna máquina del catálogo, se pasa a la selección manual. Si no hay superficie detectable, la aplicación pide tocar una; en un teléfono sin ARCore se ofrece la cámara convencional.		

Tabla 4-11. Story Card del escáner de máquinas con superposición de información

Fuente: Elaboración propia.

4.3.4.2. Codificación

Funcionamiento del escáner. El escáner utiliza cinco estados para organizar la captura, el reconocimiento y la confirmación de la máquina (Figura 4-11). Una vez autorizado el acceso a la cámara, el usuario toma una fotografía y la aplicación la envía al servicio de reconocimiento. Al confirmar el resultado, se muestran los datos de la máquina y un acceso a su ficha. La identificación se realiza sobre la fotografía, no de forma continua sobre el vídeo.

En los dispositivos compatibles, `CoachAR` permite fijar la información en el espacio mediante ARCore. En los demás, `expo-camera` muestra la ficha superpuesta sobre la imagen. Esta distinción corresponde al alcance definido en las secciones 2.6.3 y 4.1.5.

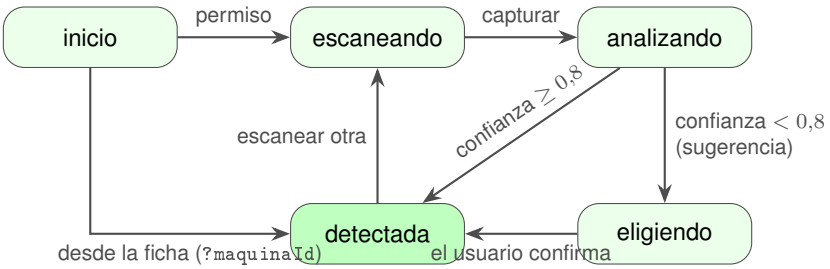


Figura 4-11. Diagrama de estados del escáner de máquinas

Fuente: Elaboración propia.

Reconocimiento local. La función `reconocerMaquina` recorta la fotografía al cuadrado central, la reduce a 224×224 píxeles y ejecuta el clasificador de TensorFlow Lite en el dispositivo. Devuelve la máquina más probable, su confianza y dos alternativas. Si la confianza alcanza 0,8, el controlador acepta el resultado; por debajo de ese umbral, el usuario confirma la sugerencia o selecciona otra máquina. Si el modelo falla, se mantiene la selección manual (Código 4-6).

```
// services/ai/machineRecognition.ts
export interface ResultadoReconocimiento {
  maquinaId: string;
  confianza: number; // 0..1
  caja?: CajaDeteccion; // solo en el modo nube: x1, y1, x2, y2 en 0..1
  alternativas: { maquinaId: string; confianza: number; caja?: CajaDeteccion }[];
}
export const CONFIANZA_MINIMA = 0.8; // calibrado en validación cruzada
export async function reconocerMaquina(
  fotoUri: string,
): Promise<ResultadoReconocimiento | null> { /* ... TFLite ... */ }

// controllers/MachinesController.ts
async reconocerDesdeFoto(fotoUri: string): Promise<ResultadoEscaneo | null> {
  const resultado = await reconocerMaquina(fotoUri);
  if (!resultado) return null;
  const maquina = await obtenerMaquina(resultado.maquinaId);
  if (!maquina) return null;
  return { maquina, confianza: resultado.confianza,
    requiereConfirmacion: resultado.confianza < CONFIANZA_MINIMA };
},
```

Código 4-6. Contrato del reconocimiento de máquinas y su uso en el controlador

Modelo local. Se implementó un clasificador de 35 máquinas mediante aprendizaje por transferencia con MobileNetV2. Sus características visuales alimentan un clasificador de regresión logística, convertido en una capa densa con activación *softmax*. El modelo se exportó a TensorFlow Lite con una entrada RGB de 224×224 píxeles y un tamaño de 4,6 MB. El entrenamiento empleó fotografías de catálogo y aumento de datos mediante cambios de fondo, encuadre, brillo y desenfoque.

Reconocimiento en la nube con Ultralytics. El detector final es YOLO26m, entrenado durante 80 épocas con imágenes de 640×640 píxeles y 52 clases anotadas en Ultralytics Platform. Su salida se mapea a las 35 máquinas del catálogo. La distribución de datos, la versión inicial y los resultados se detallan en la sección 4.4.1.4.

Integración del reconocimiento. El modo en la nube se activa con `EXPO_PUBLIC_RECONOCIMIENTO_NUBE`. La aplicación reduce la fotografía a 640 píxeles en su lado mayor, sin recortarla, y la envía a la Cloud Function `reconocerMaquina`. Esta función consulta el detector publicado en Ultralytics Platform y mantiene la clave de acceso en el servidor.

La respuesta contiene la clase, la confianza y la caja de cada detección. La función convierte las clases en identificadores del catálogo, normaliza las coordenadas y devuelve hasta tres candidatas. Si hay varias detecciones de una misma máquina, conserva la de mayor confianza. Cuando el servicio no responde, la aplicación recurre al clasificador local y, si este tampoco produce un resultado, permite elegir la máquina manualmente.

Realidad aumentada con ARCore. El anclaje espacial se implementó en `modules/coach-ar/`, un módulo local de Expo escrito en Kotlin que expone la vista nativa `CoachAR`. El módulo utiliza ARCore 1.52.0 y comparte la sesión de cámara con la captura enviada al detector. En equipos sin soporte se utiliza la cámara convencional con información superpuesta. La Figura 4-12 resume el proceso de reconocimiento y anclaje.

1. Al pulsar «Identificar máquina», la vista toma del fotograma actual de ARCore la imagen de la cámara en formato YUV y sus parámetros intrínsecos, y crea un ancla temporal en la pose de la cámara en ese instante (Google for Developers, s. f.-d).
2. La imagen se convierte a JPEG, se gira a la orientación de la pantalla y se envía al reconocimiento en la nube, que devuelve las candidatas con sus cajas.
3. Cuando la persona confirma una candidata, el escáner entrega a la vista únicamente la caja de esa máquina y su ficha. El código nativo calcula el centro de la caja, lo convierte en un rayo óptico con los parámetros intrínsecos y lo expresa en el mundo con la pose del ancla temporal, no con la del teléfono en ese momento. Esto importa porque la respuesta de la red tarda y el teléfono se habrá movido: con la pose actual, el rayo apuntaría a un lugar distinto del que se fotografió.
4. ARCore ejecuta una prueba de impacto con ese rayo. Solo se aceptan impactos sobre geometría con seguimiento: planos dentro de su polígono, puntos de característica con normal estimada o, si el dispositivo la admite, puntos de profundidad, a una distancia de entre 0,15 y 8 metros (Google for Developers, s. f.-b, s. f.-d). Con el impacto se crea el ancla definitiva y se libera la temporal.
5. En cada fotograma, el nombre y la instrucción de la máquina se proyectan a la pantalla con las matrices de vista y proyección de ARCore. La etiqueta se oculta si queda detrás de la cámara o fuera del encuadre, o si se pierde el seguimiento.

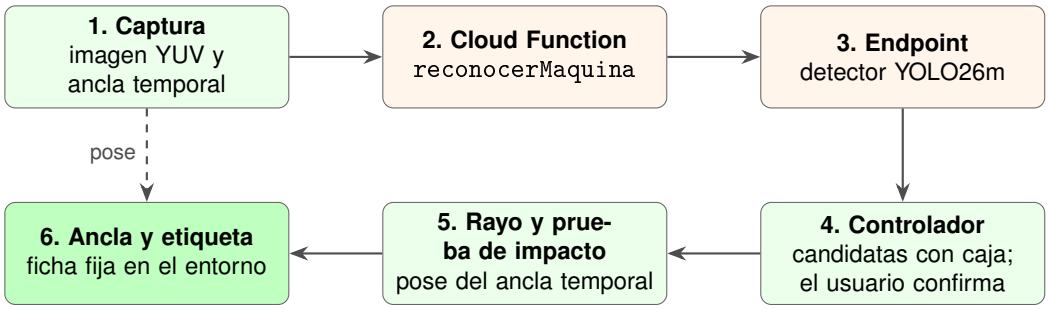
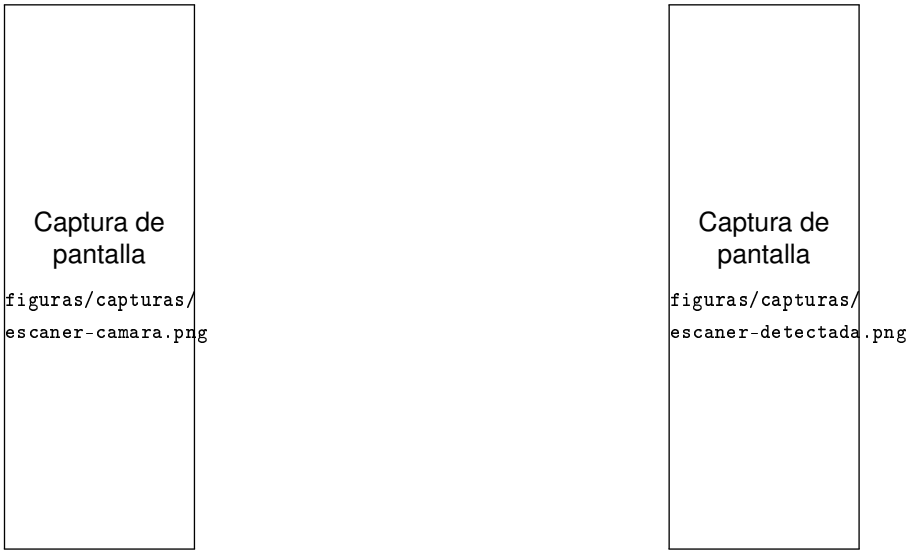


Figura 4-12. Flujo de reconocimiento y anclaje espacial de la ficha

Fuente: Elaboración propia.

Si no se encuentra una superficie, el usuario coloca la información mediante un toque. Esta alternativa también se utiliza cuando no hay caja de detección o la captura supera los 90 segundos. El ancla permanece fija en el entorno y no sigue el movimiento de la máquina. Al salir o pasar a segundo plano se pausa la sesión, se liberan las anclas y se descartan las respuestas anteriores. Las fotografías y poses no se guardan en Firestore.



(a) Escaneo (b) Ficha superpuesta

Figura 4-13. Escáner de máquinas con superposición de información

Fuente: Elaboración propia.

4.3.5. Iteración 4: Rutinas personalizadas y planificación

4.3.5.1. Planificación

Story Card	**Nombre del proyecto**	**Estimación**
06	CoachApp	Por completar
Nombre. Rutinas personalizadas y planificación		**Fecha.** Por completar
Descripción		
El usuario puede generar una rutina semanal adecuada a su objetivo y su nivel, o armar la suya día por día, eligiendo los ejercicios y sus series, repeticiones y descansos. Puede agregar a su rutina un ejercicio que ve en la ficha de una máquina, tener varias rutinas guardadas, elegir cuál es la activa y eliminar las que ya no usa. Al tocar un ejercicio de su rutina, abre la ficha de su máquina.		
Prueba de aceptación		
Para cada una de las doce combinaciones de objetivo y nivel, comprobar que la rutina generada tiene el número de días, la división y las variables de carga de las Tablas 4-13 a 4-15.		
Crear una rutina propia de 1 a 7 días y verificar que se guarda con los ejercicios, series, repeticiones y descansos elegidos.		
Agregar a la rutina un ejercicio desde la ficha de una máquina y comprobar que se agrega al día elegido.		
Guardar varias rutinas, marcar una como activa y verificar que es la que se muestra en el inicio; eliminar una rutina que ya no se usa.		
Tocar un ejercicio de la rutina asociado a una máquina y comprobar que se abre su ficha.		
Excepciones		**Riesgo.** Por evaluar
Si la rutina no tiene nombre, algún día no tiene ejercicios o hay valores fuera de rango, no se puede guardar y se muestra el motivo. Si el ejercicio ya existe en el día elegido, se informa y no se duplica.		

Tabla 4-12. Story Card de rutinas personalizadas y planificación

Fuente: Elaboración propia.

4.3.5.2. Codificación

Generador de rutinas basado en reglas. El componente inteligente del sistema es un generador de rutinas basado en conocimiento, cuya estructura sigue la de un sistema experto (sección 2.7):

- La **memoria de trabajo** contiene los hechos del usuario: su objetivo y su nivel y, cuando están registrados, su edad, su IMC y los días que puede entrenar, tomados del perfil.
- La **base de conocimiento** está formada por tres conjuntos de reglas derivadas de las pautas de prescripción del ejercicio (Tablas 4-13, 4-14 y 4-15).
- El **motor de inferencia** aplica las reglas en secuencia y produce un plan semanal, que luego se completa con ejercicios reales de la API de wger.

La primera regla determina la frecuencia semanal según la experiencia, siguiendo la progresión recomendada por Kraemer y Ratamess (2004).

Tabla 4-13. Reglas de frecuencia semanal según el nivel

Regla	Días por semana
SI nivel = principiante	3
SI nivel = intermedio	4
SI nivel = avanzado	5

Fuente: Archivo `utils/routineGenerator.ts`, con base en Kraemer y Ratamess (2004).

La Tabla 4-14 define las series, repeticiones y descansos asignados a cada objetivo.

Tabla 4-14. Reglas de variables de carga según el objetivo

Regla	Series	Repeticiones	Descanso
SI objetivo = ganar masa muscular	4	9	90 s
SI objetivo = mantenimiento	3	12	60 s
SI objetivo = perder peso	3	15	45 s
SI objetivo = resistencia	3	18	30 s

Fuente: Archivo `utils/routineGenerator.ts`, con base en Kraemer y Ratamess (2004) y Schoenfeld (2010).

La división semanal combina el objetivo y la frecuencia en seis plantillas (Tabla 4-15). Los bloques agrupan ejercicios de empuje, tirón, pierna, cardio y cuerpo completo. Las reglas de la Tabla 4-16 adaptan el plan a la edad, el IMC y la disponibilidad del usuario.

Tabla 4-15. Reglas de división semanal

Días	Objetivo de fuerza (ganar masa, mantenimiento)	Objetivo con énfasis cardio (perder peso, resistencia)
3	Empuje – Tirón – Pierna	Empuje – Cardio – Pierna
4	Empuje – Tirón – Pierna – Cuerpo completo	Empuje – Tirón – Cardio – Pierna
5	Empuje – Tirón – Pierna – Empuje – Tirón	Empuje – Tirón – Pierna – Cardio – Cuerpo completo

Fuente: Archivo `utils/routineGenerator.ts`.

Tabla 4-16. Reglas de ajuste según los datos físicos

Regla	Efecto
SI días disponibles $<$ días del nivel	Se usan los días disponibles; nunca se supera el máximo del nivel (3, 4 o 5).
SI IMC $\geq$ 30	Se elige la plantilla con énfasis cardio aunque el objetivo sea de fuerza.
SI edad $\geq$ 60 y esquema de hipertrofia o metabólico	Se reemplaza por el esquema general (intensidad moderada).
SI edad $<$ 16 y esquema de hipertrofia	Se reemplaza por el esquema general.

Fuente: Función `decidirClases` de `utils/routineGenerator.ts`; criterios de moderación por edad e IMC tomados de las pautas generales del ACSM y la OMS citadas en el código.

Cada bloque del plan se completa con entre cuatro y seis ejercicios de wger. El controlador distribuye la selección entre las categorías correspondientes y elige los ejercicios al azar, sin repetirlos. La Figura 4-14 presenta el proceso desde la lectura del perfil hasta el guardado de la rutina.

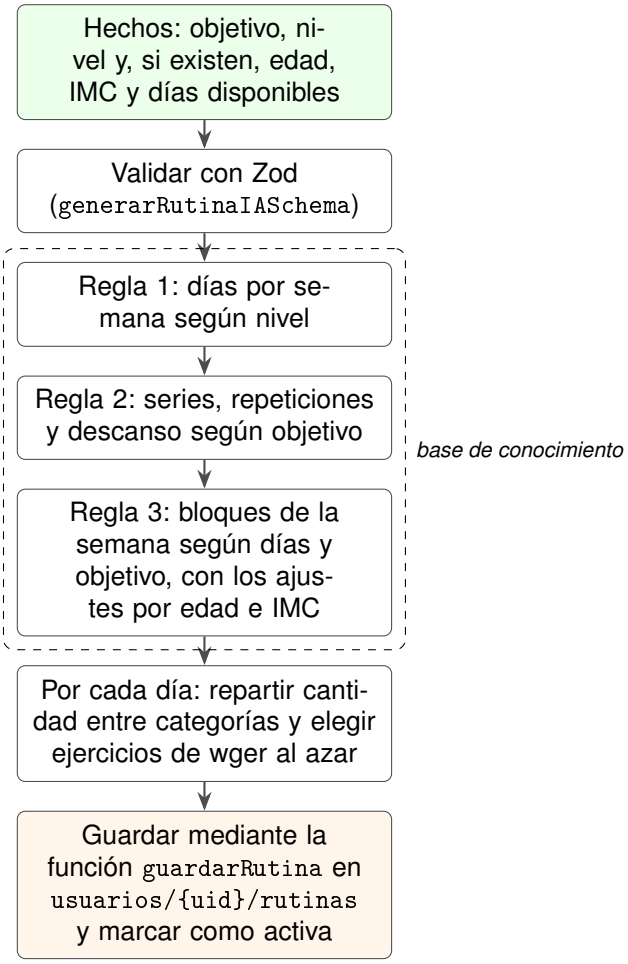


Figura 4-14. Flujo del generador de rutinas basado en reglas

Fuente: Elaboración propia.

La función `decidirClases` selecciona la plantilla y el esquema de carga; `construirPlan` produce el plan semanal. Ambas funciones se ejecutan sin acceso a la red (Código 4-7).

```
export function decidirClases(perfil: PerfilPlan): ClasesPlan {
  const { objetivo, nivel, edad, imc, diasDisponibles } = perfil;
  const maximoNivel = DIAS_POR_NIVEL[nivel];
  const dias = Math.min(diasDisponibles ?? maximoNivel, maximoNivel);
  const enfasisCardio =
    objetivo === "perder_peso" || objetivo === "resistencia" ||
    (imc !== undefined && imc >= UMBRAL_IMC_CARDIO);
  let esquema = ESQUEMA_POR_OBJETIVO[objetivo];
  // ... moderación por edad (>= 60 y < 16): esquema general ...
  return { indicePlantilla: indicePlantilla(dias, enfasisCardio),
           indiceEsquema: esquema };
}

export function generarPlanHeuristico(
  objetivo: ObjetivoUsuario, nivel: NivelExperiencia,
  datos: Omit<PerfilPlan, "objetivo" | "nivel"> = {},
): PlanHeuristico {
  const clases = decidirClases({ objetivo, nivel, ...datos });
  return construirPlan(clases.indicePlantilla, clases.indiceEsquema);
}
```

Código 4-7. Punto de entrada del generador basado en reglas

Recomendador de rutinas con TensorFlow Lite. Además de las reglas, la aplicación incluye un modelo de aprendizaje automático (versión 1.1.0) que decide la estructura de la rutina. El modelo **no genera la rutina completa**: predice dos clases, una plantilla semanal entre seis y un esquema de carga entre cuatro, que el código convierte en el plan con la misma función `construirPlan` que usan las reglas. Su entrada es un vector de diez características en un orden fijo, definido en `services/ai/features.ts` (edad, género en dos columnas, IMC, nivel, días disponibles y el objetivo codificado en cuatro columnas); los datos que faltan en el perfil reciben valores por defecto. La normalización va dentro del propio modelo para no duplicarla entre Python y TypeScript.

La arquitectura es un perceptrón multicapa pequeño (Figura 4-15): una capa de normalización, dos capas densas de 64 y 32 neuronas y dos cabezas con activación *softmax* (una de seis y otra de cuatro clases) cuyas salidas se concatenan en un único tensor de diez valores. Se entrenó con el *script* `ml/train_routine_model.py` y se exportó a un archivo de TensorFlow Lite de 16 672 bytes, cuya salida coincide al 100 % con la del modelo de Keras.

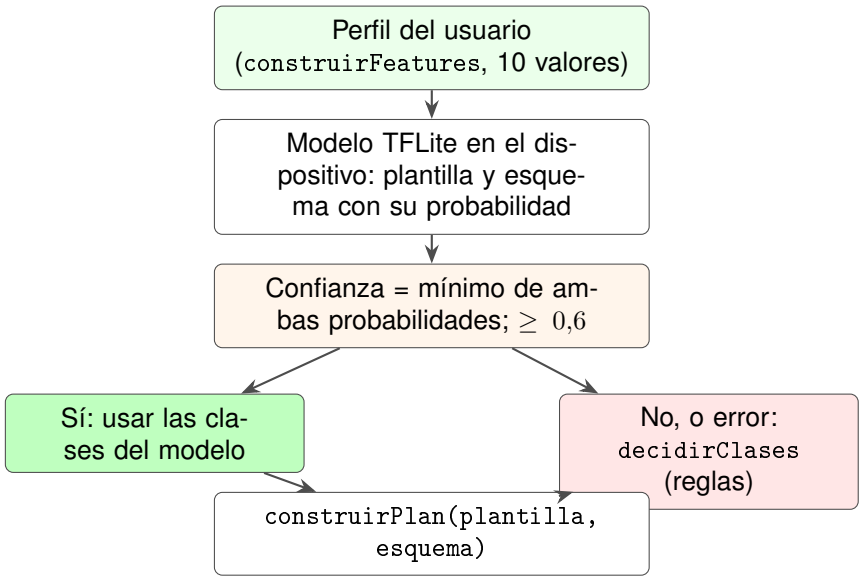


Figura 4-15. Selección de la estructura de la rutina con respaldo por reglas

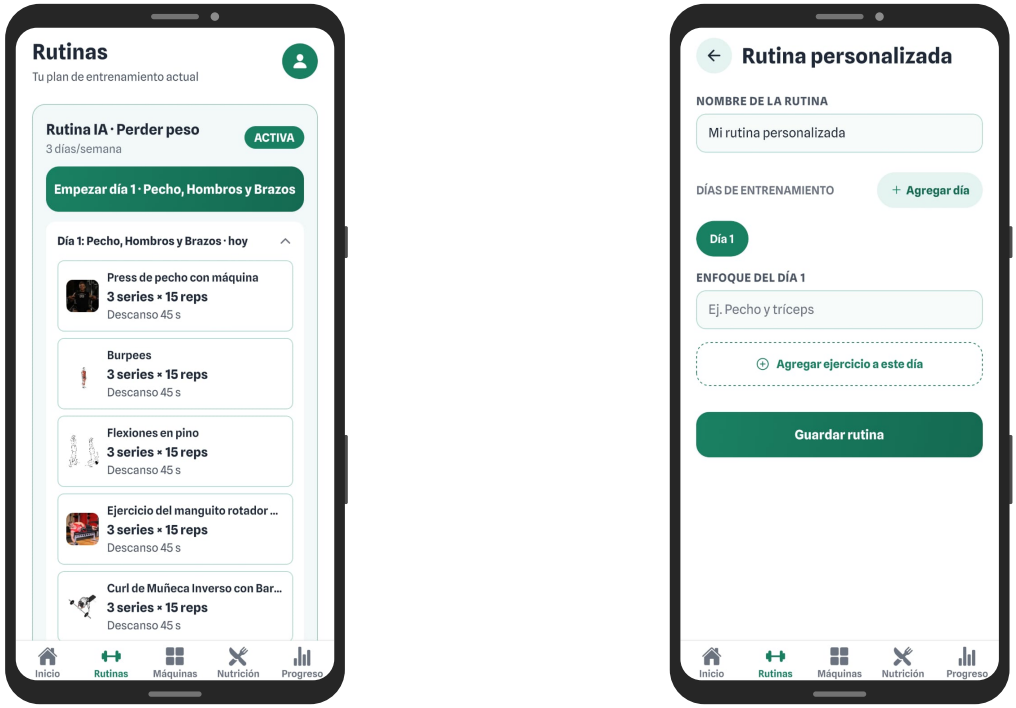
Fuente: Elaboración propia.

Datos de entrenamiento. No existen datos de usuarios reales con resultados conocidos, por lo que el conjunto de entrenamiento es **sintético**: se generaron 60 000 perfiles aleatorios (42 000 para entrenar, 9 000 para validar y 9 000 para probar) con edades de 14 a 90 años, IMC de 15 a 45, tres niveles, cuatro objetivos y de dos a seis días disponibles: el 40 % imita a usuarios típicos de gimnasio, el 35 % recorre todo el rango válido y el 25 % se concentra junto a los umbrales de las reglas. Cada perfil se etiquetó con una copia exacta de la función `decidirClases`. En consecuencia, el modelo aprende a *imitar* las reglas; su exactitud mide cuán bien las reproduce y **no** demuestra que las rutinas sean mejores para los usuarios que las producidas por las reglas.

Respaldo por reglas. La función `planificarRutina` (`services/ai/routinePlanner.ts`) ejecuta el modelo y toma el mínimo de las probabilidades de la plantilla y del esquema como confianza. Si la confianza es menor que 0,6, si el modelo no está disponible o si la inferencia falla, usa `decidirClases`. Ambos caminos terminan en `construirPlan`, por lo que la rutina tiene siempre la misma forma. La rutina guardada registra su origen (`modelo` o `heuristica`), la versión del modelo y la confianza, de modo que cada recomendación puede auditarse.

Rutina personalizada y planificación. Para el objetivo OE4 se construyó el planificador `routines/custom`, donde el usuario agrega días, asigna a cada uno un enfoque y elige ejercicios desde un modal de búsqueda, ajustando series, repeticiones y descanso. Los datos se validan con Zod con límites razonables (de 1 a 7 días, de 1 a 20 series, de 1 a 100 repeticiones y hasta 10 minutos de descanso). Desde la ficha de cualquier máquina, el botón “Agregar a Mi Rutina” permite añadir uno de sus ejercicios a un día de la rutina activa; el controlador rechaza duplicados en el mismo día.

La pantalla `routines/index` presenta el plan semanal de la rutina activa como una lista de días expandibles y permite cambiar la rutina activa, eliminar rutinas y crear nuevas. Las rutinas se observan en tiempo real con `onSnapshot`, por lo que cualquier cambio se refleja sin recargar (RNF3). La creación y la actualización de rutinas no se realizan mediante escritura directa en Firestore, sino a través de la función callable `guardarRutina`, que valida la estructura de los datos, asigna el propietario a partir de la identidad autenticada (y no del contenido enviado por el cliente) y limita la frecuencia de uso a 20 solicitudes por minuto y 200 por hora por usuario; las reglas de seguridad rechazan la creación directa de documentos en esa colección (Anexo B).

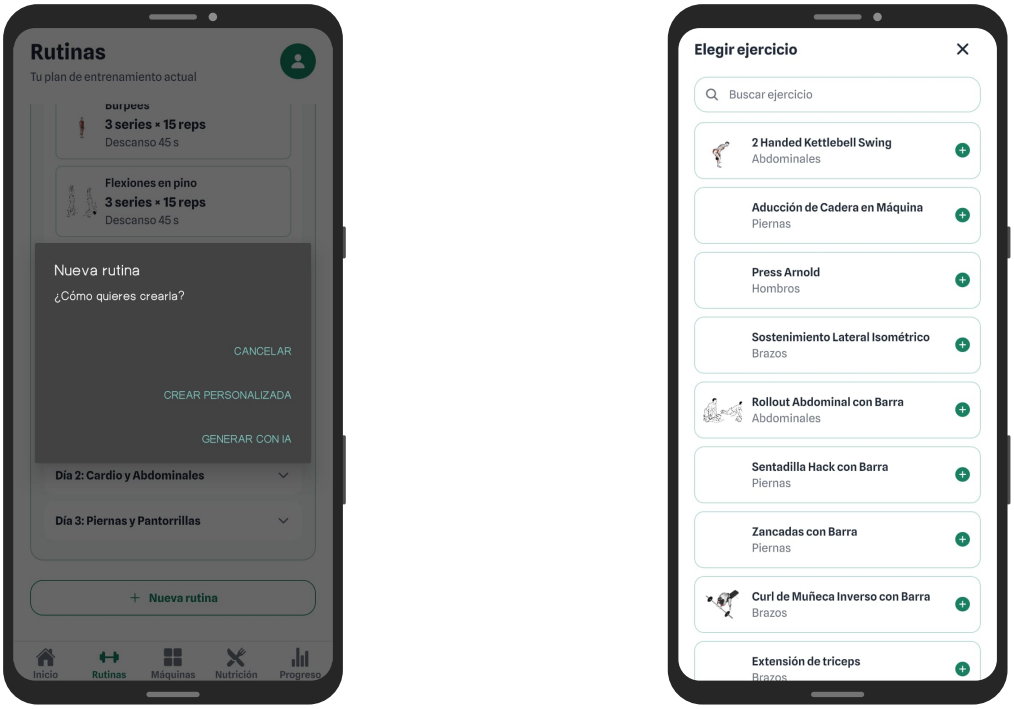


(a) Rutina generada (b) Planificador personalizado

Figura 4-16. Módulo de rutinas

Fuente: Elaboración propia.

La Figura 4-17 muestra la selección del tipo de rutina y el buscador de ejercicios del planificador.



(a) Elección del tipo de rutina (b) Selector de ejercicios

Figura 4-17. Creación de una rutina personalizada

Fuente: Elaboración propia.

4.3.6. Iteración 5: Nutrición, diario de alimentación e IMC

4.3.6.1. Planificación

Story Card	**Nombre del proyecto**	**Estimación**
07	CoachApp	Por completar
Nombre. Nutrición, diario de alimentación e IMC		**Fecha.** Por completar
Descripción		
El usuario ingresa su peso y su estatura y conoce su IMC y su clasificación; cada cálculo queda guardado en su historial. Puede buscar un alimento escribiendo en español e indicar cuántos gramos comió, agregar con un toque los alimentos que come a diario o ingresar uno manualmente. El resumen del día muestra las calorías y los macronutrientes consumidos frente a una meta calculada con sus datos físicos.		
Prueba de aceptación		
Ingresar 70 kg y 175 cm y comprobar que se obtiene un IMC de 22.9, categoría normal, y que el cálculo crea un documento en `registrosNutricionales` con la fecha del servidor.		
Buscar “pechuga de pollo” y verificar que se consulta “chicken breast” en USDA; registrar 150 g de un alimento de 165 kcal por 100 g y comprobar que se registran 248 kcal.		
Agregar los ocho alimentos rápidos y un alimento mediante el formulario manual, y verificar que cada uno crea un registro de ingesta válido.		
Revisar el resumen del día y comprobar que suma solo los alimentos de la fecha actual, limita los porcentajes al 100 % y usa metas calculadas con los datos físicos del usuario.		
Excepciones		**Riesgo.** Por evaluar
Si el peso o la estatura están fuera de rango, se muestra un mensaje de validación. Si el perfil está incompleto, el resumen usa metas de referencia.		

Tabla 4-17. Story Card de nutrición, diario de alimentación e IMC

Fuente: Elaboración propia.

4.3.6.2. Codificación

La pantalla `nutrition` tiene dos vistas que se alternan con un control segmentado: *Diario* y *Calc. IMC*.

Calculadora de IMC. `NutritionController` valida el peso y la estatura, calcula el IMC con `bmiCalculator` y guarda el resultado en `usuarios/{uid}/registrosNutricionales`. La Tabla 4-18 presenta las categorías utilizadas y el Código 4-8 muestra su implementación. El cálculo se realiza en el dispositivo; si el guardado falla, el resultado sigue disponible en pantalla.

Tabla 4-18. Clasificación del IMC según la OMS

Categoría	**IMC (kg/m^2)**	**Categoría interna**
Bajo peso	< 18.5	`BAJO`
Normal	$18.5 - 24.9$	`NORMALIDAD`
Sobrepeso	$25 - 29.9$	`SOBREPESO`
Obesidad	≥ 30	`OBESIDAD`

Fuente: Archivo `utils/bmiCalculator.ts`, con base en los rangos de la Organización Mundial de la Salud (2000).

```
export function calcularIMC(pesoKg: number, alturaCm: number): number {
  const alturaM = alturaCm / 100;
  return Number((pesoKg / (alturaM * alturaM)).toFixed(1));
```

```
}

export function clasificarIMC(imc: number): BmiCategory {
  if (imc < 18.5) return "BAJO";
  if (imc < 25) return "NORMALIDAD";
  if (imc < 30) return "SOBREPESO";
  return "OBESIDAD";
}
```

Código 4-8. Cálculo y clasificación del IMC

Diario de alimentación. La búsqueda utiliza USDA FoodData Central. Como la consulta requiere términos en inglés, `foodTranslator` traduce las expresiones conocidas mediante un diccionario y conserva las palabras que no encuentra. La función `buscarAlimentos` consulta los registros de tipo *Foundation* y *SR Legacy*, mantiene la clave de la API en el servidor y controla el acceso mediante autenticación, App Check y límites de consultas.

El usuario indica la cantidad consumida en gramos. A partir de los valores por 100 g, `calcularPorcion` obtiene las calorías y los macronutrientes de la porción (Código 4-9). El cálculo se comparte entre la vista previa y el guardado. Cada ingesta conserva el identificador `fdcId` del alimento consultado.

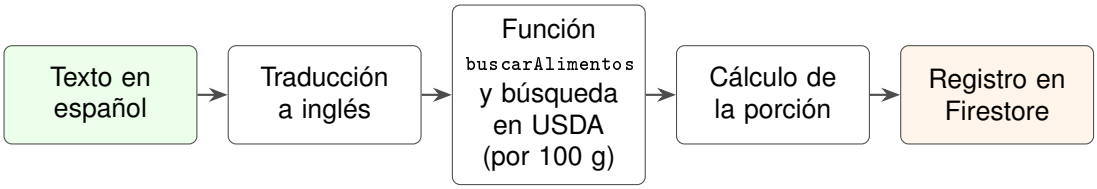


Figura 4-18. Flujo de registro de un alimento

Fuente: Elaboración propia.

```
export function calcularPorcion(
  alimento: AlimentoUSDA, gramos: number,
): PorcionCalculada {
  const factor = gramos > 0 ? gramos / 100 : 0;
  return {
    calorias: Math.round(alimento.caloriasPor100g * factor),
    proteinas: redondear1(alimento.proteinasPor100g * factor),
    carbohidratos: redondear1(alimento.carbohidratosPor100g * factor),
    grasas: redondear1(alimento.grasasPor100g * factor),
  };
}
```

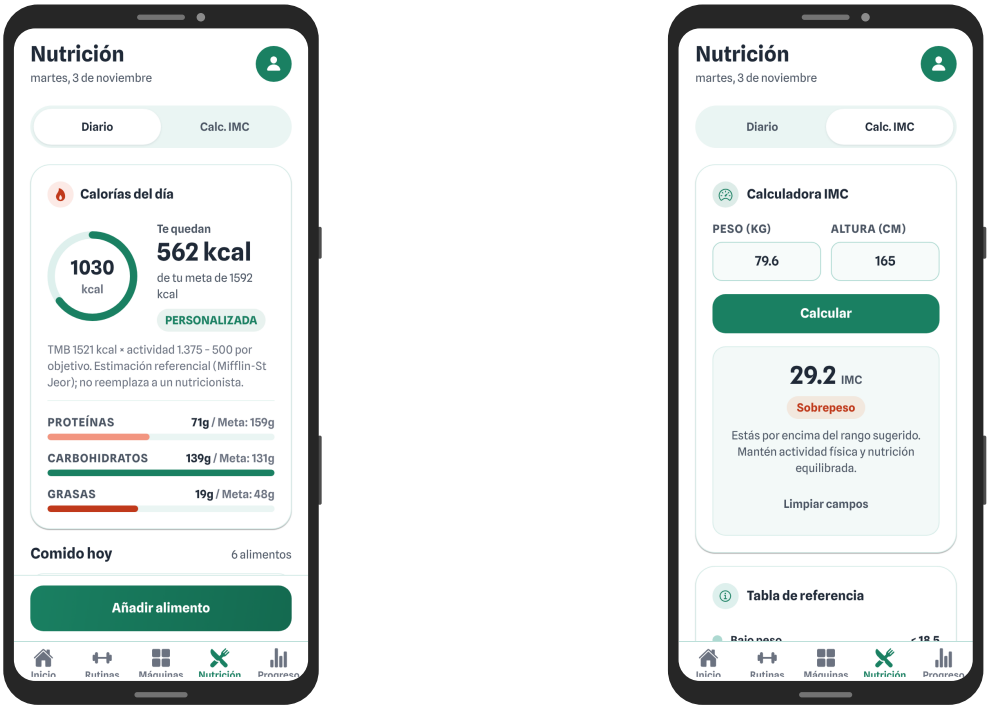
Código 4-9. Cálculo de la porción consumida

Resumen diario. El diario muestra las ingestas de la fecha local, sus calorías y macronutrientes acumulados y la opción de eliminar registros. Los indicadores comparan los totales con las metas calculadas a partir del perfil mediante `calcularMetasNutricionales` (Tabla 4-19). La función aplica la ecuación de Mifflin-St Jeor, un factor de actividad y un ajuste por objetivo. Se mantienen los mínimos calóricos implementados: 1500, 1200 y 1350 kcal según la categoría del perfil. Si faltan peso, estatura o edad, se usan metas de referencia de 2000 kcal, 150 g de proteínas, 200 g de carbohidratos y 65 g de grasas. Estas estimaciones tienen el alcance orientativo definido para el sistema.

Tabla 4-19. Reglas de las metas nutricionales diarias

Elemento	Regla
Tasa metabólica basal	$10\,p + 6{,}25\,a - 5\,e + c$, con p en kg, a en cm, e en años y $c = 5$ (hombre), -161 (mujer) o -78 (otro o sin dato).
Factor de actividad (días de entrenamiento por semana)	0–1: 1,2; 2–3: 1,375; 4–5: 1,55; 6 o más: 1,725.
Ajuste por objetivo	Perder peso: -500 kcal; ganar masa: $+300$ kcal; mantenimiento y resistencia: 0.
Proteínas	Perder peso: 2,0 g/kg; ganar masa: 1,8 g/kg; mantenimiento: 1,6 g/kg; resistencia: 1,4 g/kg.
Grasas	El mayor valor entre el 25 % de las calorías y 0,6 g/kg.
Carbohidratos	Las calorías restantes, a 4 kcal/g.

Fuente: Archivo `utils/nutritionGoals.ts`, con base en Mifflin et al. (1990).



(a) Diario de alimentación (b) Calculadora de IMC

Figura 4-19. Módulo de nutrición

Fuente: Elaboración propia.

4.3.7. Iteración 6: Progreso físico, entrenamientos y reportes

4.3.7.1. Planificación

Story Card	**Nombre del proyecto**	**Estimación**
08	CoachApp	Por completar
Nombre. Progreso físico, entrenamientos y reportes		**Fecha.** Por completar
Descripción		
El usuario registra su peso periódicamente sin volver a escribir su estatura, y ve un gráfico de la evolución de su peso y de su IMC, cuánto ha variado desde el primer registro y su categoría de IMC actual. Puede marcar que completó el entrenamiento de un día de su rutina y ver su racha, los días que entrenó esa semana y el total de entrenamientos. En el inicio ve un resumen de su rutina activa y de sus registros, y puede generar un reporte de su progreso en PDF para guardarlo o compartirlo.		
Prueba de aceptación		
Registrar el peso y comprobar que la estatura del perfil aparece precargada y que al guardar se crea el registro con su IMC.		
Agregar un registro y verificar que el gráfico se actualiza y muestra como máximo los ocho últimos.		
Con registros de 80 kg y 77.5 kg, comprobar que se muestra una variación de -2.5 kg y que la categoría corresponde al IMC del último registro.		
Marcar un día de la rutina como completado y verificar que la racha cuenta días consecutivos y no se rompe mientras el día de hoy aún no termina.		
Abrir el inicio y comprobar que muestra la rutina activa, su número de ejercicios y días por semana, y el número real de registros de peso.		
Generar el reporte eligiendo el período (7, 30, 90 días o todo el historial) y comprobar que el PDF incluye perfil, peso e IMC, entrenamientos y alimentación, y que se abre la hoja de compartir del sistema.		
Excepciones		**Riesgo.** Por evaluar
Marcar el mismo día dos veces no duplica el registro, y desmarcarlo lo elimina.		

Tabla 4-20. Story Card de progreso físico, entrenamientos y reportes

Fuente: Elaboración propia.

4.3.7.2. Codificación

Registro del progreso. `ProgressController` valida el peso (entre 10 y 500 kg), la estatura opcional y unas notas de hasta 500 caracteres; si hay estatura, calcula el IMC reutilizando `bmiCalculator`, y guarda el registro en `usuarios/{uid}/progresoFisico`. Para evitar que el usuario reescriba su estatura, la pantalla la precarga desde el perfil.

Visualización del progreso. La vista *Cuerpo* presenta los últimos ocho registros en un gráfico de peso o IMC, la variación respecto al primer registro, la categoría de IMC actual y el historial de mediciones. Estos indicadores se calculan a partir de los registros observados en tiempo real, sin crear una colección adicional de reportes.

Reporte personalizado en PDF. El usuario puede generar un reporte de 7, 30 o 90 días, o de todo su historial. `ReportsController` reúne los datos; `progressReport.ts` calcula el resumen y construye el HTML; y `pdfService.ts` genera y comparte el PDF mediante `expo-print` y `expo-sharing`. El documento incluye perfil, peso e IMC, entrenamientos y alimentación (Figura 4-20). La comprobación en el dispositivo de prueba se documenta en la sección 4.4.

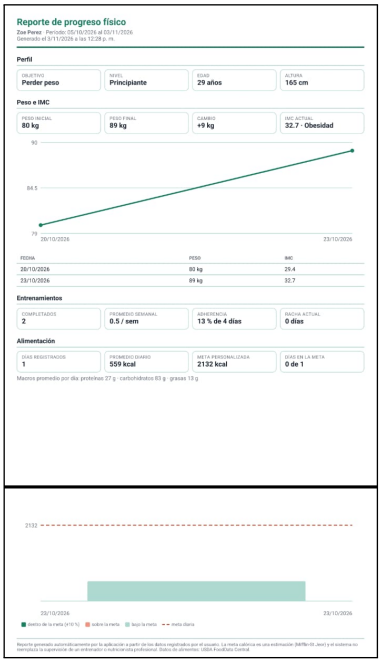


Figura 4-20. Reporte de progreso físico en PDF generado en el dispositivo de prueba

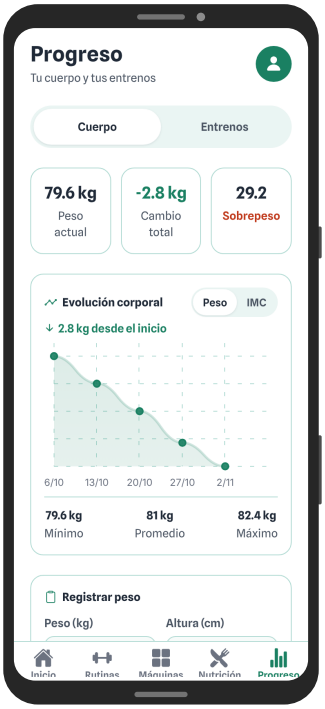
Fuente: Elaboración propia. Captura de pantalla del reporte generado por la aplicación en el Infinix HOT 50 Pro+ (3 de noviembre de 2026).

Seguimiento de entrenamientos. El usuario marca los días completados de su rutina. `WorkoutsController` registra cada entrenamiento con la fecha local como identificador, lo que evita duplicar el registro del mismo día. A partir de esas fechas se calculan la racha, los días entrenados en la semana y el total acumulado. La racha se cuenta desde ayer si el entrenamiento de hoy aún no se ha completado, para mantenerla vigente durante el día (Código 4-10).

```
export function calcularRacha(
  fechasCompletadas: ReadonlySet<string>, ahora: Date,
): number {
  const cursor = inicioDelDia(ahora);
  // Si hoy aun no se completo, la racha se evalua desde ayer.
  if (!fechasCompletadas.has(formatearFechaISO(cursor))) {
    cursor.setDate(cursor.getDate() - 1);
  }
  let racha = 0;
  while (fechasCompletadas.has(formatearFechaISO(cursor))) {
    racha++;
    cursor.setDate(cursor.getDate() - 1);
  }
  return racha;
}
```

Código 4-10. Cálculo de la racha de entrenamientos

Inicio. La pantalla principal reúne el nombre del usuario, la rutina activa, el número de ejercicios y días de entrenamiento, los registros de peso y los accesos al escáner, la guía y las rutinas.



(a) Peso e IMC



(b) Seguimiento de entrenamientos

Figura 4-21. Módulo de progreso

Fuente: Elaboración propia.



Figura 4-22. Pantalla de inicio

Fuente: Elaboración propia.

4.3.8. Fase de estabilización

En esta fase se concentró el trabajo en integrar los módulos, uniformar la arquitectura y verificar la calidad del código. Las adiciones funcionales corresponden al componente de servidor (funciones `buscarAlimentos` y `guardarRutina`), que se incorporó para proteger la clave de la API de nutrición y para validar en el servidor la creación de rutinas, y, el 3 de noviembre de 2026, al reconocimiento de máquinas en la nube (`reconocerMaquina`) y al módulo nativo de ARCore (sección 4.3).

4.3.8.1. Integración de los módulos

Los módulos desarrollados en las ramas `feature/auth`, `feature/machines` y `feature/ejercicios` se integraron en `develop` mediante las solicitudes de integración #1, #3 y #4. Los módulos de rutinas, escáner, nutrición y progreso se desarrollaron en `feature/routines`, integrada mediante la solicitud #5 (18 de octubre de 2026). La solicitud #6 (19 de octubre) incorporó el cambio de nombre de los archivos a nombres en inglés (`refactor/english-file-names`) y la solicitud #7 (27 de octubre) integró la rama `feature/modelAI`, con los modelos de TensorFlow Lite, el componente de servidor y las pruebas de seguridad. La navegación principal quedó unificada en la barra de pestañas de `(app)` (Inicio, Máquinas, Progreso y Nutrición), con las rutinas y el perfil accesibles desde el inicio, y la protección de rutas centralizada en el diseño raíz.

4.3.8.2. Refactorización por capas

Durante las primeras iteraciones, solo el módulo de nutrición cumplía completamente la separación en capas; las pantallas de autenticación, de máquinas y de rutinas llamaban a Firebase directamente. En la estabilización se extendió el patrón a todo el sistema, de modo que cada módulo cuenta con su controlador, sus entidades, su esquema de validación y su repositorio (Tabla 4-21).

Tabla 4-21. Componentes de cada módulo por capa

Módulo	Controlador	Modelo (entidad / repositorio)	Vista
Autenticación	`AuthController`	`Usuario` / `userRepository`	`(auth)`, `(onboarding)`, `profile`
Máquinas	`MachinesController`	`Maquina` / `machineRepository`	`machines/*`
Ejercicios	`ExercisesController`, `WgerExercisesController`	`Ejercicio`, `EjercicioWger` / repositorios de ejercicios	Modales de selección
Rutinas	`RoutinesController`	`Rutina` / `routineRepository`	`routines/*`
Nutrición	`NutritionController`	`RegistroNutricional`, `AlimentoUSDA` / `nutritionRepository`, `foodRepository`	`nutrition`
Progreso	`ProgressController`	`ProgresoFisico` / `progressRepository`	`progress/*`, inicio
Entrenamientos	`WorkoutsController`	`Entrenamiento` / `workoutRepository`	`progress/*`

Otras mejoras aplicadas en la refactorización fueron:

- **Tiempo real en lugar de recarga manual.** Las pantallas que consultaban los datos una sola vez y ofrecían una función de recarga pasaron a suscribirse con `onSnapshot` a través de los métodos `observar` de cada controlador, con la cancelación de la suscripción al desmontar la pantalla.

- **Manejo de errores de las suscripciones.** Se añadió un manejador por defecto que, ante un error de lectura, registra un aviso y entrega un resultado vacío para que la interfaz no quede esperando indefinidamente. El error de permisos que se produce al cerrar sesión con suscripciones activas se reconoce como esperado y se ignora.
- **Limpieza de campos indefinidos.** Firestore rechaza los campos con valor `undefined`; en lugar de exigir que cada repositorio lo tenga en cuenta, la limpieza se centralizó en `firestoreService`, el único archivo que usa el SDK.
- **Validación en el controlador.** Todas las operaciones de escritura validan sus datos con Zod antes de llegar al repositorio, y las reglas de seguridad repiten las validaciones críticas en el servidor.
- **Reglas alineadas con el modelo real.** Las reglas de Firestore se revisaron campo por campo contra las entidades y los repositorios, de modo que cada escritura que realiza la aplicación satisface su regla y ninguna otra es aceptada.
- **Convenciones de código.** Importaciones con el alias `@/` en lugar de rutas relativas largas, formato uniforme con Prettier y reglas de análisis estático de React (incluidas las de *hooks*) sin desactivar.

4.3.8.3. Métricas del código

La Tabla 4-22 resume el tamaño del código fuente por capa al cierre de la estabilización.

Tabla 4-22. Tamaño del código fuente por capa

Capa	Carpetas	Archivos	Líneas
Vista	`app/`, `components/`	87	11 410
Controlador	`controllers/`, `utils/`, `store/`, `hooks/`	41	5 454
Modelo	`models/`, `schemas/`	32	1 852
Servicios	`services/`, `config/`	20	1 423
Otros	`constants/`, `types/`	6	93
Total		**186**	**20 232**

Fuente: Conteo de archivos `.ts` y `.tsx` de la carpeta `src/`, sin las pruebas automáticas, incluidos comentarios y líneas en blanco, al 3 de noviembre de 2026. No incluye el código de las funciones de servidor (`functions/`) ni el del módulo nativo de ARCore (`modules/coach-ar/`, 606 líneas de Kotlin en cuatro archivos). El crecimiento respecto del 27 de octubre se debe sobre todo a los componentes compartidos de interfaz, a los servicios y esquemas del reconocimiento en la nube y a la interfaz tipada de la vista de realidad aumentada.

4.3.8.4. Verificación estática

Como criterio de cierre de la fase se estableció una secuencia de verificaciones automáticas que el código debe superar (Tabla 4-23).

Tabla 4-23. Verificaciones estáticas del código

Verificación	Qué detecta	Resultado
`tsc -noEmit`	Errores de tipos en todo el proyecto (TypeScript estricto).	Sin errores (registro del proyecto del 3 de noviembre de 2026)
`eslint .`	Errores y malas prácticas, incluidas las reglas de *hooks* de React.	Sin errores (registro del 3 de noviembre de 2026)
`prettier -check`	Formato no uniforme del código.	Conforme (registro del 3 de noviembre de 2026)

4.3.8.5. Pruebas unitarias automáticas

Como complemento de la verificación estática, se configuró Jest con el *preset* `jest-expo` y se escribieron pruebas para la lógica pura del sistema, es decir, las funciones que no dependen de Firebase ni de módulos nativos. Se ejecutan con `npm test`. La Tabla 4-24 lista los archivos de prueba de la aplicación, que son 20 y comprenden 160 casos; en la ejecución del 3 de noviembre de 2026 todos resultaron aprobados, igual que los 22 casos de las funciones del servidor y las 5 pruebas de JVM del módulo de ARCore (registro del proyecto en `docs/ARCORE.md`). Su detalle se presenta en la sección 4.4.1.2. **[PENDIENTE:** conservar la salida de `npm test` como evidencia para el tribunal.**]**

Tabla 4-24. Suites de pruebas unitarias

Suite	Qué verifica
`bmiCalculator`	Cálculo del IMC, límites de las categorías de la OMS y mensajes.
`nutritionGoals`	Ecuación de Mifflin-St Jeor, factores de actividad, ajuste por objetivo, piso calórico y coherencia de los macronutrientes.
`workoutStats`	Fechas, racha, semana y total de entrenamientos.
`workoutSession`	Creación de la sesión, series completadas y descansos.
`routineGenerator`	Orden de las plantillas y esquemas (contrato con el modelo), reglas de `decidirClases` y reparto de ejercicios.
`features`	Vector de entrada del modelo de rutinas y valores por defecto.
`tfliteService`	Carga diferida del intérprete de TensorFlow Lite.
`ProgressController`, `ReportsController`	Actualización parcial del progreso y cálculo de los reportes.
`MachinesController`, `machineRecognition`	Candidatas con su caja, que no se mezclan con las de otras máquinas; preparación del objetivo de ARCore y esquema de la respuesta del reconocimiento.
`workoutSessionStorage`	Persistencia de la sesión de entrenamiento guiada.
`exerciseDbRepository`, `wgerImageRepository`, `localDemonstrationRepository`	Repositorios de ejercicios y demostraciones locales por máquina.
`persisted`	Esquemas de los datos persistidos.
`environment`, `loadingProgress`, `typography`	Variables de entorno faltantes, porcentaje de carga simulado y selección de fuentes por peso.

Fuente: Carpetas `__tests__` de `src/`. Las pruebas del componente de servidor y de las reglas de seguridad se describen en la sección 4.4.1.3.

4.4. Prueba final

La última fase de Mobile-D verifica que el producto cumpla sus requerimientos. Las pruebas se organizaron en seis niveles, de los más automáticos a los que requieren al usuario (Tabla 4-25).

Tabla 4-25. Estrategia de pruebas

Nivel	Objeto de la prueba	Técnica
Verificación estática	Tipos y calidad de todo el código.	Compilador y analizadores (sección 4.3.8)
Lógica de negocio	Funciones puras: generador de rutinas, IMC, estadísticas de entrenamiento, porciones de alimentos y traducción de consultas.	Pruebas unitarias con valores límite
Reglas de seguridad y funciones	Aislamiento entre usuarios, validación de datos y control de acceso en el servidor.	Escenarios permitidos y denegados (emulador de Firebase)
Funcionales	Cada requerimiento funcional en la aplicación instalada.	Caja negra, casos de prueba
Usabilidad	Facilidad de uso de las tareas principales.	Tareas con usuarios
Aceptación	Cumplimiento de cada objetivo específico.	Trazabilidad objetivo–requerimiento–prueba

4.4.1. Verificación técnica

4.4.1.1. Pruebas de la lógica de negocio

Las funciones de las reglas de negocio no dependen de la red ni de la base de datos, por lo que se ejercitaron directamente con Node.js 22 sobre el código fuente del proyecto, sin modificar su lógica. Las funciones del controlador de nutrición se copiaron textualmente a un archivo auxiliar para aislarlas del SDK de Firebase. La ejecución se realizó el 14 de octubre de 2026.

Generador de rutinas. Se evaluaron las doce combinaciones posibles de objetivo y nivel. La Tabla 4-26 muestra los resultados obtenidos, que coinciden en todos los casos con las reglas de las Tablas 4-13 a 4-15.

Tabla 4-26. Resultados del generador para las doce combinaciones de entrada

Objetivo	Nivel	Días	Series × Rep. (desc.)	División obtenida	Ejerc.
Ganar masa	Principiante	3	4×9 (90 s)	Emp. – Tir. – Pie.	15
Ganar masa	Intermedio	4	4×9 (90 s)	Emp. – Tir. – Pie. – C.C.	21
Ganar masa	Avanzado	5	4×9 (90 s)	Emp. – Tir. – Pie. – Emp. – Tir.	25
Mantenimiento	Principiante	3	3×12 (60 s)	Emp. – Tir. – Pie.	15
Mantenimiento	Intermedio	4	3×12 (60 s)	Emp. – Tir. – Pie. – C.C.	21
Mantenimiento	Avanzado	5	3×12 (60 s)	Emp. – Tir. – Pie. – Emp. – Tir.	25
Perder peso	Principiante	3	3×15 (45 s)	Emp. – Car. – Pie.	14
Perder peso	Intermedio	4	3×15 (45 s)	Emp. – Tir. – Car. – Pie.	19
Perder peso	Avanzado	5	3×15 (45 s)	Emp. – Tir. – Pie. – Car. – C.C.	25
Resistencia	Principiante	3	3×18 (30 s)	Emp. – Car. – Pie.	14
Resistencia	Intermedio	4	3×18 (30 s)	Emp. – Tir. – Car. – Pie.	19
Resistencia	Avanzado	5	3×18 (30 s)	Emp. – Tir. – Pie. – Car. – C.C.	25

Fuente: Ejecución de `generarPlanHeuristico` (comportamiento base, sin edad, IMC ni días disponibles). Emp.: empuje; Tir.: tirón; Pie.: pierna; Car.: cardio; C.C.: cuerpo completo; Ejerc.: total de ejercicios por semana.

Cálculo del IMC. Se probaron valores en los límites de cada categoría de la OMS (Tabla 4-27). Todos los casos produjeron el resultado esperado.

Tabla 4-27. Pruebas de valores límite del cálculo y la clasificación del IMC

Peso (kg)	Estatura (cm)	IMC	Categoría obtenida	Resultado
50.0	170	17.3	Bajo peso	Correcto
53.4	170	18.5	Normal	Correcto
70.0	175	22.9	Normal	Correcto
72.2	170	25.0	Sobrepeso	Correcto
86.7	170	30.0	Obesidad	Correcto
100.0	170	34.6	Obesidad	Correcto
Clasificación de 18.4 / 18.5			Bajo peso / Normal	Correcto
Clasificación de 24.9 / 25.0			Normal / Sobrepeso	Correcto
Clasificación de 29.9 / 30.0			Sobrepeso / Obesidad	Correcto

Fuente: Ejecución de `calcularIMC` y `clasificarIMC`.

Reparto, entrenamientos, porciones y traducción. La Tabla 4-28 reúne las demás pruebas de funciones puras. Para las estadísticas de entrenamiento se fijó la fecha actual en el miércoles 14 de octubre de 2026.

Tabla 4-28. Otras pruebas de la lógica de negocio

ID	Función	Entrada	Resultado obtenido
PL-01	`repartirCantidad`	5 ejercicios en 3 categorías	2 – 2 – 1
PL-02	`repartirCantidad`	6 ejercicios en 3 categorías	2 – 2 – 2
PL-03	`repartirCantidad`	4 ejercicios en 2 categorías	2 – 2
PL-04	Estadísticas de entrenamiento	Días 12, 13 y 14 de octubre (hoy incluido)	Racha 3; semana 3; total 3
PL-05	Estadísticas de entrenamiento	Días 12 y 13 (hoy sin completar)	Racha 2; semana 2
PL-06	Estadísticas de entrenamiento	Días 10 y 12 (falta el día 13)	Racha 0; semana 1
PL-07	Estadísticas de entrenamiento	Diez días consecutivos hasta hoy	Racha 10; semana 3; total 10
PL-08	Estadísticas de entrenamiento	Sin registros	Racha 0; semana 0; total 0
PL-09	`calcularPorcion`	Arroz (130 kcal, 2.7 g P, 28 g C, 0.3 g G por 100 g), 200 g	260 kcal; 5.4 g P; 56 g C; 0.6 g G
PL-10	`calcularPorcion`	Pollo (165 kcal, 31 g P por 100 g), 150 g	248 kcal; 46.5 g P; 0 g C; 5.4 g G
PL-11	`calcularPorcion`	Cualquier alimento, 0 g	Todos los valores en 0
PL-12	`calcularResumenDiario`	Dos alimentos de hoy (arroz y pollo), uno de otra fecha y un cálculo de IMC	Solo suma los dos de hoy: 508 kcal, quedan 1 492; 25.4 % de la meta de referencia (2 000 kcal)
PL-13	`calcularResumenDiario`	Un alimento de 2 600 kcal	Porcentaje limitado a 100 %; restantes 0
PL-14	`traducirConsulta`	“pechuga de pollo”, “Plátano”, “leche descremada”	“chicken breast”, “banana”, “milk nonfat”
PL-15	`traducirConsulta`	“chicken” y “a”	“chicken” sin cambio; “a” no genera consulta

4.4.1.2. Pruebas unitarias automáticas

Las pruebas anteriores se ejecutaron una vez, con *scripts* auxiliares. Para que las reglas de negocio puedan verificarse de forma repetible después de cada cambio, se escribieron 20 archivos de prueba de Jest (Tabla 4-24), que se ejecutan con `npm test`. Las funciones probadas son puras, por lo que las pruebas no requieren Firebase ni un dispositivo. Entre los casos cubiertos están:

- **IMC:** los límites exactos de cada categoría de la OMS (18,4 es bajo peso; 18,5 es normal; 24,9 es normal; 25 es sobrepeso; 30 es obesidad).
- **Metas nutricionales:** que un hombre de 25 años, 75 kg y 178 cm, que entrena cuatro días para ganar masa, obtenga una tasa basal de 1 743 kcal y una meta de 3 001 kcal; que perder peso aplique un déficit de 500 kcal; que nunca se baje del piso calórico (1 200 kcal para mujeres y 1 500 kcal para hombres); y que los macronutrientes no sean negativos y reconstruyan las calorías con una tolerancia de 10 kcal.
- **Generador de rutinas:** que el orden de las seis plantillas y de los cuatro esquemas coincida con el del *script* de entrenamiento del modelo (si cambiara, el modelo dejaría de ser válido); que IMC $\geq$ 30 active el énfasis en cardio; que los mayores de 60 años no reciban hipertrofia ni esquema metabólico y los menores de 16 no reciban hipertrofia; y que cualquier combinación de perfil produzca clases válidas.
- **Vector de entrada del modelo:** el orden de las diez características, la codificación *one-hot*

del objetivo y los valores por defecto cuando faltan datos del perfil.

- **Estadísticas y sesión de entrenamiento:** racha, semana y total; creación de la sesión, series completadas y descansos.
- **Reconocimiento y realidad aumentada:** que las cajas de las candidatas alternativas no se mezclen con las de la máquina principal, que el objetivo que recibe la vista de ARCore tome la caja de la candidata confirmada, y que el esquema de la respuesta rechace cajas inválidas.
- **Otros componentes:** carga diferida de TensorFlow Lite, esquemas de los datos persistidos, actualización parcial del progreso, persistencia de la sesión de entrenamiento, demostraciones locales por máquina y verificación de las variables de entorno.

4.4.1.3. Pruebas de las reglas de seguridad y de las funciones del servidor

Las reglas de seguridad y las funciones del servidor constituyen la barrera frente a un acceso malintencionado, porque las credenciales de Firebase están contenidas en la aplicación y no son secretas; App Check añade una verificación complementaria de la procedencia de las solicitudes. La Tabla 4-29 define los escenarios evaluados y el resultado que exigen las reglas del Anexo B. Los escenarios se automatizaron con el emulador de Firebase (comando `npm run test:security`), autenticando como un usuario A y operando sobre datos propios o de otro usuario B: el archivo `tests/security/rules.test.mjs` declara 11 pruebas sobre Firestore y Storage, y `functions/test/security.integration.mjs` declara 4 pruebas de integración de las funciones. Las pruebas de los esquemas del servidor (`npm run test:backend`), que incluyen la interpretación de la respuesta del reconocimiento (clases, equivalencias, cajas inválidas y confianza fuera de rango), se ejecutaron el 3 de noviembre de 2026 y aprobaron 22 de 22 casos. Los 19 escenarios de seguridad de la Tabla 4-29 también se completaron satisfactoriamente: los resultados obtenidos coincidieron con los esperados.

Tabla 4-29. Escenarios de prueba de las reglas de seguridad

ID	Escenario	Esperado	Obtenido
SEG-01	Usuario no autenticado lee `maquinas/{id}`	Denegado	Denegado
SEG-02	A lee `maquinas/{id}`	Permitido	Permitido
SEG-03	A escribe en `maquinas/{id}`	Denegado	Denegado
SEG-04	A lee `usuarios/A/rutinas`	Permitido	Permitido
SEG-05	A lee `usuarios/B/rutinas`	Denegado	Denegado
SEG-06	A envía a `guardarRutina` una rutina con `diasPorSemana = 8`	Rechazado	Rechazado
SEG-07	A envía a `guardarRutina` una rutina con `usuarioId = B` (el propietario se toma de la identidad autenticada)	Rechazado	Rechazado
SEG-08	A modifica el campo `email` de su perfil	Denegado	Denegado
SEG-09	A crea su perfil con `edad = 10`	Denegado	Denegado
SEG-10	A registra progreso con `peso = 0`	Denegado	Denegado
SEG-11	A registra ingesta con `calorias = -100`	Denegado	Denegado
SEG-12	A crea un entrenamiento cuyo `fecha` difiere del identificador del documento	Denegado	Denegado
SEG-13	A crea un entrenamiento con `dia = 8`	Denegado	Denegado
SEG-14	A crea su perfil con `creadoEn` distinto de la hora del servidor	Denegado	Denegado
SEG-15	A sube un archivo a Storage	Denegado	Denegado
SEG-16	A crea directamente un documento en `usuarios/A/rutinas`, sin usar la función	Denegado	Denegado
SEG-17	Llamada a `guardarRutina`, `buscarAlimentos` o `reconocerMaquina` sin autenticación	Rechazado	Rechazado
SEG-18	Llamada a las funciones con un usuario anónimo	Rechazado	Rechazado
SEG-19	Solicitudes concurrentes que superan la cuota por usuario	Limitadas a la cuota	Limitadas a la cuota

4.4.1.4. Evaluación de los modelos de inteligencia artificial

La evaluación distingue el clasificador local, el detector en la nube y el recomendador de rutinas. Se presentan sus resultados, las condiciones de evaluación y las limitaciones. Los resultados proceden de los registros citados en cada tabla; no equivalen a una validación con usuarios.

Evaluación del clasificador local de máquinas

Evaluación. Se evaluó con validación cruzada de cinco pliegues y cuidando que las versiones aumentadas de una misma foto nunca estuvieran en entrenamiento y validación a la vez. La Tabla 4-30 resume los resultados con imágenes limpias y con el escenario de campo simulado.

Tabla 4-30. Exactitud del reconocimiento de máquinas (validación cruzada, 35 clases)

Escenario	Acierto top-1	Acierto top-3
Imágenes limpias del catálogo	43,4 %	65,9 %
Campo simulado (fondos y cámara)	38,5 %	61,4 %

Fuente: Archivo `ml/results/recognition/recognition_metrics.json`. El azar acertaría el 2,9 % (1 de 35).

El umbral de confianza se calibró sobre esas mismas predicciones (Tabla 4-31): a mayor umbral, más seguro es el reconocimiento automático, pero menos veces se usa. Se eligió 0,8 porque, cuando la aplicación acepta la máquina sin preguntar, acierta aproximadamente 85 de cada 100 veces; en las demás ocasiones el usuario confirma una sugerencia ya preseleccionada.

Tabla 4-31. Calibración del umbral de confianza

Umbral	Cobertura	Precisión
0,5	44,4 %	60,5 %
0,7	23,6 %	75,1 %
0,8	**15,8 %**	**85,4 %**
0,9	9,2 %	92,2 %

Fuente: Archivo `ml/results/recognition/recognition_metrics.json`. Cobertura: proporción de fotos que superan el umbral; precisión: aciertos entre ellas.

Limitaciones. El resultado es modesto y debe leerse como tal. Con solo 399 fotografías de catálogo (de una a 25 por máquina) el modelo distingue bien las máquinas visualmente más características, como la caminadora (93 % de acierto en el campo simulado) o la bicicleta estática (67 %), pero casi no reconoce otras: curl de bíceps, pantorrillas, torso rotatorio, elevaciones laterales y remo en T se acercan al 0 %, y dos máquinas con una sola foto no pudieron evaluarse. Además, las fotografías de catálogo no son imágenes tomadas en un gimnasio real. Por ello la máquina se acepta directamente solo cuando la confianza alcanza el umbral de 0,8, con una cobertura del 15,8 % de las fotografías y una precisión del 85,4 % en la validación cruzada; en los demás casos el reconocimiento se presenta como una *sugerencia* que el usuario confirma o corrige. La mejora esperada proviene de más fotografías propias o autorizadas, tomadas en gimnasios reales (sección 6.2).

Evolución y evaluación del detector en la nube

Primera versión del detector (YOLO11x). Esta versión parte de YOLO11x, el tamaño extragrande de la generación YOLO11, con los pesos preentrenados que distribuye Ultralytics (Khanam & Hussain, 2024; Jocher et al., 2023). Se ajustó en Google Colab con un conjunto anotado en Roboflow (Roboflow, Inc., s. f.) y exportado en formato YOLO. El conjunto tiene 458 imágenes y 625 cajas de 42 clases, repartidas en 327 imágenes de entrenamiento, 88 de validación y 43 de prueba. Se entrenó durante 15 épocas con imágenes de 640×640 píxeles y lotes de 16, con los aumentos de datos por defecto de Ultralytics (mosaico, cambios de color y volteo horizontal). La evaluación se hizo sobre la partición de prueba, en CPU, tras comprobar con *hashes* que ninguna imagen de prueba fuera idéntica a una de entrenamiento o validación (Tabla 4-32).

Tabla 4-32. Métricas del detector de máquinas en la partición de prueba

Métrica	Valor
Precisión	79,8 %
Exhaustividad (*recall*)	29,9 %
mAP50	39,8 %
mAP50–95	37,6 %

Fuente: Archivo `ml/detection/runs/best42-evaluation/results.json`, generado con `evaluate_best42.py`. Partición de prueba: 43 imágenes, 65 cajas, 24 clases con ejemplos; entrada de 640 píxeles; evaluación en CPU de un computador, no en un teléfono. Definición de las métricas en Padilla et al. (2021).

Estos resultados deben leerse con cautela por cuatro razones. La primera es que la partición de prueba es mínima: 43 imágenes con 65 cajas, de las cuales 24 pertenecen a una sola clase (`hip_adduction_machine`) y 18 de las 42 clases no tienen ningún ejemplo. La segunda es el desbalance del entrenamiento: esa misma clase aporta 91 cajas, la mediana es de 8 cajas por clase, y cuatro clases (extensión de cuádriceps, poleas de doble cable, rack de sentadilla y remo alto) no

tienen ninguna, por lo que el detector no puede reconocerlas. La tercera es que varias imágenes parecen series de una misma máquina vista desde ángulos cercanos; aunque no haya duplicados exactos, la prueba puede resultar más fácil que una situación real en un gimnasio. La cuarta es que una exhaustividad del 29,9 % significa que, en el punto de operación que reporta Ultralytics, el detector encuentra menos de un tercio de las cajas reales. Por todo ello las cifras muestran que el procedimiento funciona de extremo a extremo, no que el detector sea fiable en un gimnasio.

El proyecto incluye además un *script* para exportar la primera versión del detector al formato LiteRT, el sucesor de TensorFlow Lite, y ejecutarlo en el propio teléfono (`export_best42.py`). Está preparado, pero requiere Linux o macOS y no se ha ejecutado ni probado en Android.

Versión final del detector: YOLO26m en Ultralytics Platform. La versión final se entrenó y se publicó en Ultralytics Platform con YOLO26 en su tamaño mediano (m), a partir de los pesos preentrenados que distribuye Ultralytics (Jocher et al., 2026; Ultralytics, s. f.-c). YOLO26 es la generación más reciente de la familia y sus autores la orientan a dispositivos de borde, con una inferencia que no requiere supresión de no máximos (Sapkota et al., 2025). El conjunto de datos (`gym-machines-data`) se creó en el editor de la plataforma, que permite anotar con polígonos y con la ayuda de un modelo de segmentación (SAM) (Ultralytics, s. f.-a): cada máquina se delimitó con un polígono y se le asignó una de las 52 clases definidas para el proyecto. La tarea de entrenamiento es de detección, de modo que el modelo predice cajas delimitadoras y no contornos.

Según el gráfico de distribución de clases de la plataforma, el conjunto reúne 6 681 ejemplos anotados (Tabla 4-33). La clase más frecuente (`prensa-piernas`) aporta 615 ejemplos, el 9,2 % del total, y cinco clases tienen cuatro ejemplos o menos (`rack-sentadilla`, `press-alto`, `press-asistida`, `extension-cuadriceps` y `remo-alto`). Las 24 clases con 100 ejemplos o más concentran el 88,9 % de las anotaciones, y las seis con menos de diez apenas el 0,3 %. El desbalance es fuerte (de 615 ejemplos a uno en la clase más rara) y, con tan pocos ejemplos, esas clases difícilmente pueden aprenderse o evaluarse: su resultado individual no es interpretable y las métricas globales se apoyan sobre todo en las clases frecuentes. Además, algunas etiquetas parecen nombrar la misma máquina con criterios distintos (por ejemplo, `abductores-aductores` y `hip_adduction_machine`, o `curl-biceps` y `bicep_curl_machine`), lo que puede repartir los ejemplos de una máquina entre dos clases.

Tabla 4-33. Distribución de los ejemplos anotados por clase en `gym-machines-data`

Ejemplos por clase	Clases	Ejemplos	% del total
100 o más	24	5 939	88,9 %
De 30 a 99	11	473	7,1 %
De 10 a 29	11	248	3,7 %
Menos de 10	6	21	0,3 %
Total	**52**	**6 681**	**100 %**

Fuente: Elaboración propia con los conteos por clase del gráfico de distribución de clases de Ultralytics Platform.

El entrenamiento duró 80 épocas, con imágenes de 640×640 píxeles, lotes de 32, una GPU, semilla 42 en modo determinista y precisión mixta automática. El optimizador se dejó en automático y se mantuvieron los aumentos de datos por defecto de Ultralytics (mosaico, que se desactiva en las últimas 10 épocas, volteo horizontal, variaciones de color y borrado aleatorio). No se configuró parada temprana (paciencia igual a 0). La corrida se registró como `version-5` del proyecto `detect-machine`. La Tabla 4-34 resume las métricas de la última época, calculadas sobre la partición de validación.

Tabla 4-34. Métricas del detector final (YOLO26m) en la partición de validación

Métrica	Valor
Precisión	57,7 %
Exhaustividad (*recall*)	45,0 %
mAP50	48,5 %
mAP50–95	44,8 %
Pérdida de caja (entrenamiento / validación)	0,243 / 0,612
Pérdida de clasificación (entrenamiento / validación)	0,286 / 2,690

Fuente: Registro de entrenamiento de Ultralytics Platform (corrida `version-5`, época 80). Definición de las métricas en Padilla et al. (2021). La validación no es una partición de prueba independiente.

Estas cifras son las del propio entrenamiento y deben leerse con cautela. La partición de validación es la que Ultralytics usa también para elegir el mejor punto de control, de modo que tiende a favorecer al modelo; no se evaluó con una partición de prueba independiente ni con fotografías de un gimnasio real. Además, la pérdida de clasificación en validación (2,69) es unas diez veces la de entrenamiento (0,29), y la de caja (0,61) más del doble (0,24), lo que apunta a sobreajuste: el modelo se ajusta bien a las imágenes que ya vio y generaliza menos a las nuevas. Por todo ello las cifras muestran que el procedimiento funciona de extremo a extremo, no que el detector sea fiable en un gimnasio.

Dispositivo utilizado para las pruebas

Dispositivo de prueba. El teléfono de prueba es un Infinix HOT 50 Pro+ (modelo X6880), un equipo de gama media cuyas características resume la Tabla 4-35. Según la lista de dispositivos compatibles de Google, este modelo admite ARCore y la API de profundidad (Google for Developers, s. f.-a), que es lo que necesita el módulo CoachAR. Con un único equipo no puede afirmarse que la aplicación se comporte igual en otros teléfonos, con cámaras, memoria o soporte de ARCore distintos.

Tabla 4-35. Características del dispositivo de prueba

Característica	Valor
Modelo	Infinix HOT 50 Pro+ (X6880)
Procesador	MediaTek Helio G100
Memoria RAM	8 GB (más 5 GB de RAM extendida)
Almacenamiento	256 GB
Cámara trasera	50 MP + 2 MP
Cámara frontal	13 MP
Pantalla	1 080 $\times$ 2 436 píxeles
Batería	5 000 mAh
Sistema operativo	XOS 15.1.2 (basado en Android 15)

Fuente: Pantalla «Mi teléfono» de los ajustes del dispositivo. Compatibilidad con ARCore: Google for Developers (s. f.-a).

Verificación de la realidad aumentada La aplicación se probó en un dispositivo físico y cumplió los criterios de verificación de la realidad aumentada. El modelo reconoció la máquina y el módulo `CoachAR` mostró su información mediante ARCore. El seguimiento espacial, la consulta al servicio de reconocimiento y la visualización de la ficha funcionaron correctamente de forma integrada.

Los casos CP-42 a CP-44 del Anexo C se completaron satisfactoriamente.

Evaluación del recomendador de rutinas

Resultados. En el conjunto de prueba el modelo acertó la plantilla en el 99,8 % de los casos, el esquema de carga en el 99,97 % y ambas a la vez en el 99,8 % (Tabla 4-36). Como comprobación adicional se evaluó el modelo sobre una cuadrícula exhaustiva de 28 512 perfiles que recorre todas las combinaciones de objetivo, nivel, género, días, edades y valores de IMC alrededor de los umbrales: coincidió con las reglas en 27 689 (97,1 %). Los desacuerdos se concentran justo en los umbrales, por ejemplo con 59 años (un año antes del cambio de esquema) o con IMC de 29,9 (justo antes del énfasis cardio), donde un modelo continuo suaviza la frontera que las reglas trazan de forma abrupta.

Tabla 4-36. Desempeño del recomendador de rutinas sobre datos sintéticos

Medida	Resultado
Exactitud de la plantilla semanal (prueba, 9 000 perfiles)	99,8 %
Exactitud del esquema de carga (prueba)	99,97 %
Exactitud de ambas clases (prueba)	99,8 %
Coincidencia con las reglas en la cuadrícula exhaustiva (28 512 perfiles)	97,1 %
Coincidencia del archivo `.tflite` con el modelo de Keras	100 %

Fuente: Archivos `ml/results/metrics.json` y `ml/results/model_metadata.json`.

Comprobación del reporte PDF Según el registro consignado en el documento, el 3 de noviembre de 2026 se generó un reporte de 30 días en el Infinix HOT 50 Pro+. El archivo incluyó perfil, peso e IMC, entrenamientos y alimentación y se abrió en el visor del teléfono. La Figura 4-20 presenta datos de demostración. Esta comprobación corresponde al reporte y no valida el módulo de ARCore.

4.4.2. Pruebas funcionales en dispositivos

Las pruebas funcionales se realizan con la técnica de caja negra sobre el *development build* instalado en un dispositivo Android, siguiendo los criterios de aceptación definidos en cada iteración. Se definieron 44 casos de prueba que cubren los requerimientos funcionales RF1 a RF6; la matriz completa, con los pasos, el resultado esperado y el resultado obtenido, se encuentra en el Anexo C. La Tabla 4-37 resume su distribución.

Tabla 4-37. Distribución de los casos de prueba funcionales

Módulo	Casos	Requerimientos
Autenticación, onboarding y perfil	9	RF1.1 a RF1.5
Guía de máquinas y ejercicios	6	RF2.1 a RF2.3
Escáner con reconocimiento y realidad aumentada	7	RF2.4
Rutinas y planificación	7	RF3.1 a RF3.3
Nutrición (alimentos e IMC)	7	RF4.1, RF5.1 a RF5.3
Progreso, entrenamientos y reportes	8	RF4.2, RF6.1, RF6.2
Total	**44**	

4.4.3. Pruebas funcionales con usuarios

4.4.3.1. Pruebas de usabilidad

Las pruebas de usabilidad permitieron comprobar la realización de las tareas principales de la aplicación (Nielsen, 1993). Las seis tareas evaluadas se completaron correctamente y cumplieron sus criterios de éxito. La Tabla 4-38 presenta los resultados obtenidos.

Tabla 4-38. Resultados de las pruebas de usabilidad

ID	Tarea	Criterio de éxito	Resultado
T1	Registrarse y completar el *onboarding*.	Llega a la pantalla de inicio.	Cumple
T2	Encontrar la ficha de la prensa de piernas.	Abre la ficha desde el catálogo.	Cumple
T3	Escanear una máquina y abrir su ficha completa.	Abre la ficha desde la superposición.	Cumple
T4	Generar una rutina y marcar como completado el día de hoy.	La rutina queda activa y el día marcado.	Cumple
T5	Registrar 150 g de arroz en el diario.	El alimento aparece en la lista del día.	Cumple
T6	Registrar su peso y ver el gráfico de evolución.	Ve el nuevo punto en el gráfico.	Cumple

4.4.4. Pruebas de aceptación por objetivo

La Tabla 4-39 relaciona cada objetivo específico con los requerimientos que lo implementan y con las pruebas que lo verifican. Esta trazabilidad es la base de la evaluación del cumplimiento de objetivos que realiza el tribunal (Artículo 64 del Reglamento de Titulación).

Tabla 4-39. Trazabilidad entre objetivos, requerimientos y pruebas

Obj.	Funcionalidad entregada	Req.	Pruebas
OE1	Guía de 35 máquinas con búsqueda, filtros, fichas y escáner con reconocimiento asistido (modelo local o detector en la nube) y ficha anclada con ARCore en los teléfonos compatibles.	RF2.1, RF2.2, RF2.4	CP-10 a CP-13, CP-15 a CP-19, CP-42 a CP-44, sección 4.4.1.4
OE2	Ejercicios con animación, músculo objetivo y dificultad.	RF2.3	CP-14
OE3	*Onboarding* de objetivo, nivel y datos físicos; rutinas generadas por un recomendador de TensorFlow Lite con respaldo de reglas.	RF1.4, RF3.1	CP-06 a CP-08, CP-20, CP-21, Tabla 4-26, secciones 4.4.1.2 y 4.4.1.4
OE4	Planificador de rutinas por día, rutina activa y agregado desde la ficha.	RF3.2, RF3.3	CP-22 a CP-26
OE5	Diario de alimentación con calorías y macronutrientes, metas personalizadas; calculadora de IMC con clasificación e historial.	RF4.1, RF5	CP-27 a CP-33, Tablas 4-27 y 4-28, sección 4.4.1.2
OE6	Registro de peso, estatura e IMC.	RF4.2	CP-34, CP-35
OE7	Reporte con gráfico, variación y categoría actual; reporte en PDF por período; seguimiento de entrenamientos con racha.	RF6	CP-36 a CP-41, Tabla 4-28

4.4.5. Cumplimiento del alcance y desviaciones

La Tabla 4-40 contrasta lo planificado en los Capítulos I y III con lo que efectivamente se entrega. Se incluye para que el alcance del producto se evalúe con transparencia.

Tabla 4-40. Contraste entre lo planificado y lo implementado

Elemento planificado	Estado	Observación
Rutinas personalizadas con IA mediante TensorFlow Lite	Completo, con salvedad	Un modelo entrenado decide la estructura de la rutina en el dispositivo y las reglas actúan como respaldo. Se entrenó con perfiles sintéticos etiquetados por las reglas, por lo que imita el criterio experto y no se validó con resultados de usuarios reales.
Guía y escaneo de máquinas con realidad aumentada	Parcial	Implementado: reconocimiento local (TensorFlow Lite, 399 fotos de catálogo) o en la nube (detector YOLO26m con cajas) y anclaje de la ficha con ARCore en un módulo nativo propio, con cámara convencional como alternativa. Verificado en un dispositivo físico: el seguimiento espacial y la consulta al servicio de reconocimiento funcionaron correctamente durante las pruebas. El anclaje se fija a un punto de una superficie, no a la geometría de la máquina, y el reconocimiento ocurre por captura, no de forma continua. El acierto es modesto (38,5 % del modelo local en campo simulado; detector con mAP50 de 48,5 % en validación): solo se acepta automáticamente la máquina con confianza $\geq 0{,}8$ y en los demás casos el usuario confirma.
Registro nutricional (solo IMC según el alcance)	Ampliado	Se añadió un diario de alimentación con USDA y metas diarias personalizadas con la ecuación de Mifflin-St Jeor.
Progreso y reportes	Completo	Gráfico de peso e IMC, variación, categoría, seguimiento de entrenamientos con racha y reporte en PDF por período, verificado en el dispositivo de prueba (CP-41).
Sincronización en tiempo real	Completo	Escuchas *onSnapshot*; no se configuró la persistencia de datos sin conexión.
Android 7.0 (API 24) o superior	Verificado en Android 15	La aplicación funcionó correctamente en el dispositivo de prueba con Android 15. Esta comprobación corresponde a esa versión; la compatibilidad con Android 7.0 no se verificó directamente.
Componente de servidor (no previsto en la planificación inicial)	Agregado	Las funciones `buscarAlimentos`, `guardarRutina` y `reconocerMaquina` (Cloud Functions) protegen la clave de USDA, validan la creación de rutinas y ocultan la clave del servicio de reconocimiento; añaden dependencia de la infraestructura de Firebase y de Ultralytics.
Dispositivos externos (*smartwatch*)	Fuera de alcance	Excluidos por el alcance del proyecto.

CAPÍTULO V: RESULTADOS Y DISCUSIÓN

Este capítulo reúne los resultados del proyecto, los contrasta con los objetivos específicos y con los antecedentes citados, y declara las amenazas a la validez que limitan su interpretación. Las cifras de inteligencia artificial proceden de los archivos de resultados del repositorio (`ml/results/`); cuando una medida no se realizó se indica de forma explícita.

5.1. Resultados por objetivo específico

La Tabla 5-1 resume, para cada objetivo específico, lo que se construyó, la evidencia disponible y el grado de cumplimiento.

Tabla 5-1. Cumplimiento de los objetivos específicos

Objetivo	Resultado	Cumplimiento
OE1. Guiar sobre el uso correcto de máquinas	Catálogo de 35 tipos de máquinas con guía de uso, errores comunes y prevención de lesiones; escáner con reconocedor local (acierto top-1 de 38,5 % en campo simulado, Tabla 4-30), detector en la nube con cajas (YOLO26m, mAP50 de 48,5 % en validación, Tabla 4-34) y ficha anclada en el entorno con ARCore.	Parcial: guía completa; reconocimiento modesto y sin prueba en gimnasios reales; anclaje implementado pero no validado en un teléfono.
OE2. Instruir en la ejecución correcta	95 animaciones de ejecución con músculo objetivo y dificultad, asociadas a las máquinas; entrenamiento guiado paso a paso.	Parcial: contenido instructivo, sin verificación automática de la postura.
OE3. Rutinas personalizadas y estructuradas	Generador basado en reglas con recomendador de TensorFlow Lite y respaldo por reglas (Tabla 4-36).	Cumplido; la calidad de las rutinas para el usuario no se evaluó.
OE4. Herramientas de planificación	Plan semanal por días disponibles y registro de la racha de entrenamiento.	Cumplido.
OE5. Registro de ingesta calórica	Búsqueda de alimentos en FoodData Central mediante una Cloud Function con cuotas por usuario, cálculo de porciones y del IMC.	Cumplido dentro del alcance (conteo calórico, no consejo nutricional).
OE6. Seguimiento del progreso físico	Registro de peso, estatura y edad con cálculo y clasificación del IMC, sincronizado con Firestore.	Cumplido.
OE7. Reportes personalizados	Reportes de progreso y metas; reporte en PDF por período.	Cumplido: el PDF se generó y se abrió en el dispositivo de prueba (CP-41, Figura 4-20).

Fuente: Elaboración propia a partir del capítulo IV y del repositorio del proyecto.

5.2. Resultados de los modelos de inteligencia artificial

5.2.1. Reconocimiento de máquinas

Con 399 fotografías de 35 clases, validación cruzada de cinco pliegues y 30 aumentos por fotografía, el modelo acierta la máquina en el 38,5 % de los casos en el escenario de campo simulado y en el 61,4 % entre sus tres primeras sugerencias, frente al 2,9 % que lograría el azar (Tabla 4-30). Al fijar el umbral de confianza en 0,8, la aplicación acepta automáticamente solo el 15,8 % de las imágenes, con una precisión del 85,4 % (Tabla 4-31).

El análisis por clase muestra que el rendimiento es muy desigual: de las 33 clases evaluables, 25 quedan por debajo del 50 % de acierto en el campo simulado, mientras que la caminadora alcanza el 92,9 % y la bicicleta estática el 66,7 %. Las confusiones se concentran entre máquinas de aspecto similar: el 50 % de las muestras de validación de la máquina combinada de extensión y curl de piernas se clasificó como curl femoral, y el 26,5 % de las de press de hombro como press de pecho (`matriz_confusion_campo` en `recognition_metrics.json`). Este patrón es coherente con un conjunto de entrenamiento pequeño y con imágenes de catálogo, y sugiere que la mejora proviene de más fotografías tomadas en gimnasios reales antes que de una arquitectura distinta (Sandler et al., 2018).

5.2.2. Detector de máquinas en la nube

El detector de la nube responde a una limitación distinta a la del clasificador local: además de la clase, entrega la caja de la máquina en la imagen, que es lo que permite anclar la ficha en el entorno. El proyecto tuvo dos versiones. La primera (YOLO11x, entrenada en Colab con un conjunto anotado en Roboflow) obtuvo, en una partición de prueba de 43 imágenes, una precisión del 79,8 %, una exhaustividad del 29,9 % y una mAP50 del 39,8 % (Tabla 4-32). La versión final (YOLO26m, entrenada y publicada en Ultralytics Platform con un conjunto anotado con polígonos) alcanzó en la partición de validación una precisión del 57,7 %, una exhaustividad del 45,0 %, una mAP50 del 48,5 % y una mAP50–95 del 44,8 % (Tabla 4-34).

Frente a la primera versión, sube la exhaustividad (de 29,9 % a 45,0 %) y la mAP50 (de 39,8 % a 48,5 %), y baja la precisión (de 79,8 % a 57,7 %): el detector final encuentra más máquinas, pero también señala algunas que no son o se equivoca de clase con más frecuencia. La comparación es indicativa y no concluyente. Los conjuntos de datos y las particiones son distintos (prueba frente a validación) y la validación también sirve para elegir el punto de control, lo que favorece a la versión final. Además, la pérdida de clasificación en validación es unas diez veces la de entrenamiento, una señal habitual de sobreajuste. Lo que sí permiten afirmar estos resultados es que el procedimiento completo (anotación, entrenamiento, publicación del servicio y consumo desde la aplicación) está construido y es repetible.

Tampoco son comparables con las del clasificador local, que se evaluó con validación cruzada sobre fotografías de catálogo de 35 clases y que siempre devuelve una máquina, mientras que el detector puede no encontrar ninguna. Con la evidencia disponible no puede afirmarse que el detector supere al clasificador local; esa comparación exige evaluar ambos con las mismas fotografías de un gimnasio real (sección 5.4). La literatura sobre detectores de la familia YOLO reporta sus resultados sobre conjuntos de referencia de gran tamaño, como COCO (Terven et al., 2023); el proyecto parte de un conjunto propio mucho más pequeño, y el rendimiento limitado que se observa es el que cabe esperar en ese régimen.

5.2.3. Recomendador de rutinas

El recomendador reproduce las reglas con una exactitud del 99,8 % en datos sintéticos y del 97,1 % en una cuadrícula exhaustiva de 28 512 perfiles (Tabla 4-36). Dado que las etiquetas se

generaron con las mismas reglas, estas cifras miden la fidelidad del modelo y no demuestran que sus rutinas sean mejores que las del generador por reglas, cuya base de conocimiento se apoya en la literatura de prescripción del entrenamiento de fuerza (Kraemer & Ratamess, 2004). La ventaja práctica del modelo es arquitectónica: demuestra la integración de inferencia local en el dispositivo con respaldo por reglas.

5.3. Discusión

5.3.1. Frente al problema de adherencia

La introducción parte del abandono temprano en los gimnasios. La evidencia revisada indica que menos del 40 % de los socios mantiene un ejercicio regular a largo plazo (Gjestvang et al., 2021), por lo que las barreras de orientación, estructura y seguimiento que abordan los objetivos específicos son pertinentes. Sin embargo, este proyecto no midió si la aplicación mejora la adherencia: la hipótesis queda sin contrastar y requiere un estudio con usuarios reales, consentimiento informado y un periodo de seguimiento (sección 6.2).

5.3.2. Frente a la visión artificial para la ejecución de ejercicios

Silva Amado (2024) propone un sistema de visión artificial que estima la pose con MediaPipe, calcula ángulos articulares y los compara con umbrales para identificar la ejecución de ejercicios, y lo evalúa de forma cualitativa. Los enfoques son complementarios: aquel verifica *cómo* se ejecuta un movimiento; el presente trabajo reconoce *con qué máquina* se entrena, genera la rutina y registra el progreso, y aporta métricas cuantitativas reproducibles. En consecuencia, la instrucción de la ejecución correcta (OE2) se resuelve aquí con contenido guiado y no con verificación automática de la postura, que se identifica como línea de trabajo futuro.

5.3.3. Sobre la realidad aumentada

El escáner combina lo real y lo virtual en todos los dispositivos, lo que cumple la primera característica de la realidad aumentada descrita por Azuma (1997). En los teléfonos compatibles, el módulo `CoachAR` añade el registro en tres dimensiones: una vez reconocida la máquina, la ficha se fija a un punto del entorno mediante un ancla de ARCore y se mantiene allí mientras la persona camina o gira el teléfono (sección 4.3.4). Ese es un avance frente a la versión anterior, que solo superponía la ficha en pantalla. Con todo, la afirmación debe acotarse en tres sentidos. Primero, se comprobó su funcionamiento junto con el modelo de reconocimiento en un dispositivo físico; esta prueba funcional no permite generalizar el resultado a otros equipos ni cuantificar la precisión del anclaje. Segundo, el ancla se fija a un punto de una superficie estimado a partir de la caja del detector y no a la geometría de la máquina; no corrige errores de profundidad ni sigue a la máquina si se desplaza. Tercero, el reconocimiento sigue ocurriendo por captura y no de forma continua, así que la interacción en tiempo real es parcial. Además, ARCore solo funciona en teléfonos certificados; por eso la cámara convencional se mantiene como alternativa. Hasta contar con mediciones (error de colocación, pérdidas de seguimiento, modelos de teléfono probados), no hay evidencia de que la realidad aumentada mejore la experiencia ni la adherencia; la revisión sistemática de Vinolo Gil et al. (2021), en un contexto distinto (fisioterapia), concluye que su eficacia es potencial pero no concluyente.

5.4. Amenazas a la validez y limitaciones

- **Validez de constructo del reconocedor.** Las 399 fotografías provienen de catálogos y no de gimnasios; el escenario de campo simulado aproxima, pero no reemplaza, las condiciones reales de luz, encuadre y oclusión.
- **Validez del detector.** Las métricas de la versión final (YOLO26m) se calcularon sobre la

partición de validación, que también sirve para elegir el punto de control, y no sobre una partición de prueba independiente; la pérdida de validación, muy superior a la de entrenamiento, sugiere sobreajuste. La primera versión se evaluó en solo 43 imágenes y 65 cajas, de las cuales 24 pertenecían a una sola clase; varias imágenes parecían series de la misma máquina desde ángulos cercanos. Ninguna versión se midió en un teléfono: no se conoce la latencia del *endpoint*, que escala a cero y puede tardar casi un minuto en responder tras un período inactivo (Ultralytics, s. f.-b). El dispositivo de prueba es uno solo (Tabla 4-35), de gama media, y no se probaron otros equipos. El umbral de confianza de 0,8 se calibró para el modelo local y no se recalibró para el detector.

- **Validación de la realidad aumentada.** Se comprobó el funcionamiento integrado de ARCore y el modelo de reconocimiento en un dispositivo físico. No se realizaron mediciones del error de colocación ni de la latencia. Además, la detección de superficies depende de los rasgos visuales de la escena, por lo que suelos uniformes y espejos de un gimnasio podrían dificultarla (Google for Developers, s. f.-b).
- **Dependencia de un servicio externo y su licencia.** El modo en la nube depende de la disponibilidad de Ultralytics Platform. La biblioteca Ultralytics se distribuye con licencia AGPL-3.0 o empresarial (Ultralytics, s. f.-c), por lo que, si el producto se publica más allá del ámbito académico, habrá que revisar qué licencia corresponde.
- **Circularidad del recomendador.** Los datos sintéticos están etiquetados por las reglas que el modelo imita; la métrica no evalúa la utilidad para el usuario.
- **Ausencia de evaluación con usuarios.** No se midieron usabilidad, satisfacción ni adherencia.
- **Cobertura de pruebas.** Las pruebas automatizadas cubren la lógica pura, la interpretación de la respuesta del reconocimiento, la geometría de la vista de ARCore y las reglas de seguridad; no cubren la interfaz ni se midieron la latencia y la exactitud de los modelos en un dispositivo físico.
- **Plataforma y conectividad.** El sistema funciona en Android y requiere conexión a internet; la persistencia sin conexión no está configurada.
- **Alcance nutricional.** El registro de alimentos se limita al conteo calórico y no sustituye la supervisión de un nutricionista.

CAPÍTULO VI: CONCLUSIONES Y RECOMENDACIONES

6.1. Conclusiones

El presente proyecto de grado desarrolló un sistema para el entrenamiento en gimnasios que proporciona una guía estructurada y un seguimiento personalizado de los ejercicios, con lo que se cumple el objetivo general planteado. El producto es una aplicación Android construida con React Native, Expo y TypeScript sobre los servicios de Firebase, organizada en una arquitectura cliente–servidor con capas Modelo–Vista–Controlador y desarrollada con la metodología Mobile-D en seis iteraciones. Dos de sus componentes, la inteligencia artificial y la realidad aumentada, se implementaron con un alcance acotado, como se detalla a continuación: la primera mediante modelos de TensorFlow Lite entrenados para este proyecto y un detector YOLO26m publicado en Ultralytics Platform, cuya validación se limita a datos sintéticos, de catálogo y a las particiones del propio entrenamiento; la segunda mediante un módulo nativo basado en ARCore que ancla la ficha de la máquina en el entorno de los teléfonos compatibles y, en los demás, una superposición en pantalla sobre la cámara. Según la definición de Azuma (1997), el anclaje aporta el registro tridimensional, pero aún no se ha validado en un teléfono físico y el reconocimiento sigue ocurriendo por captura y no de forma continua (sección 2.6.3).

Respecto a cada objetivo específico, se concluye lo siguiente:

OE1. Guiar sobre el uso correcto de máquinas. Se construyó un catálogo de 35 tipos de máquinas, con 288 modelos comerciales como variantes, en el que cada ficha incluye descripción, músculos trabajados, guía de uso paso a paso, errores comunes y pautas de prevención de lesiones redactadas como contenido propio. El catálogo se consulta con búsqueda tolerante a tildes y filtros, y se complementa con un escáner que superpone la ficha de la máquina sobre la vista de la cámara y, en los teléfonos compatibles con ARCore, la ancla a un punto del entorno. Un clasificador de TensorFlow Lite (MobileNetV2 con una capa entrenada), entrenado con las 399 fotografías del catálogo, propone la máquina a partir de una foto; su acierto es modesto (38,5 % en un escenario de campo simulado de 35 clases y 61,4 % entre las tres primeras sugerencias), por lo que solo se acepta sin preguntar con una confianza de al menos 0,8, umbral con el que la validación cruzada arrojó una precisión del 85,4 % sobre el 15,8 % de las fotografías; en los demás casos, el usuario confirma la sugerencia o elige la máquina en una lista. Como alternativa opcional, un detector YOLO26m entrenado y publicado como *endpoint* en Ultralytics Platform devuelve además la caja de la máquina; en validación alcanzó una mAP50 de 48,5 % y una exhaustividad de 45,0 %, cifras que muestran que el procedimiento funciona de extremo a extremo, no que el detector sea fiable en un gimnasio. El reconocimiento se ejecuta sobre una fotografía fija y su latencia en el dispositivo no se ha medido.

OE2. Instruir en la ejecución correcta de ejercicios. Cada máquina presenta al menos un ejercicio con su animación de ejecución, su músculo objetivo y su nivel de dificultad, con animaciones y pasos ilustrados procedentes de varias fuentes de datos: WorkoutX (s. f.), ExerciseDB (AscendAPI, s. f.), Gym Visual (s. f.), wger (wger Project, s. f.) y free-exercise-db (Yuhonas, s. f.), que se complementan con una guía de entrenamiento paso a paso. El sistema no verifica automáticamente la postura del usuario.

OE3. Diseñar rutinas personalizadas. Se implementó un generador de rutinas basado en reglas cuya base de conocimiento traduce las pautas de prescripción del ejercicio en reglas de frecuencia, de variables de carga y de división semanal, y que se ajusta con la edad, el IMC y los días disponibles del usuario. Sobre esas reglas se entrenó un modelo de

TensorFlow Lite que decide la plantilla semanal y el esquema de carga y se ejecuta en el dispositivo; si su confianza es menor que 0,6 o falla, la aplicación usa las reglas. Como no existen datos de usuarios reales con resultados conocidos, el modelo se entrenó con 60 000 perfiles sintéticos etiquetados por las propias reglas: reproduce las reglas con un 99,8 % de exactitud, pero eso mide su fidelidad al criterio experto y no demuestra que sus rutinas sean mejores para los usuarios. Las pruebas sobre las doce combinaciones de objetivo y nivel confirmaron que cada rutina generada respeta las reglas.

OE4. Construir herramientas de planificación. El planificador de rutinas personalizadas permite organizar la semana día por día, ajustar series, repeticiones y descansos, mantener varias rutinas con una activa y agregar ejercicios directamente desde la ficha de una máquina.

OE5. Registrar la ingesta de alimentos. El alcance inicial limitaba este objetivo al conteo calórico mediante una calculadora de IMC; el sistema lo supera. Además de calcular y clasificar el IMC según la OMS, permite buscar alimentos en español sobre la base de USDA FoodData Central, calcular calorías y macronutrientes por porción y compararlos con metas diarias que se personalizan con la ecuación de Mifflin-St Jeor según los datos físicos, los días de entrenamiento y el objetivo del usuario. Son estimaciones referenciales y no sustituyen la indicación de un nutricionista.

OE6. Implementar el seguimiento del progreso físico. El usuario registra su peso y su estatura, el sistema calcula el IMC y el historial se sincroniza en tiempo real. Además, el usuario marca los días de su rutina que completa y el sistema calcula su racha, los días entrenados de la semana y el total de entrenamientos.

OE7. Generar reportes de progreso. El reporte presenta la evolución del peso y del IMC en un gráfico, la variación acumulada y la categoría de IMC actual, calculados al vuelo a partir del historial para reflejar siempre los datos vigentes. Además, el usuario puede generar un reporte en PDF por período (7, 30, 90 días o todo el historial) con su perfil, su peso, sus entrenamientos y su alimentación frente a la meta, y compartirlo desde el sistema; su generación se verificó en el dispositivo de prueba.

En el plano técnico, se concluye además que:

- La separación en capas resultó determinante para la mantenibilidad: al aislar el acceso a Firebase en los repositorios y servicios, cambios como el paso de colecciones raíz a subcolecciones por usuario, o la incorporación de nuevas fuentes de datos como USDA, se resolvieron sin modificar las pantallas.
- Las reglas de seguridad y las funciones del servidor son indispensables en una arquitectura basada en servicios gestionados, porque las credenciales del cliente son públicas por diseño. La incorporación de dos funciones (`buscarAlimentos` y `guardarRutina`) con App Check, rechazo de usuarios anónimos y cuotas por usuario permitió ocultar la clave de USDA y validar en el servidor la creación de rutinas. Esta protección aún no cuenta con resultados de ejecución de las pruebas con emulador (sección 4.4.1.3).
- Concentrar las reglas de negocio en funciones puras (generador de rutinas, IMC, racha de entrenamientos, porciones de alimentos) permitió verificarlas de forma automática y reproducible sin depender de la red ni de la base de datos.
- Aislar los modelos de aprendizaje automático tras un contrato definido (las clases que producen y el vector de entrada) y conservar las reglas como respaldo permitió integrarlos sin afectar al resto del sistema y garantiza que la aplicación siga funcionando si un modelo falla. Además, verificar con pruebas automáticas que el orden de las clases en TypeScript coincida con el del *script* de entrenamiento evita un error silencioso al reentrenar.

- Una métrica alta no basta: el recomendador de rutinas alcanza un 99,8 % porque imita reglas conocidas, mientras que el reconocedor de máquinas, entrenado con pocas fotografías, acierta menos del 40 % en campo simulado. Informar ambas cifras con su contexto fue necesario para no sobrestimar el aporte de la inteligencia artificial.

Como limitaciones del producto actual cabe señalar que las pruebas automatizadas cubren la lógica pura y los componentes de servicio (20 archivos de prueba en la aplicación, con 160 casos aprobados el 3 de noviembre de 2026) pero no la interfaz, que la persistencia de datos sin conexión no está configurada, que los modelos no se han medido en un dispositivo ni con usuarios reales, que el anclaje con ARCore no se ha validado en un teléfono físico y que el ancla no sigue a la máquina si esta se mueve, que los reconocedores aciertan poco con las máquinas menos distintivas, que el uso del detector en la nube depende de un servicio externo con licencia propia, y que las metas nutricionales son estimaciones referenciales.

6.2. Recomendaciones

A partir de la experiencia del desarrollo se recomienda, como trabajo futuro:

- **Recomendador de rutinas con datos reales.** Registrar, con consentimiento, el perfil de los usuarios, la rutina recibida y su adherencia y progreso, para reentrenar el modelo con resultados reales en lugar de etiquetas sintéticas y medir si sus rutinas superan a las reglas. Las rutinas ya guardan su origen, la versión del modelo y la confianza, lo que facilita este análisis.
- **Más fotografías para el reconocimiento de máquinas.** Ampliar el conjunto con fotografías propias de las 35 máquinas, tomadas en gimnasios reales y con autorización, con varias por máquina (hoy hay desde una hasta 25), y reentrenar el clasificador y el detector. En el detector, conviene ampliar el conjunto de datos y las clases con menos ejemplos, reservar una partición de prueba independiente y compararlo con otros tamaños de YOLO26 (por ejemplo, el nano, pensado para dispositivos de borde; Jocher et al., 2026) y con una versión ejecutada en el propio teléfono. Después, recalibrar el umbral de confianza y medir los modelos en dispositivos. No deben usarse las animaciones de WorkoutX para el entrenamiento, porque sus condiciones de uso lo prohíben sin permiso escrito.
- **Validar y ampliar la realidad aumentada.** El anclaje con ARCore ya está implementado, pero falta lo esencial para darlo por validado: ejecutar el protocolo de pruebas (CP-42 a CP-44) en teléfonos físicos y en un gimnasio, y medir el error de colocación, las pérdidas de seguimiento y los modelos de teléfono compatibles. Después se proponen tres líneas de trabajo, en orden creciente de complejidad: (a) ejecutar el detector sobre los fotogramas del vídeo, no solo sobre una captura, y estabilizar el resultado en el tiempo; (b) combinar la caja del detector con la profundidad de ARCore en los dispositivos que la admiten, para acercar el ancla a la geometría de la máquina; y (c) anclar indicaciones tridimensionales (puntos de ajuste, trayectoria del movimiento) sobre la máquina real (Google for Developers, s. f.-f). Revisiones del campo (Azuma et al., 2001; Billinghurst et al., 2015) señalan el registro preciso y el seguimiento como los problemas técnicos centrales, por lo que cada línea debe evaluarse con medidas de latencia y estabilidad en un dispositivo físico antes de afirmar que se logró el registro tridimensional.
- **Verificación de la postura.** Evaluar la estimación de pose corporal en el dispositivo (por ejemplo, BlazePose; Bazarevsky et al., 2020) como línea futura para retroalimentar la ejecución de los ejercicios, en la línea del antecedente de Silva Amado (2024); queda fuera del alcance actual.
- **Progresión de la carga.** Ampliar la base de conocimiento del generador con reglas de

sobrecarga progresiva que ajusten la rutina según el historial de entrenamientos del usuario.

- **Metas nutricionales más precisas.** Permitir que el usuario indique su nivel de actividad diaria (hoy se deduce de los días de entrenamiento) y ajuste sus metas, manteniendo la advertencia de que no sustituyen a un nutricionista.
- **Persistencia sin conexión.** Configurar la caché persistente de Firestore para que la guía de máquinas y la rutina activa estén disponibles sin Internet, algo muy útil en gimnasios con mala cobertura.
- **Pruebas automatizadas de interfaz y de seguridad.** Agregar React Native Testing Library para probar las pantallas, ejecutar la suite de Jest y las pruebas con el emulador de Firebase (`npm run test:security`) en integración continua, y registrar sus resultados.
- **Privacidad y consentimiento.** Redactar la política de privacidad y el consentimiento informado para el tratamiento de los datos físicos y alimentarios, y para el uso de fotografías de usuarios si se amplía el conjunto de entrenamiento.
- **Validación con usuarios.** Realizar un estudio de usabilidad con usuarios reales de gimnasio, por ejemplo con el cuestionario SUS, y medir el efecto de la aplicación en la permanencia de los usuarios nuevos.
- **Licencia de Ultralytics.** La biblioteca y los modelos de Ultralytics se distribuyen con licencia AGPL-3.0 o con una licencia empresarial (Ultralytics, s. f.-c). Si la aplicación se publica o se comercializa, hay que definir cuál aplica, o sustituir el detector por otra alternativa, antes de la distribución.
- **Publicación.** Antes de publicar la aplicación en Google Play, obtener la autorización del proveedor de las fotografías del catálogo y confirmar las condiciones de uso de las animaciones y los datos de ExerciseDB, Gym Visual, WorkoutX y wger, en particular las restricciones de uso comercial y de atribución.

BIBLIOGRAFÍA

Abrahamsson, P., Hanhineva, A., Hulkko, H., Ihme, T., Jäälinoja, J., Korkala, M., Koskela, J., Kyllönen, P., & Salo, O. (2004). Mobile-D: An agile approach for mobile application development. En J. Vlissides (Ed.), *OOPSLA'04: Companion to the 19th annual ACM SIGPLAN conference on object-oriented programming, systems, languages, and applications* (pp. 174–175). ACM. https://doi.org/10.1145/1028664.1028736

Abstracta Team. (2024). *Metodologías de desarrollo de software: guía para optimizar tu proyecto*. Abstracta. https://es.abstracta.us/blog/metodologias-de-desarrollo-de-software-guia-completa/

Afonso, J., Ramirez-Campillo, R., Moscão, J., Rocha, T., Zacca, R., Martins, A., Milheiro, A. A., Ferreira, J., Sarmento, H., & Clemente, F. M. (2021). Strength training versus stretching for improving range of motion: A systematic review and meta-analysis. *Healthcare, 9*(4), Article 427. https://doi.org/10.3390/healthcare9040427

Alarcon, A. (s. f.). *Treadmills at gym* [Fotografía]. U.S. Air Force; Wikimedia Commons. https://commons.wikimedia.org/wiki/File:Treadmills_at_gym.jpg

Alizadeh, S., Daneshjoo, A., Zahiri, A., Anvar, S. H., Goudini, R., Hicks, J. P., Konrad, A., & Behm, D. G. (2023). Resistance training induces improvements in range of motion: A systematic review and meta-analysis. *Sports Medicine, 53*(3), 707–722. https://doi.org/10.1007/s40279-022-01804-x

Amazon Web Services. (2025). *¿Qué es la IA? Explicación de la inteligencia artificial*. https://aws.amazon.com/es/what-is/artificial-intelligence/

American College of Sports Medicine. (2009). American College of Sports Medicine position stand. Progression models in resistance training for healthy adults. *Medicine & Science in Sports & Exercise, 41*(3), 687–708. https://doi.org/10.1249/MSS.0b013e3181915670

Android Developers. (2024). *Introducción a Android Studio*. https://developer.android.com/studio/intro?hl=es-419

Apple Inc. (2023). *Apple Developer documentation*. https://developer.apple.com/

Apple Inc. (2025). *ARKit 6: Augmented reality*. Apple Developer. https://developer.apple.com/augmented-reality/arkit/

AppMaster. (2023, 4 de septiembre). *Programación móvil*. https://appmaster.io/es/glossary/programacion-movil

Apps, L. B. (2011). *Guía de aplicaciones móviles*. Fundación Telefónica.

AscendAPI. (s. f.). *ExerciseDB API* (Versión 1) [Interfaz de programación de aplicaciones]. https://docs.exercisedb.dev/

Atlassian. (2023). *About us: Trello history, logos & customers | Trello*. https://trello.com/about

Azuma, R. T. (1997). A survey of augmented reality. *Presence: Teleoperators and Virtual Environments, 6*(4), 355–385. https://doi.org/10.1162/pres.1997.6.4.355

Azuma, R., Baillot, Y., Behringer, R., Feiner, S., Julier, S., & MacIntyre, B. (2001). Recent advances in augmented reality. *IEEE Computer Graphics and Applications, 21*(6), 34–47. https://doi.org/10.1109/38.963459

B., G. (2025, 13 de marzo). *Qué es GitHub y cómo empezar a usarlo*. Hostinger Tutoriales. https://www.hostinger.com/es/tutoriales/que-es-github

Bazarevsky, V., Grishchenko, I., Raveendran, K., Zhu, T., Zhang, F., & Grundmann, M. (2020). *BlazePose: On-device real-time body pose tracking* [Preimpresión]. arXiv. https://arxiv.org/abs/2006.10204

Beck, K., & Andres, C. (2005). *Extreme programming explained: Embrace change* (2.ª ed.). Addison-Wesley.

Betts, J. G., Young, K. A., Wise, J. A., Johnson, E., Poe, B., Kruse, D. H., Korol, O., Johnson, J. E., Womble, M., & DeSaix, P. (2022). *Anatomy and physiology 2e*. OpenStax. https://openstax.org/details/books/anatomy-and-physiology-2e

Billinghurst, M., Clark, A., & Lee, G. (2015). A survey of augmented reality. *Foundations and Trends in Human–Computer Interaction, 8*(2–3), 73–272. https://doi.org/10.1561/1100000049

Burrows, T. L., Ho, Y. Y., Rollo, M. E., & Collins, C. E. (2019). Validity of dietary assessment methods when compared to the method of doubly labeled water: A systematic review in adults. *Frontiers in Endocrinology, 10*, Article 850. https://doi.org/10.3389/fendo.2019.00850

Canadian Food Inspection Agency. (2025, 3 de junio). *Elements within the Nutrition Facts table*. Recuperado el 4 de noviembre de 2026, de https://inspection.canada.ca/en/food-labels/labelling/industry/nutrition-labelling/elements-within-nutrition-facts-table

Centers for Disease Control and Prevention. (2024, 19 de marzo). *Adult BMI categories*. Recuperado el 4 de noviembre de 2026, de https://www.cdc.gov/bmi/adult-calculator/bmi-categories.html

Centers for Disease Control and Prevention. (2025, 16 de diciembre). *About body mass index (BMI)*. https://www.cdc.gov/bmi/about/index.html

Coleman, M., Burke, R., Benavente, C., Piñero, A., Augustin, F., Maldonado, J., Fisher, J. P., Oberlin, D., Vigotsky, A. D., & Schoenfeld, B. J. (2023). Supervision during resistance training positively influences muscular adaptations in resistance-trained individuals. *Journal of Sports Sciences, 41*(12), 1207–1217. https://doi.org/10.1080/02640414.2023.2261090

Cruz. (2024). **[PENDIENTE:** completar título, tipo de trabajo (proyecto de grado), universidad y enlace**]**

Cuadrado, G. C. (2022, 22 de julio). *Qué es Visual Studio Code y qué ventajas ofrece*. OpenWebinars. https://openwebinars.net/blog/que-es-visual-studio-code-y-que-ventajas-ofrece/

Currier, B. S., McLeod, J. C., Banfield, L., Beyene, J., Welton, N. J., D'Souza, A. C., Keogh, J. A. J., Lin, L., Coletta, G., Yang, A., Colenso-Semple, L., Lau, K. J., Verboom, A., & Phillips, S. M. (2023). Resistance training prescription for muscle strength and hypertrophy in healthy adults: A systematic review and Bayesian network meta-analysis. *British Journal of Sports Medicine, 57*(18), 1211–1220. https://doi.org/10.1136/bjsports-2023-106807

Deloitte. (2019). *¿Qué es React Native?* Deloitte España. https://www.deloitte.com/es/es/services/consulting/blogs/todo-tecnologia/que-es-react-native.html

DuBois, P. (2013). *MySQL* (5.ª ed.). Addison-Wesley Professional.

Fernández. (2018). **[PENDIENTE:** completar nombre del autor, título, tipo de trabajo (proyecto de grado), universidad y enlace**]**

Flutter. (s. f.). *Flutter: crea aplicaciones para cualquier pantalla*. `https://esflutter.dev/`

Gabriel Maide, E., & Pacienzia, J. (2020, noviembre). *Metodologías de desarrollo de software aplicadas a proyectos de automatización de procesos*. Pistas Educativas. `https://pistaseducativas.celaya.tecnm.mx/index.php/pistas/article/viewFile/2381/1900`

Gironés, T. (2012). *Aplicaciones móviles: Conceptos y desarrollo.*

Git. (s. f.). *Git*. `http://git-scm.com/`

Gjestvang, C., Abrahamsen, F., Stensrud, T., & Haakstad, L. A. H. (2020). Motives and barriers to initiation and sustained exercise adherence in a fitness club setting—A one-year follow-up study. *Scandinavian Journal of Medicine & Science in Sports, 30*(9), 1796–1805. `https://doi.org/10.1111/sms.13736`

Gjestvang, C., Stensrud, T., Paulsen, G., & Haakstad, L. A. H. (2021). Stay true to your workout: Does repeated physical testing boost exercise attendance? A one-year follow-up study. *Journal of Sports Science and Medicine, 20*(1), 35–44. `https://doi.org/10.52082/jssm.2021.35`

Gómez, & Hernández. (2016). Mobile-D: Metodología ágil para el desarrollo de aplicaciones móviles. *Revista Orbis*.

Gómez, A. A. (2015). *Comparación de dos tecnologías de desarrollo de aplicaciones móviles desde la perspectiva de los atributos de calidad* [Tesis de maestría]. Universidad Nacional de Colombia.

Google Cloud. (2025). *Inteligencia artificial (IA): una guía fácil de entender*. `https://cloud.google.com/learn/what-is-artificial-intelligence?hl=es-419`

Google for Developers. (s. f.-a). *ARCore supported devices* [Documentación de ARCore]. `https://developers.google.com/ar/devices`

Google for Developers. (s. f.-b). *Depth adds realism* [Documentación de ARCore]. `https://developers.google.com/ar/develop/depth`

Google for Developers. (s. f.-c). *Enable AR in your Android app* [Documentación de ARCore]. `https://developers.google.com/ar/develop/java/enable-arcore`

Google for Developers. (s. f.-d). *Frame* [Referencia de la API de ARCore para Java]. `https://developers.google.com/ar/reference/java/com/google/ar/core/Frame`

Google for Developers. (s. f.-e). *Fundamental concepts* [Documentación de ARCore]. `https://developers.google.com/ar/develop/fundamentals`

Google for Developers. (s. f.-f). *Working with anchors* [Documentación de ARCore]. `https://developers.google.com/ar/develop/anchors`

Google for Developers. (2025). *ARCore: Crea nuevas experiencias de realidad aumentada*. `https://developers.google.com/ar?hl=es-419`

Google. (2023). *Android Developers: Official documentation*. `https://developer.android.com/`

Grgic, J., Schoenfeld, B. J., Skrepnik, M., Davies, T. B., & Mikulic, P. (2018). Effects of rest interval duration in resistance training on measures of muscular strength: A systematic review.

Sports Medicine, 48(1), 137–151. https://doi.org/10.1007/s40279-017-0788-x

Gurnov, A., & Wrike. (2025, 13 de noviembre). *Las 21 mejores herramientas de gestión de proyectos*. Wrike. https://www.wrike.com/es/project-management-guide/faq/que-son-las-herramientas-de-gestion-de-proyectos/

Gym Visual. (s. f.). *Gym Visual: exercise animations and images* [Sitio web]. https://gymvisual.com/

Haugen, M. E., Vårvik, F. T., Larsen, S., Haugen, A. S., van den Tillaar, R., & Bjørnsen, T. (2023). Effect of free-weight vs. machine-based strength training on maximal strength, hypertrophy and jump performance—A systematic review and meta-analysis. *BMC Sports Science, Medicine and Rehabilitation, 15*, Article 103. https://doi.org/10.1186/s13102-023-00713-4

Hayes, M., & Downie, A. (2025, 28 de noviembre). *Realidad aumentada*. IBM. https://www.ibm.com/es-es/think/topics/augmented-reality

Huang, S. (2020). *NoSQL databases: Concepts and applications*. Morgan Kaufmann.

IBM. (2025). *¿Qué son las redes neuronales?* https://www.ibm.com/es-es/think/topics/neural-networks

Ionic. (2023). *Ionic Framework*. Ionic Documentation. https://ionicframework.com/docs

Isaac. (2025). *Firebase: qué es y cómo funciona la plataforma de Google*. Mundobytes. https://mundobytes.com/firebase-que-es/

Jäger, R., Kerksick, C. M., Campbell, B. I., Cribb, P. J., Wells, S. D., Skwiat, T. M., Purpura, M., Ziegenfuss, T. N., Ferrando, A. A., Arent, S. M., Smith-Ryan, A. E., Stout, J. R., Arciero, P. J., Ormsbee, M. J., Taylor, L. W., Wilborn, C. D., Kalman, D. S., Kreider, R. B., Willoughby, D. S., . . . Antonio, J. (2017). International Society of Sports Nutrition position stand: Protein and exercise. *Journal of the International Society of Sports Nutrition, 14*, Article 20. https://doi.org/10.1186/s12970-017-0177-8

Jayedi, A., Soltani, S., Emadi, A., Zargar, M.-S., & Najafi, A. (2024). Aerobic exercise and weight loss in adults: A systematic review and dose-response meta-analysis. *JAMA Network Open, 7*(12), e2452185. https://doi.org/10.1001/jamanetworkopen.2024.52185

Jocher, G., Qiu, J., & Chaurasia, A. (2023). *Ultralytics YOLO* (Versión 8.0.0) [Software]. https://github.com/ultralytics/ultralytics

Jocher, G., Qiu, J., Liu, M., Lyu, S., Akyon, F. C., & Kalfaoglu, M. E. (2026). *Ultralytics YOLO26: Unified real-time end-to-end vision models* [Preimpresión]. arXiv. https://doi.org/10.48550/arXiv.2606.03748

KeepActive Australia. (2023, 29 de diciembre). *Dumbells and free weights in a gym* [Fotografía]. Wikimedia Commons. https://commons.wikimedia.org/wiki/File:Dumbells_and_free_weights_in_a_gym.jpg

Khanam, R., & Hussain, M. (2024). *YOLOv11: An overview of the key architectural enhancements* [Preimpresión]. arXiv. https://arxiv.org/abs/2410.17725

Kolt, G. S., & Snyder-Mackler, L. (Eds.). (2007). *Physical therapies in sport and exercise* (2.ª ed.). Churchill Livingstone Elsevier.

Kraemer, W. J., & Ratamess, N. A. (2004). Fundamentals of resistance training: Progression and exercise prescription. *Medicine & Science in Sports & Exercise, 36*(4), 674–688. https://doi.org/10.1249/01.MSS.0000121945.36635.61

Laux, E., Storm, J., & Prado, S. (2016). *BodBot AI entrenador personal* [Aplicación móvil]. https://www.bodbot.com/

Leap Fitness Group. (2021). *Ejercicios en el Gimnasio* [Aplicación móvil]. Google Play. https://play.google.com/store/apps/details?id=gymworkout.gym.gymlog.gymtrainer&hl=es_BO

Leyva, R., León, G., & García, F. (2016). Metodología Mobile-D para el desarrollo de aplicaciones móviles. *Revista Cubana de Ciencias Informáticas, 10*(3), 45–57.

López Mora, S. (2025, noviembre). *Firebase: qué es, para qué sirve, funcionalidades y ventajas*. DIGITAL55. https://digital55.com/blog/que-es-firebase-funcionalidades-ventajas-conclusiones/

Lorenzetti, S., Ostermann, M., Zeidler, F., Zimmer, P., Jentsch, L., List, R., Taylor, W. R., & Schellenberg, F. (2018). How to squat? Effects of various stance widths, foot placement angles and level of experience on knee, hip and trunk motion and loading. *BMC Sports Science, Medicine and Rehabilitation, 10*, Article 14. https://doi.org/10.1186/s13102-018-0103-7

Mayo Clinic. (2025, 21 de noviembre). *Video: Chest press with weight machine*. https://www.mayoclinic.org/healthy-lifestyle/fitness/multimedia/chest-press/vid-20084687

Mayo Clinic. (s. f.). *Weight training: Do's and don'ts of proper technique*. Recuperado el 4 de noviembre de 2026, de https://www.mayoclinic.org/healthy-lifestyle/fitness/in-depth/weight-training/art-20045842

McCrary, J. M., Ackermann, B. J., & Halaki, M. (2015). A systematic review of the effects of upper body warm-up on performance and injury. *British Journal of Sports Medicine, 49*(14), 935–942. https://doi.org/10.1136/bjsports-2014-094228

Meier, R., & Lake, I. (2018). *Professional Android* (4.ª ed.). Wrox.

Microsoft. (2022). *Xamarin documentation*. Microsoft Learn. https://learn.microsoft.com/en-us/xamarin/

Mifflin, M. D., St Jeor, S. T., Hill, L. A., Scott, B. J., Daugherty, S. A., & Koh, Y. O. (1990). A new predictive equation for resting energy expenditure in healthy individuals. *The American Journal of Clinical Nutrition, 51*(2), 241–247. https://doi.org/10.1093/ajcn/51.2.241

Mitchell, T. M. (1997). *Machine learning*. McGraw-Hill.

MongoDB, Inc. (2024). *MongoDB Atlas documentation*. https://www.mongodb.com/docs/atlas/

National Center for Health Statistics. (2021). *National Health and Nutrition Examination Survey (NHANES): Anthropometry procedures manual*. Centers for Disease Control and Prevention. https://wwwn.cdc.gov/nchs/data/nhanes/public/2021/manuals/2021-Anthropometry-Procedures-Manual-508.pdf

National Heart, Lung, and Blood Institute. (2022, 24 de marzo). *Overweight and obesity: Causes and risk factors*. Recuperado el 4 de noviembre de 2026, de https://www.nhlbi.nih.gov/health/overweight-and-obesity/causes

Navarro, F., Martínez, A., & Martínez, J. M. (2018). *Realidad virtual y realidad aumentada: Desarrollo de aplicaciones*. Ra-Ma.

Nielsen, J. (1993). *Usability engineering*. Academic Press.

Open Handset Alliance. (2022). *Android Open Source Project overview*. https://source.android.com/?hl=es-419

Oracle Corporation. (2024). *MySQL documentation*. MySQL. https://dev.mysql.com/doc/

Organización Mundial de la Salud. (2000). *Obesity: Preventing and managing the global epidemic. Report of a WHO consultation* (WHO Technical Report Series, 894).

Organización Mundial de la Salud. (2020). *WHO guidelines on physical activity and sedentary behaviour*. https://www.who.int/publications/i/item/9789240015128

Organización Mundial de la Salud. (2024, 26 de junio). *Physical activity*. https://www.who.int/news-room/fact-sheets/detail/physical-activity

Orientanet. (s. f.). *¿Qué es entrenamiento físico y tipos?* https://www.orientanet.es/que-es-entrenamiento-fisico-y-tipos/

Padilla, R., Passos, W. L., Dias, T. L. B., Netto, S. L., & da Silva, E. A. B. (2021). A comparative analysis of object detection metrics with a companion open-source toolkit. *Electronics, 10*(3), Artículo 279. https://doi.org/10.3390/electronics10030279

Plotkin, D., Coleman, M., Van Every, D., Maldonado, J., Oberlin, D., Israetel, M., Feather, J., Alto, A., Vigotsky, A. D., & Schoenfeld, B. J. (2022). Progressive overload without progressing load? The effects of load or repetition progression on muscular adaptations. *PeerJ, 10*, e14142. https://doi.org/10.7717/peerj.14142

Pressman, R. S. (2010). *Software engineering: A practitioner's approach* (7.ª ed.). McGraw-Hill Higher Education.

Pykes, K. (2024). *Aprendizaje automático en Python: tutorial de Scikit-Learn*. DataCamp. https://www.datacamp.com/es/tutorial/machine-learning-python

React Native. (2025). *Core components and native components*. Meta. https://reactnative.dev/docs/intro-react-native-components

Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You only look once: Unified, real-time object detection. En *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (pp. 779–788). IEEE. https://openaccess.thecvf.com/content_cvpr_2016/html/Redmon_You_Only_Look_CVPR_2016_paper.html

Roboflow, Inc. (s. f.). *Roboflow: Computer vision tools* [Plataforma de software]. https://roboflow.com/

Rosa, A., Vazquez, G., Grgic, J., Balachandran, A. T., Orazem, J., & Schoenfeld, B. J. (2023). Hypertrophic effects of single- versus multi-joint exercise of the limb muscles: A systematic review and meta-analysis. *Strength & Conditioning Journal, 45*(1), 49–57. https://doi.org/10.1519/SSC.0000000000000720

SAgbley. (2021, 30 de abril). *Lat Pull down Machine* [Fotografía]. Wikimedia Commons. https://commons.wikimedia.org/wiki/File:Lat_Pull_down_Machine.jpg

Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., & Chen, L.-C. (2018). MobileNetV2: Inverted residuals and linear bottlenecks. En *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (pp. 4510–4520). IEEE. https://openaccess.thecvf.com/content_cvpr_2018/html/Sandler_MobileNetV2_Inverted_Residuals_CVPR_2018_paper.html

Santander Open Academy. (2020, 21 de diciembre). *Metodologías de desarrollo de software: ¿qué son?* https://www.santanderopenacademy.com/es/blog/metodologias-

desarrollo-software.html

Sanz, D. (2019). *Introducción a Android*. https://dspace.itsjapon.edu.ec/jspui/bitstream/123456789/434/1/introduccion-android.pdf

Sapkota, R., Cheppally, R. H., Sharda, A., & Karkee, M. (2025). *YOLO26: Key architectural enhancements and performance benchmarking for real-time object detection* [Preimpresión]. arXiv. https://arxiv.org/abs/2509.25164

Schoenfeld, B. J. (2010). The mechanisms of muscle hypertrophy and their application to resistance training. *Journal of Strength and Conditioning Research, 24*(10), 2857–2872. https://doi.org/10.1519/JSC.0b013e3181e840f3

Schumann, M., Feuerbacher, J. F., Sünkeler, M., Freitag, N., Rønnestad, B. R., Doma, K., & Lundberg, T. R. (2022). Compatibility of concurrent aerobic and strength training for skeletal muscle size and function: An updated systematic review and meta-analysis. *Sports Medicine, 52*(3), 601–612. https://doi.org/10.1007/s40279-021-01587-7

Schwaber, K., & Sutherland, J. (2020). *The Scrum guide: The definitive guide to Scrum: The rules of the game*. Scrum.org. https://scrumguides.org/

Silva Amado, H. S. (2024). *Sistema de visión artificial para la identificación de ejecución de ejercicios físicos* [Trabajo de grado, Universidad Santo Tomás]. Tunja, Colombia.

Singer, A., Wolf, M., Generoso, L., Arias, E., Delcastillo, K., Echevarria, E., Martinez, A., Androulakis Korakakis, P., Refalo, M. C., Swinton, P. A., & Schoenfeld, B. J. (2024). Give it a rest: A systematic review with Bayesian meta-analysis on the effect of inter-set rest interval duration on muscle hypertrophy. *Frontiers in Sports and Active Living, 6*, Article 1429789. https://doi.org/10.3389/fspor.2024.1429789

SQLite Consortium. (2024). *SQLite home page*. SQLite.org. https://sqlite.org/

StatCounter Global Stats. (2025, noviembre). *Mobile operating system market share Plurinational State of Bolivia*. https://gs.statcounter.com/os-market-share/mobile/bolivia

Teddyhtsai. (2021, 26 de octubre). *Diverging Lat Pulldown Machine* [Fotografía]. Wikimedia Commons. https://commons.wikimedia.org/wiki/File:Diverging_Lat_Pulldown_Machine.jpg

Terven, J., Córdova-Esparza, D.-M., & Romero-González, J.-A. (2023). A comprehensive review of YOLO architectures in computer vision: From YOLOv1 to YOLOv8 and YOLO-NAS. *Machine Learning and Knowledge Extraction, 5*(4), 1680–1716. https://doi.org/10.3390/make5040083

Tomás, J. (2019). *El gran libro de Android* (7.ª ed.). Alfaomega.

U.S. Department of Agriculture, Agricultural Research Service. (s. f.). *FoodData Central*. https://fdc.nal.usda.gov/

U.S. Department of Agriculture, Agricultural Research Service. (2024). *FoodData Central Foundation Foods documentation*. https://fdc.nal.usda.gov/docs/Foundation_Foods_Documentation_Apr2024.pdf

Ultralytics. (s. f.-a). *Annotation editor* [Documentación de Ultralytics Platform]. https://docs.ultralytics.com/platform/data/annotation

Ultralytics. (s. f.-b). *Dedicated endpoints* [Documentación de Ultralytics Platform]. https://docs.ultralytics.com/platform/deploy/endpoints

Ultralytics. (s. f.-c). *Ultralytics YOLO26* [Documentación de modelos]. https://docs.ultralytics.com/models/yolo26/

Valentin, J., Kowdle, A., Barron, J. T., Wadhwa, N., Dzitsiuk, M., Schoenberg, M., Verma, V., Csaszar, A., Turner, E., Dryanovski, I., Afonso, J., Pascoal, J., Tsotsos, K., Leung, M., Schmidt, M., Guleryuz, O. G., Khamis, S., Tankovich, V., Fanello, S., ... Rhemann, C. (2018). Depth from motion for smartphone AR. *ACM Transactions on Graphics, 37*(6), Artículo 193. https://doi.org/10.1145/3272127.3275041

Viera, U. (s. f.). *Introducción a React Native: crea apps móviles nativas en iOS y Android*. urianviera.com. https://urianviera.com/react-native/introduccion-a-react-native

Vinolo Gil, M. J., Gonzalez-Medina, G., Lucena-Anton, D., Perez-Cabezas, V., Ruiz-Molinero, M. C., & Martín-Valero, R. (2021). Augmented reality in physical therapy: Systematic review and meta-analysis. *JMIR Serious Games, 9*(4), Artículo e30985. https://doi.org/10.2196/30985

wger Project. (s. f.). *wger Workout Manager: REST API*. https://wger.de/en/software/api

WorkoutX. (s. f.). *WorkoutX API: exercise database and GIF animations*. https://workoutxapp.com/

Yépez, M. D. (2017). *Club Ensayos*. https://www.clubensayos.com/Negocios/Proyecto-de-grado-Creaci%C3%B3n-de-un-centro-de/4574087.html

Yuhonas. (s. f.). *free-exercise-db: open public domain exercise dataset*. GitHub. https://github.com/yuhonas/free-exercise-db

ANEXO A: CUADROS COMPARATIVOS DEL MARCO TEÓRICO

Este anexo reúne los cuadros comparativos que sustentan las decisiones tecnológicas y metodológicas descritas en el Capítulo II.

Metodología	Descripción	Características
Waterfall	La metodología waterfall o cascada, es un proceso secuencial, el proyecto se divide en fases, las cuales deben completarse completamente antes de pasar a la siguiente. El avance del progreso va de arriba hacia abajo.	▪ Enfoque lineal y estructurado. ▪ Requiere una planificación previa. ▪ No permite adaptaciones una vez iniciada la ejecución. ▪ Fácil de entender.
Espiral	Esta metodología integra la gestión de riesgos a lo largo del proceso de desarrollo. El desarrollo ocurre en ciclos o vueltas y cada uno de estos cuenta con 4 fases: planificación, análisis de riesgos, ingeniería y evaluación por parte del cliente.	▪ Enfoque iterativo basado en ciclos sucesivos. ▪ Permite incorporar ajustes o mejoras en cada iteración. ▪ Contempla análisis de riesgos periódico.
Incremental	Esta metodología combina elementos de la metodología cascada, permite desarrollar el producto de forma progresiva, cada etapa incremental constituye una nueva funcionalidad, lo que permite ver resultados de forma más rápida en comparación del modelo cascada.	▪ Entrega parcial del sistema. ▪ Facilita la detección de errores temprana. ▪ Recopila comentarios de clientes después de cada entrega. ▪ Flexible a los cambios. ▪ Combinación de enfoques lineales e iterativos.
Prototipado	Esta metodología desarrolla un modelo preliminar y tangible de software, esto permite que el usuario pruebe y de retroalimentación, para así poder ajustar los requisitos y mejorar el diseño del sistema final.	▪ Facilita la comunicación con el usuario. ▪ Reduce la incertidumbre. ▪ Es iterativa. ▪ Reduce riesgos y costos. ▪ Es flexible y versátil.
Diseño rápido de aplicaciones RAD	Esta metodología permite desarrollar software de alta calidad en un corto periodo de tiempo. Los costes son mucho más altos y el desarrollo es flexible, aunque requiere mayor intervención de los usuarios.	▪ Desarrollo basado en prototipos funcionales. ▪ Iteraciones cortas y desarrollo incremental. ▪ Alta participación del usuario.

Tabla A-1. Comparación de Metodologías tradicionales de desarrollo de software. *Fuente:* Elaboración propia.

Conclusión: De acuerdo al análisis comparativo presentado en la Tabla A-1, se concluye que la principal diferencia entre las metodologías tradicionales radica en su tolerancia al cambio y la gestión de riesgo. Mientras que el modelo cascada prioriza una estructura secuencial y rígida, los modelos espiral, incremental y prototipado introducen cambios iterativos que permiten la validación progresiva del usuario. Por lo tanto, estas metodologías son más aptas para proyectos con requerimientos claros, estables que no cambiaran con el tiempo.

Metodología	Descripción	Características
Scrum	Es una metodología ágil, que divide los requerimientos y tareas de forma similar a Kanban. Está compuesto por iteraciones fijas y cortas denominadas Sprints, así como una estructura basada en roles, eventos y artefactos definidos. Su objetivo es entregar incrementos funcionales del producto en ciclos de 2 a 4 semanas, fomentando la colaboración y la inspección continua (Schwaber & Sutherland, 2020).	▪ Roles definidos (Product Owner, Scrum Master, Equipo). ▪ Iteraciones cortas y fijas. ▪ Entregas incrementales al final de cada sprint. ▪ Alta colaboración con el cliente. ▪ Uso de backlog y tableros para la gestión visual del trabajo. (Schwaber & Sutherland, 2020).
Extreme Programming (XP)	Es una metodología basada en relaciones interpersonales, la colaboración y la retroalimentación continua. Su objetivo principal es crear un entorno de trabajo basado en la comunicación constante, simplicidad del diseño y participación activa del cliente. Incorpora prácticas de ingeniería como pair programming, test-driven development, refactorización constante e integración continua (Beck & Andres, 2005).	▪ trabajo colaborativo. ▪ Diseño sencillo y adaptables. ▪ Desarrollo guiado por pruebas. ▪ Refactorización continua. ▪ Integración continua. ▪ Entregas frecuentes y pequeñas iteraciones. ▪ Planificación basada en historias de usuario. ▪ Programación en parejas.
Mobile-D	Es una metodología ágil para el desarrollo de aplicaciones móviles para equipos pequeños, está centrada en ciclos de desarrollo rápidos y entrega continua. Combina prácticas ágiles con procesos orientados al hardware y ciclos muy cortos de entrega (Abrahamsson et al., 2004).	▪ Diseñada para desarrollo móvil. ▪ Iteraciones rápidas. ▪ Adaptación a restricciones de hardware. ▪ Enfocado en grupos pequeños.
Kanban	Es una metodología de trabajo inventada por la empresa de automóviles Toyota. Consiste en dividir las tareas en porciones mínimas y organizarlas en un tablero de trabajo dividido en tareas pendientes, en curso y finalizadas. De esta forma se crea un flujo de trabajo muy visual basado en tareas prioritarias e incrementando el valor del producto (Santander Open Academy, 2020).	▪ Visualización del trabajo. ▪ Límite de trabajo en curso. ▪ Mejora continua. ▪ No requiere iteraciones fijas. ▪ Fácil implementación en equipos individuales. ▪ Gestión visual del trabajo.

Tabla A-2. Comparación de metodologías ágiles móviles. *Fuente:* Elaboración propia a partir de Abrahamsson et al. (2004), Beck y Andres (2005), Santander Open Academy (2020) y Schwaber y Sutherland (2020).

Conclusión: De acuerdo con la Tabla A-2, todas las metodologías ágiles comparten principios como la flexibilidad y la entrega continua, sin embargo, la metodología Mobile-D es la más adecuada para proyectos móviles por su rapidez, sus iteraciones cortas y su adaptación a las particularidades del hardware.

Características	Android	iOS
Propietario	Google	Apple
Familia S.O.	Linux	UNIX
Tipo de licencia	Código abierto	Código cerrado y propietario
Programado en	C, C++, Java	Swift, Objective-C, C, C++

Características	Android	iOS
Lenguajes de programación	Java, Kotlin	Swift
Entorno de desarrollo	Android Studio	Xcode
Arquitectura del sistema	Basada en kernel Linux + HAL + Runtime ART	Basada en kernel XNU + frameworks de alto nivel
Presencia en el mercado boliviano	90.74 % (StatCounter Global Stats, 2025)	8.7 % (StatCounter Global Stats, 2025)
Diseño	Diseños flexibles, personalizado, colorido, adaptable a muchas marcas y dispositivos.	Diseño minimalista, elegante, limpio y extremadamente consistente.
Personalización	Permite la personalización de la interfaz mediante launchers personalizados, permite utilizar paquetes de temas, ajustar el panel de configuración rápida y modificar el comportamiento de aplicaciones predeterminadas (Gironés, 2012).	Su personalización es limitada, prioriza la simplicidad, estabilidad y coherencia visual, lo que reduce riesgos de incompatibilidad (Apple Inc., 2023).
Fragmentación	Samsung, Huawei, Xiaomi, Motorola, Sony.	Exclusivo para dispositivos Apple (iPhone, iPad).
Distribución de Apps	Play Store y permite instalación manual (APK/Sideloading).	App Store
Asistente Virtual	Google Now, Google Assistant	Siri

Tabla A-3. Comparación de Android vs iOS. *Fuente:* Elaboración propia.

Conclusión: De acuerdo con la Tabla A-3, Android destaca por su capacidad de adaptabilidad, y su compatibilidad con distintos lenguajes de programación, lo que lo hace más versátil; su flexibilidad y escalabilidad lo hacen la plataforma idónea para el desarrollo de una aplicación móvil.

Framework	Descripción	Características	Ventajas
React Native	Es un framework de Meta para construir aplicaciones nativas para iOS y Android, usando JavaScript. Este framework usa el paradigma fundamental de construcción de bloques de UI (componentes visuales con los que interacciona el usuario) (Deloitte, 2019).	▪ Multiplataforma. ▪ Fast Refresh y Hot Reload. ▪ Acceso a componentes y APIs nativas. ▪ Gran comunidad y ecosistema amplio. ▪ Integración con librerías como Redux, Axios, Firebase.	Alto rendimiento. Reutilización de código. Amplia compatibilidad.
Flutter	Es un framework de código abierto de Google basado en Dart con motor de render propio (Flutter, s. f.).	▪ Widgets personalizables. ▪ Animaciones. ▪ Hot Reload.	UI consistente. Excelente rendimiento gráfico.

Framework	Descripción	Características	Ventajas
Ionic	Ionic es un framework híbrido basado en tecnologías web (HTML, CSS, JavaScript) que funciona mediante WebView. Permite construir aplicaciones con enfoques multiplataforma usando Angular, React o Vue. Es especialmente útil para aplicaciones orientadas a contenidos web o prototipado rápido (Ionic, 2023).	▪ Multi-framework. ▪ Amplia colección de componentes UI. ▪ Soporte para Progressive Web Apps.	Desarrollo rápido. Curva de aprendizaje baja.
Xamarin	Xamarin es un framework de Microsoft basado en C# y .NET, que compila aplicaciones nativas para Android, iOS y Windows. Permite compartir hasta 90 % de la lógica de negocio entre plataformas y se integra profundamente con Visual Studio. Es utilizado ampliamente en entornos empresariales (Microsoft, 2022).	▪ Acceso directo a APIs nativas. ▪ Integración con Visual Studio. ▪ Integración con .NET MAUI.	Sólido para apps empresariales. Reutilización de lógica.
Jetpack Compose	Jetpack Compose es el framework declarativo moderno de Android, basado en Kotlin. Permite construir interfaces de usuario mediante funciones composables, eliminando XML y ofreciendo mayor control del sistema y rendimiento nativo. Es el estándar actual para desarrollar aplicaciones Android modernas (Android Developers, 2024).	▪ UI declarativa. ▪ Integración con Android Jetpack. ▪ Animaciones avanzadas.	Rendimiento máximo. Control total del sistema.

Tabla A-4. Comparación de frameworks de desarrollo Mobile. *Fuente:* Elaboración propia a partir de Android Developers (2024), Deloitte (2019), Flutter (s. f.) y Microsoft (2022).

Conclusión: De acuerdo con la Tabla A-4, React Native destaca frente a los demás frameworks debido a su equilibrio entre rendimiento y desarrollo multiplataforma, su capacidad de generar componentes nativos le da mayor fluidez a diferencia de Ionic y Flutter. Así también, permite desarrollar aplicaciones de forma eficiente y se adapta al entorno de Android.

Base de Datos	Descripción	Características	Ventajas
Firebase Firestore	Es una base de datos NoSQL en la nube que permite sincronización en tiempo real y almacenamiento distribuido (Isaac, 2025). Su base de datos principal, Cloud Firestore, es una solución NoSQL escalable diseñada para sincronización en tiempo real, rendimiento móvil y arquitectura serverless.	▪ Escalable. ▪ Documentos y colecciones. ▪ Soporte offline.	▪ Fácil integración móvil. ▪ Sin servidor. ▪ Arquitectura serverless. ▪ Integración con el ecosistema de Google.

Base de Datos	Descripción	Características	Ventajas
SQLite	Es una biblioteca en lenguaje C, es el motor de base de datos más utilizado en el mundo. Se encuentra integrado en todos los teléfonos móviles y en la mayoría de las computadoras, es de código abierto (SQLite Consortium, 2024).	▪ Escalable. ▪ Documentos y colecciones. ▪ Soporte offline.	▪ Fácil integración móvil. ▪ Ligera y de bajo consumo. ▪ Amplio soporte y estabilidad. ▪ Sin servidor. ▪ Tiempo real.
MongoDB Atlas	Es un servicio de base de datos multicloud, simplifica la implementación y gestión de bases de datos. Permite manejar grandes volúmenes de datos, consultas avanzadas y procesamiento en la nube con altos niveles de seguridad y escalabilidad (Huang, 2020; MongoDB, Inc., 2024).	▪ Ligero. ▪ Transaccional. ▪ Estructura relacional.	▪ Ideal para uso offline. ▪ Ampliamente soportado. ▪ Integración nativa con la nube. ▪ Funciones avanzadas de seguridad.
MySQL	MySQL es un sistema de gestión de bases de datos relacional (RDBMS) de código abierto ampliamente utilizado en aplicaciones web y móviles. Originalmente desarrollado por MySQL AB y actualmente mantenido por Oracle Corporation, MySQL se ha consolidado como una solución flexible, rápida y confiable para gestionar datos estructurados (DuBois, 2013; Oracle Corporation, 2024).	▪ Lenguaje SQL para definición y manipulación de datos. ▪ Soporte para múltiples motores de almacenamiento. ▪ Ejecución de transacciones ACID. ▪ Alta compatibilidad con lenguajes de programación.	▪ Flexible. ▪ Escalable. ▪ Adecuada para grandes volúmenes.
PostgreSQL	Sistema de gestión de bases relacional ampliamente utilizado. Destaca por su arquitectura orientada a objetos, su capacidad para manejar grandes volúmenes de datos y su soporte para consultas complejas, extensiones y tipos de datos personalizados.	▪ SQL. ▪ Robusto. ▪ Seguro. ▪ Gestión avanzada de concurrencia. ▪ Soporte para JSON y JSONB para datos semiestructurados.	▪ Ideal para Backend estructurado. ▪ Rendimiento estable. ▪ Perfecto para aplicaciones móviles.

Tabla A-5. Comparación de bases de datos para aplicaciones móviles. *Fuente:* Elaboración propia a partir de Huang (2020), MongoDB, Inc. (2024) y Oracle Corporation (2024).

Conclusión: De acuerdo con la Tabla A-5, Firebase Firestore es la mejor alternativa, por su arquitectura NoSQL, reduce la complejidad del Backend y es escalable.

ANEXO B: REGLAS DE SEGURIDAD DE FIREBASE

Este anexo reproduce los archivos de reglas de seguridad vigentes en el repositorio del proyecto al 27 de octubre de 2026 y resume las funciones del servidor que complementan la protección de los datos.

B.1. Reglas de Cloud Firestore

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    function autenticado() { return request.auth != null; }
    function dueno(uid) { return autenticado() && request.auth.uid == uid; }
    function texto(v, min, max) { return v is string && v.size() >= min && v.size() <= max; }
    function numero(v, min, max) { return v is number && v >= min && v <= max; }
    function opcNumero(d, k, min, max) { return !(k in d) || numero(d[k], min, max); }
    function opcTexto(d, k, max) { return !(k in d) || texto(d[k], 0, max); }
    function creado() {
      return request.resource.data.creadoEn == request.time
        && !('actualizadoEn' in request.resource.data);
    }
    function actualizado() {
      return request.resource.data.creadoEn == resource.data.creadoEn
        && request.resource.data.actualizadoEn == request.time;
    }
    function listadoAcotado() { return request.query.limit != null && request.query.limit <= 500; }
    match /{catalogo}/{id} {
      allow get: if autenticado() && catalogo in ['maquinas', 'ejercicios', 'ejerciciosWger'];
      allow list: if autenticado() && catalogo in ['maquinas', 'ejercicios', 'ejerciciosWger'] && listadoAcotado();
    }
    match /usuarios/{uid} {
      function perfil(d) {
        return d.keys().hasOnly(['nombre', 'apellidos', 'email', 'genero', 'altura', 'pesoActual', 'edad', '
    diasDisponibles', 'nivel', 'objetivo', 'rutinaActivaId', 'creadoEn', 'actualizadoEn'])
          && texto(d.nombre, 1, 100) && texto(d.email, 3, 254)
          && opcTexto(d, 'apellidos', 100)
          && opcNumero(d, 'altura', 50, 300) && opcNumero(d, 'pesoActual', 10, 500)
          && (!('edad' in d) || (d.edad is int && numero(d.edad, 14, 90)))
          && (!('diasDisponibles' in d) || (d.diasDisponibles is int && numero(d.diasDisponibles, 2, 6)))
          && (!('genero' in d) || d.genero in ['masculino', 'femenino'])
          && (!('nivel' in d) || d.nivel in ['principiante', 'intermedio', 'avanzado'])
          && (!('objetivo' in d) || d.objetivo in ['perder_peso', 'ganar_masa', 'mantenimiento', 'resistencia'])
          && (!('rutinaActivaId' in d) || d.rutinaActivaId == null ||
            (texto(d.rutinaActivaId, 1, 128) && d.rutinaActivaId.matches('^[^/]+$') &&
              exists(/databases/$(database)/documents/usuarios/$(uid)/rutinas/$(d.rutinaActivaId))));
      }
      allow get, delete: if dueno(uid);
      allow create: if dueno(uid) && perfil(request.resource.data) && creado()
        && request.resource.data.email == request.auth.token.email;
      allow update: if dueno(uid) && perfil(request.resource.data) && actualizado()
        && request.resource.data.email == resource.data.email;

      // Nested plans are validated in the authenticated guardarRutina function.
      // Admin SDK bypasses rules; the function derives UID exclusively from Auth.
      match /rutinas/{id} {
        allow get, delete: if dueno(uid);
        allow list: if dueno(uid) && listadoAcotado();
        allow create, update: if false;
      }
      match /registrosNutricionales/{id} {
        function imc(d) {
          return d.keys().hasOnly(['usuarioId', 'weight', 'height', 'bmi', 'category', 'creadoEn', 'actualizadoEn'])
            && numero(d.weight, 10, 500) && numero(d.height, 50, 300)
            && numero(d.bmi, 1, 100) && d.category in ['BAJO', 'NORMALIDAD', 'SOBREPESO', 'OBESIDAD'];
        }
        function ingesta(d) {
          return d.keys().hasOnly(['usuarioId', 'alimento', 'calorias', 'cantidad', 'proteinas', 'carbohidratos', '
      grasas', 'fdcId', 'creadoEn', 'actualizadoEn'])
            && texto(d.alimento, 1, 200) && numero(d.calorias, 0, 10000)
            && opcTexto(d, 'cantidad', 100) && opcNumero(d, 'proteinas', 0, 1000)
            && opcNumero(d, 'carbohidratos', 0, 1000) && opcNumero(d, 'grasas', 0, 1000)
            && (!('fdcId' in d) || (d.fdcId is int && numero(d.fdcId, 1, 2147483647)));
        }
        function valido(d) {
          return d.usuarioId == uid && (('weight' in d && imc(d)) || ('alimento' in d && ingesta(d)));
        }
        allow get, delete: if dueno(uid);
        allow list: if dueno(uid) && listadoAcotado();
        allow create: if dueno(uid) && valido(request.resource.data) && creado();
        allow update: if dueno(uid) && valido(request.resource.data) && actualizado();
      }
      match /progresoFisico/{id} {
        function valido(d) {
          return d.keys().hasOnly(['usuarioId', 'peso', 'altura', 'imc', 'notas', 'creadoEn', 'actualizadoEn'])
            && d.usuarioId == uid && numero(d.peso, 10, 500)
            && opcNumero(d, 'altura', 50, 300) && opcNumero(d, 'imc', 0.01, 2000)
            && opcTexto(d, 'notas', 500);
        }
        allow get, delete: if dueno(uid);
        allow list: if dueno(uid) && listadoAcotado();
        allow create: if dueno(uid) && valido(request.resource.data) && creado();
        allow update: if dueno(uid) && valido(request.resource.data) && actualizado();
```

```
      }
      match /entrenamientos/{id} {
        function valido(d) {
          return d.keys().hasOnly(['usuarioId', 'rutinaId', 'dia', 'fecha', 'creadoEn', 'actualizadoEn'])
            && d.usuarioId == uid && texto(d.rutinaId, 1, 128) && d.rutinaId.matches('^[^/]+$')
            && d.dia is int && numero(d.dia, 1, 7)
            && d.fecha == id && d.fecha.matches('^[0-9]{4}-(0[1-9]|1[0-2])-(0[1-9]|[12][0-9]|3[01])$')
            && exists(/databases/$(database)/documents/usuarios/$(uid)/rutinas/$(d.rutinaId));
        }
        allow get, delete: if dueno(uid);
        allow list: if dueno(uid) && listadoAcotado();
        allow create: if dueno(uid) && valido(request.resource.data) && creado();
        allow update: if dueno(uid) && valido(request.resource.data) && actualizado();
      }
    }
    // Default deny also protects server quota documents and unknown collections.
    match /{document=**} { allow read, write: if false; }
  }
}
```

Código B.1. Archivo `firestore.rules`

B.2. Reglas de Cloud Storage

```
rules_version = '2';
service firebase.storage {
  match /b/{bucket}/o {
    match /maquinas/{path=**} { allow read: if request.auth != null; }
    match /ejercicios/{path=**} { allow read: if request.auth != null; }
    match /{path=**} { allow read, write: if false; }
  }
}
```

Código B.2. Archivo `storage.rules`

B.3. Funciones del servidor

La Tabla B-1 resume las dos funciones *callable* de Cloud Functions que acompañan a las reglas. Ambas se despliegan en la región `us-central1` con un máximo de cinco instancias, exigen la verificación de App Check y rechazan a los usuarios anónimos. Las cuotas se registran en documentos de la colección `_cuotas`, identificados por un código *hash* y de lectura y escritura reservadas al servidor.

Tabla B-1. Funciones del servidor y controles aplicados

Función	Propósito	Cuotas
`buscarAlimentos`	Intermediaria entre la aplicación y USDA FoodData Central; la clave de la API se guarda como secreto del servidor y se valida la consulta y la respuesta.	15 por minuto y 120 por hora por usuario; 800 por hora en total
`guardarRutina`	Única vía para crear o actualizar documentos de `usuarios/{uid}/rutinas`; valida el plan completo, toma el propietario de la identidad autenticada y conserva las fechas del servidor.	20 por minuto y 200 por hora por usuario

Fuente: Archivos `functions/src/index.mjs` y `functions/src/quota.mjs` del repositorio.

ANEXO C: CASOS DE PRUEBA FUNCIONALES

Los casos de prueba se ejecutan sobre el *development build* instalado en el dispositivo de prueba (Infinix HOT 50 Pro+, Tabla 4-35), con una cuenta de prueba y el catálogo de máquinas cargado en Firestore. En la columna *Resultado* se registra si el caso pasa o falla.

Tabla C-1. Casos de prueba funcionales

ID	RF	Pasos	Resultado esperado	Resultado
Autenticación, onboarding y perfil				
CP-01	1.1	Registrarse con nombre, apellido, correo válido y contraseña de 6 o más caracteres.	Se crea la cuenta y el perfil; se abre el *onboarding*.	☐ Pasa ☐ Falla
CP-02	1.1	Registrarse con correo inválido, contraseñas distintas o de menos de 6 caracteres.	Se muestra el mensaje de validación; no se crea la cuenta.	☐ Pasa ☐ Falla
CP-03	1.2	Iniciar sesión con correo y contraseña correctos, y luego con una contraseña incorrecta.	El primero ingresa; el segundo muestra un mensaje de error.	☐ Pasa ☐ Falla
CP-04	1.2	Iniciar sesión con Google por primera vez; cerrar sesión y volver a ingresar.	El perfil se crea solo la primera vez; no se sobrescribe.	☐ Pasa ☐ Falla
CP-05	1.3	Solicitar la recuperación de contraseña desde el inicio de sesión.	Llega el correo de restablecimiento.	☐ Pasa ☐ Falla
CP-06	1.4	Ingresar con un usuario sin perfil completo.	Se muestra el *onboarding*; las pestañas de la app no están disponibles.	☐ Pasa ☐ Falla
CP-07	1.4	En datos físicos, ingresar edad 13 o estatura 20 cm.	Se muestra el mensaje de rango; no se avanza.	☐ Pasa ☐ Falla
CP-08	1.4	Completar objetivo, nivel y datos físicos y elegir “Generar con IA”.	Se guardan los datos; se abre la generación de rutina.	☐ Pasa ☐ Falla
CP-09	1.5	Editar el objetivo y el peso en el perfil; cerrar sesión.	Los cambios persisten; al cerrar sesión se vuelve a la bienvenida.	☐ Pasa ☐ Falla
Guía de máquinas y ejercicios				
CP-10	2.1	Abrir la pestaña Máquinas.	Se muestran las 35 máquinas en cuadrícula.	☐ Pasa ☐ Falla
CP-11	2.1	Buscar “biceps” y luego “bíceps”.	Ambas búsquedas devuelven los mismos resultados.	☐ Pasa ☐ Falla
CP-12	2.1	Aplicar un filtro de grupo muscular y uno de tipo.	Solo se muestran las máquinas que cumplen ambos filtros.	☐ Pasa ☐ Falla
CP-13	2.2	Abrir la ficha de la prensa de piernas.	Se ven descripción, músculos, guía de uso, errores, seguridad y variantes.	☐ Pasa ☐ Falla
CP-14	2.3	En la ficha, revisar la sección de ejercicios.	Cada ejercicio muestra su animación, músculo objetivo y dificultad.	☐ Pasa ☐ Falla
CP-15	2.2	Revisar la galería de variantes de una ficha.	Se muestran hasta ocho modelos comerciales.	☐ Pasa ☐ Falla

ID	RF	Pasos	Resultado esperado	Resultado
Escáner con superposición de información				
CP-16	2.4	Abrir el escáner y negar el permiso de la cámara.	Se muestra el aviso del permiso; no se activa la cámara.	☐ Pasa ☐ Falla
CP-17	2.4	Conceder el permiso y capturar una máquina.	Pasa por “analizando”; si la confianza es de al menos 0,8 la máquina queda detectada, y si no, se abre la lista con la sugerencia preseleccionada para confirmarla.	☐ Pasa ☐ Falla
CP-18	2.4	En la lista, buscar y elegir una máquina.	La ficha resumida (nombre, tipo, músculos y ejercicios) se superpone sobre la cámara.	☐ Pasa ☐ Falla
CP-19	2.4	Abrir el escáner desde una ficha; tocar “escanear otra”.	La máquina aparece directamente; luego se reinicia el escaneo.	☐ Pasa ☐ Falla
Rutinas y planificación				
CP-20	3.1	Con objetivo “ganar masa” y nivel principiante, generar una rutina.	Rutina de 3 días (empuje, tirón, pierna), 4 series de 9 repeticiones, descanso 90 s.	☐ Pasa ☐ Falla
CP-21	3.1	Con objetivo “perder peso” y nivel intermedio, generar una rutina.	Rutina de 4 días con un día de cardio, 3 series de 15 repeticiones.	☐ Pasa ☐ Falla
CP-22	3.2	Crear una rutina personalizada de dos días con ejercicios.	La rutina se guarda y queda activa.	☐ Pasa ☐ Falla
CP-23	3.2	Intentar guardar sin nombre, o con un día sin ejercicios.	No se guarda; se muestra el motivo.	☐ Pasa ☐ Falla
CP-24	3.3	Desde una ficha, agregar un ejercicio a un día; repetir la acción.	La primera vez se agrega; la segunda se informa el duplicado.	☐ Pasa ☐ Falla
CP-25	3.3	Activar otra rutina y eliminar una.	La activa se muestra en el inicio; la eliminada desaparece.	☐ Pasa ☐ Falla
CP-26	3.3	Tocar un ejercicio de la rutina asociado a una máquina.	Se abre la ficha de la máquina.	☐ Pasa ☐ Falla
Nutrición: alimentos e IMC				
CP-27	4.1	Calcular el IMC con 70 kg y 175 cm.	IMC 22.9, categoría normal.	☐ Pasa ☐ Falla
CP-28	4.1	Calcular con peso 5 kg o estatura 20 cm.	Mensaje de validación; no se calcula.	☐ Pasa ☐ Falla
CP-29	4.1	Realizar un cálculo y revisar el historial.	El cálculo queda registrado.	☐ Pasa ☐ Falla
CP-30	5.1	Buscar “pollo”, elegir un resultado e ingresar 150 g.	Se listan alimentos (consulta mediante la función `buscarAlimentos`); la vista previa y el registro muestran los mismos valores.	☐ Pasa ☐ Falla
CP-31	5.2	Tocar un alimento rápido (por ejemplo, “Huevo”).	Se agrega al día con sus calorías y macronutrientes.	☐ Pasa ☐ Falla
CP-32	5.2	Ingresar un alimento manualmente con nombre y calorías.	Se agrega al día.	☐ Pasa ☐ Falla
CP-33	5.3	Revisar el resumen del día y eliminar un registro.	Las calorías y macros se actualizan; el registro desaparece.	☐ Pasa ☐ Falla

ID	RF	Pasos	Resultado esperado	Resultado
Progreso, entrenamientos y reportes				
CP-34	4.2	En Progreso, registrar un peso (la estatura aparece precargada).	Se crea el registro con su IMC.	☐ Pasa ☐ Falla
CP-35	4.2	Registrar un peso de 0 o mayor de 500 kg.	Mensaje de validación; no se guarda.	☐ Pasa ☐ Falla
CP-36	6.1	Revisar el gráfico y alternar entre peso e IMC.	Muestra como máximo los ocho últimos registros de la métrica elegida.	☐ Pasa ☐ Falla
CP-37	6.1	Con registros de 80 y 77.5 kg, revisar la variación.	Se muestra -2.5 kg; la categoría corresponde al último IMC.	☐ Pasa ☐ Falla
CP-38	6.2	Marcar un día de la rutina como completado; marcarlo otra vez; desmarcarlo.	Un solo registro por fecha; al desmarcar desaparece.	☐ Pasa ☐ Falla
CP-39	6.2	Marcar entrenamientos en días consecutivos.	Racha, días de la semana (lunes a domingo) y total se actualizan.	☐ Pasa ☐ Falla
CP-40	6.2	Abrir el inicio con una rutina activa.	Muestra la rutina, sus ejercicios, días por semana y registros de peso.	☐ Pasa ☐ Falla
CP-41	6.1	Generar el reporte en PDF para 7 días, 30 días, 90 días y todo el historial, y compartirlo.	Se genera un PDF con perfil, peso, entrenamientos y alimentación frente a la meta; se abre el menú para compartirlo.	Pasa
Reconocimiento en la nube y realidad aumentada (ARCore)				
CP-42	2.4	Con el reconocimiento en la nube activo, capturar una máquina con conexión; repetir sin conexión y con una imagen que no sea una máquina.	Con conexión se proponen hasta tres candidatas con su caja; sin conexión se usa el modelo local sin cerrar el escáner; si no se reconoce ninguna máquina del catálogo se abre la selección manual.	☐ Pasa ☐ Falla
CP-43	2.4	En un teléfono compatible con ARCore, mover lentamente el teléfono, capturar una máquina y confirmar una candidata; caminar de lado y girar el teléfono mientras la imagen se analiza.	La etiqueta con el nombre y la instrucción queda junto al punto colocado, usa la dirección original de la captura y se oculta al salir del encuadre.	☐ Pasa ☐ Falla
CP-44	2.4	Probar una escena sin superficie detectable, “Recolocar información”, “Escanear otra”, volver atrás, pasar a segundo plano, poca luz, cámara denegada y un teléfono sin ARCore.	Sin superficie se pide tocar una junto a la máquina; no reaparecen fichas ni respuestas de capturas anteriores; sin ARCore o sin cámara se ofrece la cámara convencional o el catálogo.	☐ Pasa ☐ Falla